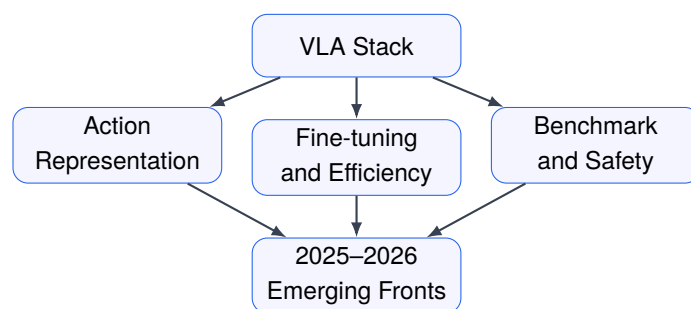


Introduction to Vision-Language-Action Models and Their Trends (2024–2026)

From OpenVLA-Style Fine-Tuning to Efficient, Robust,
and Generalist Robot Policies



Director: Giseop Kim (gsk@dgist.ac.kr)

Draft v0.4, 2026년 5월

Preface

이 책은 VLM을 이미 알고 있는 공학계 독자가 VLA(Vision-Language-Action) 모델의 구조와 2024–2026년 연구 흐름을 수업 수준으로 정리할 수 있도록 쓰는 책이다. 목표는 모델 이름을 나열하는 것이 아니라, VLA가 실제 로봇 정책이 되기 위해 필요한 행동 표현, 학습 절차, 실시간성, 벤치마크, 안전성의 기술적 조건을 정식화하고 비교하는 것이다. 이 책은 exhaustive survey가 아니라 reading-oriented book이며, 각 장의 key papers는 분야 전체를 대표하는 완전 목록이 아니라 해당 기술 병목을 설명하기 위한 anchor이다. v0.4에서는 References와 annotated catalog를 분리하고, appendix/table layout, emerging-front evidence wording, industrial VLA worked example, assignment difficulty tags를 정리했다.

Notation and Terminology

이 책에서 fine-tuning은 pretrained VLA를 특정 로봇, 과제, action space에 맞추는 지도학습 기반 적응을 뜻한다. Post-training은 fine-tuning 이후 success feedback, verifier, preference, correction signal 등을 이용해 실행 품질을 추가로 보정하는 절차를 뜻한다. Action chunking은 여러 timestep의 행동을 한 번에 생성하는 기법이고, action chunk는 그 출력 단위이다. Controller interface는 VLA 출력이 실제 low-level controller에 전달되는 좌표계, 주기, 제약, safety filter를 포함하는 실행 계약이다. 본문에서 2026년 흐름은 확정된 consensus가 아니라 2025–2026년에 관측되는 emerging research fronts로 읽어야 한다.

Contents

I	What is VLA?	1
1	왜 VLA인가: Vision, Language, Action의 결합	2
1.1	장의 목표	2
1.2	VLA를 하나의 정책으로 보는 관점	2
1.3	왜 VLM만으로는 부족한가	3
1.4	VLA 이전의 세 흐름	3
1.4.1	Language-conditioned robot policy	4
1.4.2	Vision-language model	4
1.4.3	Diffusion policy와 trajectory modeling	4
1.5	2024–2026년에 무엇이 바뀌었는가	5
1.6	성능 지표를 어떻게 볼 것인가	5
1.7	VLA 연구를 기술적으로 읽는 기준	6
1.8	왜 manipulation이 중심이 되었는가	7
1.9	효율성이 독립적인 연구 주제가 된 이유	7
1.10	이 책의 관점	7
1.11	데이터 관점에서 본 VLA	8
1.12	Pretraining, Fine-tuning, Post-training의 구분	8
1.13	실패 모드의 분해	9
1.14	VLA 논문의 최소 재현 단위	9
1.15	작업 예제: 컵을 들어 접시에 놓기	10
1.16	수업에서 요구할 산출물	11
1.17	강의에서의 사용 방식	11
1.18	요약	12
2	VLA의 기본 구조	13
2.1	장의 목표	13
2.2	표준 표기법	13

2.3	관측 인코더	14
2.4	언어 인코더와 goal representation	15
2.5	멀티모달 결합	15
2.6	행동 공간	16
2.7	정책 head	17
2.8	학습 목적	17
2.9	Inference loop	18
2.10	메모리와 장기 과제	18
2.11	Safety filter와 controller interface	19
2.12	구조 비교표	19
2.13	요약	20
3	VLA 연구를 읽는 13개 질문	21
3.1	장의 목표	21
3.2	13개 질문의 전체 구조	21
3.3	질문 1-4: 문제 설정을 검증하기	22
3.3.1	배경	22
3.3.2	문제	22
3.3.3	기존 한계	23
3.3.4	목표	23
3.4	질문 5-6: 방법의 기술적 중심 찾기	23
3.4.1	방법	23
3.4.2	핵심 아이디어	23
3.5	질문 7-9: 검증의 공정성 보기	24
3.5.1	검증	24
3.5.2	결과	24
3.5.3	비교	25
3.6	질문 10-12: 의미와 한계를 분리하기	25
3.6.1	의의	25
3.6.2	한계	25
3.6.3	향후 과제	26
3.7	질문 13: 공개 자원 확인	26
3.8	13개 질문을 표로 적용하기	26
3.9	나쁜 요약과 좋은 요약	26
3.10	리뷰와 연구 기획에서의 사용	27
3.11	요약	28

II	Core Bottlenecks	29
4	Action Representation: VLA의 진짜 병목	30
4.1	장의 목표	30
4.2	행동 표현이 어려운 이유	30
4.3	연속 행동 회귀	31
4.4	이산 action token	31
4.5	Action chunking	32
4.6	Trajectory autoregressive modeling	33
4.7	Diffusion action decoder	33
4.8	B-spline과 저차원 행동 기저	34
4.9	행동 표현과 embodiment	34
4.10	평가 기준	34
4.11	요약	35
5	Fine-Tuning과 Post-Training: VLA를 과제에 맞추는 방법	37
5.1	장의 목표	37
5.2	Pretraining과 robot adaptation	37
5.3	Supervised fine-tuning	38
5.4	Full fine-tuning과 frozen backbone	38
5.5	Adapter와 LoRA류 접근	39
5.6	Instruction tuning from understanding to manipulation	39
5.7	Post-training	40
5.8	Verifier와 reward의 위험	40
5.9	데이터 효율과 scaling	40
5.10	평가 기준	41
5.11	요약	41
6	Efficient VLA: 빠르고 작은 VLA 만들기	44
6.1	장의 목표	44
6.2	Latency budget	44
6.3	Token 수가 왜 중요한가	45
6.4	Sparsification과 routing	45
6.5	Adapter로 작게 맞추기	46
6.6	Speculative decoding	46
6.7	Diffusion step 줄이기	46
6.8	Action chunking과 효율성	47
6.9	메모리와 quantization	47
6.10	효율성 평가표	47

6.11 요약	48
III Evaluation in the Physical World	50
7 Manipulation이 VLA의 주전장이 된 이유	51
7.1 Manipulation 문제의 구조	51
7.2 왜 pick-and-place에서 시작했는가	52
7.3 Language grounding과 affordance	52
7.4 Contact dynamics가 만드는 난점	53
7.5 Closed-loop policy가 필요한 이유	53
7.6 Low-level controller와 VLA의 역할 분리	54
7.7 Data 관점: manipulation dataset의 편향	54
7.8 Evaluation: 성공률만으로 충분하지 않다	55
7.9 Long-horizon manipulation	55
7.10 Tactile과 force feedback	56
7.11 Manipulation 논문을 읽는 기준	56
7.12 요약	56
8 Benchmarks and Datasets: VLA 성능은 무엇으로 재는가	58
8.1 Benchmark가 정의하는 문제	58
8.2 Offline dataset과 real-robot evaluation	58
8.3 대표 dataset 계열	59
8.4 Generalization split	59
8.5 Metric: 무엇을 숫자로 만들 것인가	60
8.6 Success rate의 신뢰구간	61
8.7 Benchmark leakage와 overfitting	61
8.8 Simulation benchmark의 역할	62
8.9 Leaderboard보다 중요한 ablation	62
8.10 데이터셋 문서를 읽는 체크리스트	63
8.11 요약	63
9 Reasoning VLA: 로봇은 생각해야 하는가	65
9.1 Reasoning의 공학적 정의	65
9.2 언제 reasoning이 필요한가	65
9.3 Task decomposition	66
9.4 Symbolic plan과 neural policy	66
9.5 Belief와 uncertainty	67
9.6 Self-check와 verifier	67
9.7 Failure diagnosis와 recovery	67

9.8	Language as interface	68
9.9	Chain-of-thought 공개의 문제	68
9.10	Reasoning VLA 평가	69
9.11	요약	70
10	World Models and Simulation: 행동 전에 세계를 예측하기	72
10.1	World model의 기본 형태	72
10.2	왜 VLA에 world model이 필요한가	72
10.3	Video prediction과 action-conditioned prediction	73
10.4	Object-centric world model	73
10.5	Language-conditioned world model	74
10.6	Simulation as data engine	74
10.7	Model predictive control과 VLA	74
10.8	Counterfactual reasoning	75
10.9	World model 평가	75
10.10	요약	76
11	Robustness and Safety	78
11.1	Robustness의 정의	78
11.2	Failure taxonomy	78
11.3	OOD detection	79
11.4	Safety envelope	79
11.5	Calibration	80
11.6	Adversarial and ambiguous instructions	80
11.7	Human intervention	80
11.8	Red teaming for robots	81
11.9	Robustness 평가 설계	81
11.10	요약	82
IV	Generalist and Cross-Domain VLA	84
12	Beyond Manipulation	85
12.1	공통 구조	85
12.2	Navigation VLA	85
12.3	Embodied QA와 embodied instruction following	86
12.4	Autonomous driving과 VLA	86
12.5	Games and interactive agents	87
12.6	Humanoid와 general embodiment	87
12.7	Generalist policy의 의미	88

12.8	Transfer의 조건	89
12.9	요약	89
13	2024–2026 VLA Trend Map	91
13.1	큰 흐름	91
13.2	Trend axes	91
13.3	2024: generalist robot policy의 가능성	92
13.4	2025: fine-tuning과 효율성의 해	92
13.5	Manipulation 중심성	93
13.6	Benchmark와 데이터셋의 정치성	93
13.7	Reasoning과 world model의 재등장	94
13.8	Safety as first-class objective	94
13.9	연구 지형도	94
13.10	요약	95
V	Research Practice	97
14	좋은 VLA 연구 주제를 고르는 법	98
14.1	좋은 주제의 기본 조건	98
14.2	피해야 할 주제 유형	98
14.3	병목에서 시작하기	99
14.4	Contribution type 선택	99
14.5	Baseline을 먼저 설계하기	99
14.6	실험 규모와 현실성	100
14.7	논문 제목으로 바꾸기	101
14.8	13-question framework로 주제 점검	101
14.9	요약	101
15	VLA 논문 읽기 실전	103
15.1	읽기 순서	103
15.2	4-week reading plan	103
15.3	Worked example 1: OpenVLA-OFT 13-question 적용	103
15.4	Worked example 2: WorldVLA/VLA-OS 13-question 적용	104
15.5	Worked example 3: Humanoid and closed industrial VLA evidence split	104
15.6	논문 읽기 노트 템플릿	104
15.7	Assignment difficulty tags	105
15.8	VLA 연구자가 가져가야 할 7개 원칙	105

VI Appendices	108
A Annotated Reading Catalog	109
A.1 Foundation and Open VLA Sources	109
A.2 Emerging and Industrial VLA Sources	109
B Evidence Ladder	112
C Glossary	114

Part I

What is VLA?

Chapter 1

왜 VLA인가: Vision, Language, Action의 결합

1.1 장의 목표

Vision-Language-Action(VLA) 모델은 비전 입력과 자연어 지시를 받아 로봇의 행동을 직접 생성하려는 정책 모델이다. 이 장의 목표는 VLA를 “큰 VLM에 로봇 action head를 붙인 모델” 정도로 축소하지 않고, 공학적으로 어떤 문제를 풀기 위해 등장했는지 정리하는 것이다. 특히 2024–2026년 연구 흐름에서 중요한 변화는 모델 크기 자체보다 행동 표현(action representation), 후처리 학습(post-training), 실시간 추론, 벤치마크, 실패 모드 분석으로 연구의 중심이 이동했다는 점이다.

이 장을 읽은 뒤 독자는 다음 질문에 답할 수 있어야 한다.

1. VLA가 VLM, language-conditioned policy, diffusion policy와 어떻게 다른가?
2. 로봇 정책을 조건부 확률 모델로 쓰면 어떤 입력과 출력이 명확해지는가?
3. 왜 2024–2026년 VLA 연구에서 fine-tuning, action tokenization, efficiency가 동시에 중요해졌는가?
4. VLA 논문을 읽을 때 success rate 하나만 보면 왜 충분하지 않은가?

1.2 VLA를 하나의 정책으로 보는 관점

로봇 제어 문제에서 정책은 관측과 목표를 받아 행동을 내는 함수이다. 전통적인 강화학습 표기에서는 정책을 $\pi(\mathbf{a}_t | s_t)$ 로 쓴다. 그러나 실제 로봇에서 상태 s_t 는 완전히 관측되지 않는다. 카메라 영상, force/torque, joint state, gripper state, 이전 행동, 언어 명령이 뒤섞여 들어온다. VLA 모델은 이 입력을 다음과 같이 정리한다.

$$\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_{\leq t}, \mathbf{h}_{\leq t}, \ell), \quad (1.1)$$

여기서 $\mathbf{o}_{\leq t}$ 는 현재까지의 시각 관측, $\mathbf{h}_{\leq t}$ 는 proprioception과 이전 행동을 포함한 로봇 내부 이력, ℓ 은 자연어 지시, $\mathbf{a}_t$ 는 현재 시점의 로봇 행동이다. 이 표기는 VLA가 단순한 멀티모달 인식 모델이 아니라 언어 조건부 페루프 정책임을 드러낸다. VLM은 이미지와 텍스트의 의미를 맞추는데 강하지만, 로봇 정책은 그 의미를 시간적으로 안정적인 행동으로 바꾸어야 한다.

핵심 정의

이 책에서 VLA 모델은 시각 관측 $\mathbf{o}$, 언어 지시 ℓ , 로봇 이력 $\mathbf{h}$ 를 조건으로 하여 행동 $\mathbf{a}$ 또는 행동 시퀀스를 생성하는 학습 기반 정책 π_θ 를 의미한다. 출력이 실제 로봇 제어 입력으로 연결되지 않으면, 그것은 VLA라기보다 VLM 기반 planner 또는 captioning system에 가깝다.

VLA를 정책으로 보면 논문 평가 기준도 달라진다. 모델이 지시를 잘 설명하는지보다, 지시가 주어진 환경에서 행동을 얼마나 안정적으로 실행하는지가 중요하다. 같은 명령 “빨간 컵을 잡아라”에 대해 VLM은 컵의 위치를 설명할 수 있으면 충분할 수 있다. VLA는 grasp pose, arm trajectory, gripper timing, collision avoidance, 실패 후 recovery까지 포함한 행동을 생성해야 한다. 따라서 VLA의 성능은 언어 이해 성능만으로 환원되지 않는다.

1.3 왜 VLM만으로는 부족한가

VLM은 이미지와 언어의 alignment를 학습한다. 대표적인 입력-출력 구조는 다음 조건부 분포로 나타낼 수 있다.

$$p_\theta(y \mid I, x), \quad (1.2)$$

여기서 I 는 이미지, x 는 텍스트 질문 또는 지시, y 는 텍스트 응답이다. 이 구조는 시각 질의 응답, captioning, grounding에는 강하지만 로봇 제어에는 세 가지 결손이 있다.

첫째, 출력 공간이 행동 공간이 아니다. 로봇은 텍스트 문장을 실행하지 않는다. 실제 실행에는 joint velocity, end-effector pose, gripper command, waypoint, torque 등 구체적인 제어 변수가 필요하다. 둘째, 시간 축이 충분히 모델링되지 않는다. 한 장의 이미지에서 옳은 답을 말하는 것과, 5초 동안 접촉 상태가 바뀌는 물체를 안정적으로 조작하는 것은 다른 문제이다. 셋째, 실패의 비용이 다르다. caption이 조금 틀리는 것과 로봇이 물체를 떨어뜨리거나 충돌하는 것은 공학적으로 다른 실패이다.

따라서 VLA는 VLM의 semantic prior를 활용하되, 그 위에 행동 생성과 제어 안정성의 요구를 추가해야 한다. 이 차이를 무시하면 VLA 연구가 “큰 모델을 붙였더니 잘된다”는 설명으로 흐른다. 수업용 관점에서는 다음 네 가지를 항상 분리해서 봐야 한다.

1. **Perception:** 장면에서 필요한 객체, 관계, affordance를 찾는가?
2. **Language grounding:** 자연어 지시가 어떤 목표 상태와 제약을 의미하는가?
3. **Action generation:** 목표를 어떤 행동 표현으로 변환하는가?
4. **Closed-loop correction:** 실행 중 관측이 바뀔 때 행동을 수정하는가?

VLM은 앞의 두 항목에 강하고, VLA는 네 항목을 하나의 정책으로 묶으려 한다. 이 묶음이 성공하려면 모델 구조뿐 아니라 데이터셋, action representation, inference frequency, robot embodiment가 함께 맞아야 한다.

1.4 VLA 이전의 세 흐름

VLA는 갑자기 등장한 분야가 아니다. 세 흐름이 합쳐진 결과로 보는 것이 정확하다.

1.4.1 Language-conditioned robot policy

첫 번째 흐름은 언어 조건부 로봇 정책이다. 이 계열은 자연어 명령을 task embedding으로 바꾸고, 로봇 정책이 이를 조건으로 행동하도록 학습한다. 장점은 실제 로봇 과제에 가까운 formulation을 갖는다는 점이다. 약점은 언어와 시각의 일반화가 제한적이고, 데이터셋이 특정 로봇이나 환경에 묶이기 쉽다는 점이다.

이 흐름의 기본 학습 목적은 behavior cloning으로 쓸 수 있다.

$$\mathcal{L}_{\text{BC}}(\theta) = - \sum_{i=1}^N \sum_{t=1}^{T_i} \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{o}_{\leq t}^{(i)}, \mathbf{h}_{\leq t}^{(i)}, \ell^{(i)}). \quad (1.3)$$

이 식은 간단하지만 중요한 사실을 보여 준다. VLA는 결국 demonstration data의 분포를 따라 행동을 학습한다. 따라서 데이터가 부족하거나 특정 embodiment에 치우치면 모델이 아무리 커도 일반화가 제한된다.

1.4.2 Vision-language model

두 번째 흐름은 VLM이다. VLM은 대규모 이미지-텍스트 데이터로부터 semantic prior를 얻는다. VLA 연구가 VLM을 끌어온 이유는 로봇 데이터만으로는 세상의 객체, 도구, 관계, 언어 표현을 충분히 학습하기 어렵기 때문이다. 예를 들어 “투명한 컵”, “전자레인지 옆의 그릇”, “오른쪽 손잡이가 있는 서랍” 같은 지시는 로봇 데이터셋보다 웹 규모의 이미지-텍스트 데이터에서 더 풍부하게 나타난다.

그러나 VLM 지식은 곧바로 제어 지식이 되지 않는다. 컵이 무엇인지 아는 것과 컵을 미끄러뜨리지 않고 잡는 것은 다르다. 이 간극이 VLA의 중심 문제이다.

1.4.3 Diffusion policy와 trajectory modeling

세 번째 흐름은 diffusion policy, action chunking, trajectory modeling이다. 로봇 행동은 단일 token보다 연속 궤적에 가깝다. 특히 조작 과제에서는 접촉이 발생하고, 작은 제어 오차가 다음 단계로 누적된다. 이 때문에 행동을 한 step씩 autoregressive하게 내는 방식만으로는 충분하지 않을 수 있다.

행동 chunk를 쓰면 정책은 다음과 같이 H step 행동 시퀀스를 한 번에 생성한다.

$$\mathbf{A}_{t:t+H-1} = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}) \sim \pi_{\theta}(\cdot \mid \mathbf{o}_{\leq t}, \mathbf{h}_{\leq t}, \ell). \quad (1.4)$$

이 방식은 inference call 수를 줄이고 short-horizon consistency를 높일 수 있다. 반대로 환경 변화에 즉각 반응하기 어렵고, chunk 안의 오류가 누적될 수 있다. 따라서 VLA에서 action chunking은 단순 속도 최적화가 아니라 안정성과 반응성 사이의 trade-off 문제이다.

Table 1.1: 2024–2026년 VLA 연구를 읽기 위한 주요 축

축	핵심 질문
행동 표현	연속 행동을 token, chunk, trajectory, diffusion sample 중 무엇으로 표현하는가?
후처리 학습	pretrained VLM/VLA를 특정 로봇 과제에 어떻게 fine-tuning 또는 post-training하는가?
효율성	로봇 제어 주기를 만족할 만큼 inference latency와 계산량을 줄였는가?
조작 성능	contact-rich, bimanual, dexterous task에서 안정적으로 성공하는가?
벤치마크	LIBERO, Open X-Embodiment, RoboTwin 등에서 공정하게 비교되는가?
안전성	분포 변화, 언어 모호성, adversarial instruction, 장기 horizon 실패를 다루는가?

Pre-VLA와 VLA의 구분

SayCan, Diffusion Policy, ACT/ALOHA는 이 책에서 VLA 자체라기보다 선행 흐름으로 분류한다. SayCan은 language-to-skill selection과 affordance 결합을, Diffusion Policy는 continuous action distribution 학습을, ACT/ALOHA는 action chunking과 bimanual imitation learning을 보여 주는 antecedent이다. 반면 RT-2, Octo, OpenVLA, π_0 , OpenVLA-OFT, SmolVLA, GR00T N1, Gemini Robotics 계열은 vision-language representation이 robot action interface로 직접 연결되는 VLA 또는 robot foundation policy로 읽는다.

1.5 2024–2026년에 무엇이 바뀌었는가

2024–2026년 VLA 연구의 변화는 세 문장으로 요약할 수 있다. 첫째, 연구자들은 VLM의 semantic prior를 로봇 정책에 이식하기 시작했다. 둘째, 단순히 큰 모델을 쓰는 것만으로는 success rate, latency, robustness가 충분하지 않다는 점이 드러났다. 셋째, 연구의 중심이 model scaling에서 action representation, fine-tuning protocol, benchmark, safety로 이동했다.

수집한 논문군을 보면 이 변화가 제목 수준에서도 확인된다. action, policy, diffusion, flow, trajectory와 관련된 논문이 가장 큰 묶음을 형성하고, manipulation 논문이 그다음으로 크다. efficiency 관련 논문도 매우 두껍다. 이는 VLA가 이제 “무엇을 이해하는가”보다 “어떻게 행동을 내는가”를 중심으로 재편되고 있음을 의미한다.

이 표의 축은 장식적 분류가 아니다. VLA 논문을 읽을 때 contribution이 어느 축을 건드리는지 바로 표시할 수 있어야 한다. 예를 들어 VLA-Adapter류 논문은 후처리 학습과 효율성에 걸쳐 있고, VQ-VLA류 논문은 행동 표현에 집중한다. RoboTwin류 논문은 모델보다 벤치마크와 데이터 생성이 핵심이다. MemoryVLA나 CogVLA류 논문은 reasoning/planning 축으로 읽어야 한다.

1.6 성능 지표를 어떻게 볼 것인가

VLA 논문에서 가장 자주 보이는 지표는 task success rate이다. M 개의 episode 중 성공한 episode 수를 S 라고 하면 성공률은 다음과 같다.

$$SR = \frac{S}{M}. \quad (1.5)$$

그러나 성공률만으로는 좋은 VLA인지 판단하기 어렵다. 같은 성공률이라도 episode 길이, 환경 다양성, 초기 상태 분포, object set, language template, robot platform이 다르면 난이도가 달라진다. 또한 VLA는 latency가 실제 제어 성능에 직접 영향을 준다. 정책 호출 시간이 τ_{model} 이고 로봇 제어 주기가 τ_{ctrl} 일 때, 단순화하면 페루프 제어가 안정적으로 동작하려면 다음 조건이 필요하다.

$$\tau_{\text{model}} + \tau_{\text{io}} + \tau_{\text{post}} \leq \tau_{\text{ctrl}}, \quad (1.6)$$

여기서 τ_{io} 는 센서 입력과 전처리 시간, τ_{post} 는 action decoding과 safety filtering 시간을 의미한다. 이 부등식이 깨지면 모델이 offline benchmark에서 높게 나와도 실제 로봇에서는 늦게 반응한다. 따라서 2025–2026년에 efficient VLA, speculative decoding, action chunking, lightweight adapter가 중요한 연구 축이 된 것은 자연스러운 흐름이다.

또 하나의 지표는 일반화이다. 학습 task 집합을 $\mathcal{T}_{\text{train}}$, 평가 task 집합을 $\mathcal{T}_{\text{test}}$ 라고 하자. 좋은 VLA는 같은 task의 반복이 아니라, object, scene, instruction, embodiment가 변해도 성능이 유지되어야 한다.

$$\Delta_{\text{gen}} = \mathbb{E}_{\tau \sim \mathcal{T}_{\text{train}}} [\text{SR}(\tau)] - \mathbb{E}_{\tau \sim \mathcal{T}_{\text{test}}} [\text{SR}(\tau)]. \quad (1.7)$$

Δ_{gen} 가 작을수록 train-test gap이 작다. 많은 VLA 논문은 평균 성공률을 강조하지만, 실제로는 어떤 조건에서 gap이 커지는지가 더 중요하다. 예를 들어 unseen object에는 강하지만 unseen instruction에는 약할 수 있고, 시뮬레이션에서는 강하지만 real robot에서는 약할 수 있다.

1.7 VLA 연구를 기술적으로 읽는 기준

이 책에서는 VLA 논문을 다음 순서로 읽는다. 첫째, 입력과 출력의 타입을 확인한다. 둘째, 행동 표현을 확인한다. 셋째, 학습 데이터와 loss를 확인한다. 넷째, inference 절차와 latency를 확인한다. 다섯째, benchmark와 baseline의 공정성을 확인한다. 여섯째, 실패 모드와 공개 자원을 확인한다.

읽기 체크리스트

VLA 논문을 읽을 때 가장 먼저 볼 것은 모델 이름이 아니라 입력, 출력, 학습 신호, 추론 주기이다. 이 네 항목이 불명확하면 contribution이 좋아 보여도 재현성과 실제 로봇 적용 가능성을 판단하기 어렵다.

입력은 이미지 한 장인지, multi-view인지, 비디오인지, proprioception을 포함하는지에 따라 난이도가 달라진다. 출력은 end-effector delta pose인지, joint command인지, gripper open/close인지, waypoint인지, discretized action token인지에 따라 제어 특성이 달라진다. 학습 신호는 demonstration imitation인지, reward feedback인지, preference인지, simulator verification인지에 따라 논문의 주장 범위가 달라진다. 추론 절차는 closed-loop인지 open-loop인지, action chunk를 몇 step 쓰는지, re-planning 주기가 얼마인지 확인해야 한다.

이 기준을 적용하면 같은 VLA라는 이름을 쓰는 논문도 매우 다른 문제를 풀고 있음을 알 수 있다. 어떤 논문은 사실상 VLM 기반 high-level planner이고, 어떤 논문은 low-level visuomotor policy이다. 어떤 논문은 benchmark paper이고, 어떤 논문은 action representation paper이다. 이 구분을 하지 않으면 분야가 빠르게 커질수록 논문들이 모두 비슷해 보인다.

1.8 왜 manipulation이 중심이 되었는가

수집한 논문군에서 manipulation은 가장 두꺼운 응용 축이다. 이유는 명확하다. 조작 과제는 VLA의 장점과 약점을 동시에 드러낸다. 언어 지시가 중요하고, 시각 grounding이 필요하며, 행동은 연속적이고, 접촉과 실패가 자주 발생한다. 따라서 pick-and-place, drawer opening, bimanual manipulation, dexterous grasping, contact-rich task는 VLA 모델을 평가하기 좋은 시험대가 된다.

조작 과제에서는 단순히 목표 객체를 찾는 것만으로 성공하지 않는다. grasp affordance, approach direction, force control, collision, object dynamics가 모두 영향을 준다. 예를 들어 “컵을 들어 올려 접시 위에 놓아라”라는 명령은 객체 인식, 컵 손잡이 위치, gripper pose, lift trajectory, place pose, release timing을 포함한다. VLA가 이 과정을 하나의 정책으로 처리하려면 language grounding과 motion generation 사이의 정보 손실을 줄여야 한다.

이 때문에 2025–2026년에는 dexterous VLA, force-aware VLA, diffusion expert, bimanual benchmark가 많이 등장한다. 이 흐름은 VLA가 단순 semantic policy에서 물리 상호작용 policy로 이동하고 있음을 보여 준다.

1.9 효율성이 독립적인 연구 주제가 된 이유

큰 VLA 모델은 offline evaluation에서는 매력적이지만, 실제 로봇에서는 계산 시간이 곧 제어 문제이다. 모델이 1초에 한 번만 행동을 낸다면 빠르게 변하는 접촉 상황에 대응하기 어렵다. 이 문제를 해결하기 위해 두 방향이 등장했다.

첫째, action chunking 또는 trajectory output을 통해 모델 호출 빈도를 줄인다. 둘째, 모델 내부를 sparsification, routing, adapter, speculative decoding으로 경량화한다. 두 방법은 서로 보완적이지만 같은 문제를 풀지는 않는다. action chunking은 제어 인터페이스를 바꾸는 방법이고, 경량화는 모델 계산을 줄이는 방법이다.

효율성 연구를 평가할 때는 다음 지표를 함께 봐야 한다.

$$J(\theta) = \text{SR}(\theta) - \lambda_1 \cdot \text{Latency}(\theta) - \lambda_2 \cdot \text{Memory}(\theta) - \lambda_3 \cdot \text{FailureRisk}(\theta). \quad (1.8)$$

이 식은 실제 논문들이 반드시 쓰는 objective는 아니지만, 공학적 판단에는 유용하다. 성공률이 조금 높아도 latency가 크고 메모리가 과하면 배포 가치가 낮을 수 있다. 반대로 작은 모델이 성공률을 약간 잃더라도 real-time control을 만족하면 실제 시스템에서는 더 유리할 수 있다.

1.10 이 책의 관점

이 책은 VLA를 낙관적으로 소개하지 않는다. VLA는 매우 유망하지만 아직 해결되지 않은 공학 문제가 많다. 특히 다음 네 가지는 이후 장에서 반복해서 다룬다.

1. **행동 표현**: 로봇 행동을 LLM/VLM이 다룰 수 있는 형식으로 바꾸는 과정에서 정보가 손실될 수 있다.
2. **데이터 분포**: 대규모 robot dataset이라도 embodiment와 task coverage가 제한되면 일반화가 흔들린다.

3. **실시간성**: VLA가 실제 제어 주기를 만족하지 못하면 높은 benchmark score가 실제 성능으로 이어지지 않는다.
4. **검증**: benchmark가 약하면 모델의 발전과 데이터셋 bias를 구분하기 어렵다.

따라서 좋은 VLA 연구는 새로운 모델명을 제안하는 것만으로 충분하지 않다. 어떤 행동 표현을 선택했는지, 왜 그 표현이 더 안정적인지, 어떤 데이터와 baseline으로 검증했는지, latency와 실패 모드를 어떻게 보고했는지 설명해야 한다.

1.11 데이터 관점에서 본 VLA

VLA의 성능은 모델 구조만으로 결정되지 않는다. 실제로는 데이터 분포가 모델의 행동 가능 공간을 강하게 제한한다. demonstration dataset을

$$\mathcal{D} = \{(\mathbf{o}_{1:T_i}^{(i)}, \mathbf{h}_{1:T_i}^{(i)}, \ell^{(i)}, \mathbf{a}_{1:T_i}^{(i)})\}_{i=1}^N \quad (1.9)$$

라고 하자. 학습된 정책은 $\mathcal{D}$ 가 포함한 객체, 장면, 언어 표현, 로봇 kinematics, 인간 demonstrator의 습관을 모두 반영한다. 따라서 VLA 논문에서 “large-scale robot data”라는 표현을 보더라도 다음 네 가지를 따로 확인해야 한다.

1. **Embodiment coverage**: 하나의 로봇팔인지, 여러 embodiment인지, mobile manipulator나 humanoid까지 포함하는지.
2. **Task coverage**: 단순 pick-and-place인지, tool use, contact-rich manipulation, long-horizon task 까지 포함하는지.
3. **Language coverage**: template instruction인지, 사람이 자유롭게 쓴 지시인지, ambiguity가 포함되는지.
4. **Failure coverage**: 성공 demonstration만 있는지, 실패와 recovery trajectory도 있는지.

특히 실패 데이터의 부재는 실제 배포에서 큰 문제가 된다. behavior cloning은 expert trajectory를 모방하는 데 강하지만, policy가 expert distribution 밖으로 벗어났을 때 복구하는 방법을 자동으로 배우지는 못한다. 이를 distribution shift로 쓰면, 학습 데이터 분포 $p_{\mathcal{D}}(\mathbf{o}, \mathbf{h}, \ell)$ 와 배포 환경 분포 $p_{\text{deploy}}(\mathbf{o}, \mathbf{h}, \ell)$ 사이의 차이가 커질수록 성능이 떨어진다.

$$\epsilon_{\text{shift}} = d(p_{\mathcal{D}}(\mathbf{o}, \mathbf{h}, \ell), p_{\text{deploy}}(\mathbf{o}, \mathbf{h}, \ell)), \quad (1.10)$$

여기서 $d(\cdot, \cdot)$ 는 KL divergence, Wasserstein distance, 또는 실험적으로 정의한 domain gap metric일 수 있다. 중요한 것은 어떤 거리 함수를 쓰느냐보다, 논문이 train distribution과 deployment distribution의 차이를 얼마나 정직하게 다루는가이다.

1.12 Pretraining, Fine-tuning, Post-training의 구분

VLA 연구를 읽을 때 pretraining, fine-tuning, post-training을 섞어 쓰면 논문 주장이 흐려진다. 이 책에서는 다음처럼 구분한다.

Table 1.2: VLA 학습 단계의 구분

단계	공학적 의미
Pretraining	웹 이미지-텍스트, 로봇 trajectory, 멀티모달 데이터에서 일반 표현과 기본 행동 prior를 얻는 단계이다.
Fine-tuning	특정 robot embodiment, action space, task family에 맞추어 파라미터 전체 또는 일부를 조정하는 단계이다.
Adapter tuning	backbone은 대부분 고정하고 작은 모듈만 학습하여 비용과 catastrophic forgetting을 줄이는 단계이다.
Post-training	instruction following, reward feedback, simulator verification, human preference 등을 이용해 실행 품질을 보정하는 단계이다.
Deployment calibration	실제 로봇 주기, safety filter, controller interface, sensor delay에 맞추어 policy를 시스템에 연결하는 단계이다.

OpenVLA-OFT류의 문제의식은 이 구분에서 fine-tuning과 deployment calibration 사이에 위치한다. 즉, pretrained VLA가 이미 의미 있는 prior를 갖고 있더라도, 실제 task success와 inference speed를 동시에 만족하려면 action head, training recipe, inference path를 다시 설계해야 한다. 이 관점에서는 VLA의 발전을 단순히 parameter scaling으로 설명할 수 없다.

학습 목적도 하나로 고정되지 않는다. behavior cloning loss에 action regularization이나 temporal smoothness를 추가할 수 있다.

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{BC}}(\theta) + \alpha \sum_{t=2}^T \|\hat{\mathbf{a}}_t - \hat{\mathbf{a}}_{t-1}\|_2^2 + \beta \mathcal{L}_{\text{aux}}(\theta). \quad (1.11)$$

두 번째 항은 행동이 시점마다 불안정하게 흔들리는 것을 줄이는 smoothness term이고, $\mathcal{L}_{\text{aux}}$ 는 language grounding, object detection, affordance prediction, value estimation 같은 보조 손실일 수 있다. 논문을 읽을 때는 정확히 어떤 loss가 success rate 향상에 기여했는지 ablation이 있는지 확인해야 한다.

1.13 실패 모드의 분해

VLA 실패는 하나의 숫자로 요약하기 어렵다. 실패 원인을 분해하지 않으면 모델 개선 방향도 모호해진다. 예를 들어 success rate가 60%에서 70%로 올랐다고 해도, 실패가 perception에서 줄었는지, grasp planning에서 줄었는지, language ambiguity에서 줄었는지 알 수 없다.

이 분해는 논문 리뷰에도 직접 쓸 수 있다. 강한 VLA 논문은 평균 성능뿐 아니라 실패 위치를 보여 준다. 약한 논문은 성공률 표만 제시하고, 왜 실패했는지 거의 말하지 않는다. 특히 실제 로봇 배포를 주장하는 논문이라면 safety failure와 recovery failure를 따로 보고해야 한다.

1.14 VLA 논문의 최소 재현 단위

공학 수업에서 VLA 논문을 다룰 때는 결과를 믿기 전에 최소 재현 단위를 확인해야 한다. 최소 재현 단위는 다음 여섯 항목이다.

1. 모델 backbone과 학습 가능한 모듈의 범위.

Table 1.3: VLA 실패 모드의 기술적 분해

실패 위치	확인해야 할 증상
Perception failure	목표 객체를 찾지 못하거나 distractor를 목표로 잘못 선택한다.
Grounding failure	언어 지시의 관계, 순서, 제약 조건을 잘못 해석한다.
Action representation failure	의미는 맞지만 action token, waypoint, trajectory가 실제 제어에 부적절하다.
Dynamics failure	접촉, 마찰, 무게, deformable object 때문에 계획한 행동이 물리적으로 실패한다.
Temporal failure	초기 step은 맞지만 long-horizon에서 오차가 누적되어 task가 깨진다.
Recovery failure	실패 후 관측을 보고도 재시도, re-grasp, re-plan을 하지 못한다.
Safety failure	충돌, 과도한 force, 금지된 객체 조작 등 시스템 제약을 위반한다.

2. 관측 modality와 카메라 구성.
3. action space의 정의와 controller interface.
4. 학습 데이터의 출처, task split, train-test split.
5. evaluation protocol과 success 판정 기준.
6. 코드, checkpoint, dataset, benchmark script 공개 여부.

이 여섯 항목 중 action space와 evaluation protocol이 빠지면 로봇 논문으로서 재현성이 크게 떨어진다. 예를 들어 end-effector delta pose를 쓰는 정책과 joint velocity를 쓰는 정책은 같은 success rate라도 제어 난이도가 다르다. 사람이 reset한 episode와 자동 reset된 benchmark도 비교가 어렵다. 따라서 이 책에서는 각 대표 논문을 소개할 때 모델 그림보다 먼저 이 재현 단위를 표로 정리할 것이다.

1.15 작업 예제: 컵을 들어 접시에 놓기

VLA의 차이를 가장 단순하게 보려면 하나의 조작 과제를 수식과 시스템 구성으로 분해하면 된다. 명령은 “빨간 컵을 들어 접시 위에 놓아라”라고 하자. 환경에는 빨간 컵, 파란 컵, 접시, 장애물이 있고, 로봇은 wrist camera와 external camera를 사용한다. 이 과제는 겉으로는 단순하지만 VLA의 주요 병목을 모두 포함한다.

먼저 VLM planner 방식은 다음처럼 동작할 수 있다.

$$z_{1:K} = f_{\text{VLM}}(I_t, \ell), \quad (1.12)$$

여기서 $z_{1:K}$ 는 “빨간 컵 찾기”, “컵 쪽으로 이동”, “잡기”, “접시 위로 이동”, “놓기” 같은 symbolic subgoal이다. 이 방식은 설명 가능하고 high-level planning에는 편하지만, 각 subgoal을 실제 제어 명령으로 바꾸는 별도 motion planner와 controller가 필요하다.

반대로 low-level visuomotor policy는 symbolic subgoal을 거치지 않고 바로 행동을 낸다.

$$\mathbf{a}_t = g_\theta(I_t, q_t, \ell), \quad (1.13)$$

여기서 q_t 는 joint state 또는 end-effector state이다. 이 방식은 제어에 직접 연결되지만, 언어

Table 1.4: 컵-접시 과제에서 세 접근의 차이

항목	VLM planner	Visuomotor policy	VLA policy
출력	symbolic subgoal	low-level action	language-conditioned action
장점	해석 가능성	빠른 제어 연결	의미 이해와 행동 생성의 결합
약점	controller 의존	언어 일반화 약함	데이터, latency, action 표현 모두 민감
평가 포인트	plan validity	control success	task success, latency, generalization

지시의 compositional structure를 잘 활용하지 못할 수 있다. VLA는 두 접근의 중간에 있다. VLM의 semantic prior를 쓰되, 출력은 실제 action space로 내려보낸다.

$$\mathbf{A}_{t:t+H-1} = \pi_{\theta}(I_t^{\text{ext}}, I_t^{\text{wrist}}, q_t, \ell). \quad (1.14)$$

이 예제에서 좋은 VLA 논문이라면 다음 정보를 명확히 말해야 한다. 빨간 컵과 파란 컵을 어떻게 구분했는가? 접시 위라는 spatial relation을 어떤 표현으로 grounding했는가? grasp는 pose regression인가, action token인가, diffusion sample인가? 컵을 잡은 뒤 wrist camera의 시야가 바뀔 때 policy가 closed-loop로 수정되는가? 컵이 미끄러졌을 때 recovery를 하는가? 이 질문에 답하지 못하면 성공 영상이 있어도 공학적으로 충분한 설명이 아니다.

이 예제는 이후 장의 기준 예제로 계속 사용할 수 있다. action tokenization 장에서는 $\mathbf{A}_{t:t+H-1}$ 를 어떻게 이산화하는지 볼 것이고, fine-tuning 장에서는 이 과제에 맞추어 어떤 파라미터를 조정하는지 볼 것이다. efficiency 장에서는 이 policy가 제어 주기 안에 실행되는지 계산하고, robustness 장에서는 distractor, occlusion, language ambiguity가 들어왔을 때 어떤 실패가 생기는지 분석한다.

1.16 수업에서 요구할 산출물

이 책을 실제 대학원 수업에서 쓴다면 학생에게 단순 발표보다 다음 산출물을 요구하는 편이 낫다. 첫째, 선택한 논문의 policy interface를 한 장짜리 표로 정리한다. 둘째, action representation을 수식으로 쓴다. 셋째, benchmark protocol을 train/test split과 success criterion 중심으로 재작성한다. 넷째, 실패 모드를 최소 세 가지로 분해한다. 다섯째, 공개 코드와 checkpoint가 없을 경우 재현 가능한 최소 실험을 제안한다.

이 요구사항은 논문을 비판적으로 읽기 위한 최소 조건이다. VLA 논문은 모델 구조 그림이 복잡해 보일수록 핵심이 흐려질 수 있다. 반대로 입력, 출력, loss, benchmark가 명확하면 그림이 단순해도 좋은 연구일 수 있다. 공학 수업에서 중요한 것은 설명의 화려함이 아니라, 같은 조건에서 다른 연구자가 실험을 반복할 수 있는가이다.

1.17 강의에서의 사용 방식

이 장은 첫 주 강의의 기준선을 만든다. 학생들은 이후 장에서 나오는 논문을 읽을 때 매번 같은 질문으로 돌아와야 한다.

1. 이 논문은 VLA의 어느 병목을 건드리는가?
2. action representation은 무엇이고, 왜 그것을 선택했는가?
3. success rate 외에 latency, robustness, generalization을 보고하는가?
4. baseline은 충분히 강한가?
5. 실제 로봇에서 실패할 가능성이 가장 큰 위치는 어디인가?

강의 과제로는 한 논문을 골라 [table 1.3](#)의 실패 모드 표에 맞춰 다시 분석하게 할 수 있다. 이 작업은 단순 요약보다 더 중요하다. VLA 분야에서는 논문 수가 빠르게 늘기 때문에, 모델 이름을 기억하는 능력보다 contribution을 병목 축에 배치하는 능력이 연구 생산성을 결정한다.

1.18 요약

VLA는 VLM의 semantic prior와 로봇 정책 학습을 결합하려는 시도이다. 그러나 VLA의 핵심은 멀티모달 이해가 아니라 행동 생성이다. 2024–2026년 연구 흐름은 이 점을 점점 더 분명하게 보여 준다. action tokenization, diffusion action decoder, action chunking, post-training, efficient inference, robustness benchmark는 모두 같은 질문으로 모인다. “언어로 지시받은 목표를 실제 로봇이 시간 안에, 안전하게, 반복 가능하게 수행할 수 있는가?”

다음 장에서는 VLA의 기본 구조를 더 구체적으로 분해한다. 입력 modality, backbone, policy head, action representation, loss function, inference loop를 각각 따로 보고, 이후 장에서 등장할 논문들을 비교할 공통 표기법을 만든다.

Key papers

- Ahn et al. (2022), SayCan, pre-VLA language-to-skill grounding: LLM planning과 robot affordance 결합을 이해하기 위한 출발점이다.
- Brohan et al. (2022), RT-1, robot transformer: 대규모 robot trajectory가 language-conditioned policy 일반화에 어떤 역할을 하는지 보여 준다.
- Brohan et al. (2023), RT-2, VLA transfer: web-scale VLM knowledge를 robot action token으로 옮기는 대표 사례이다.
- Kim et al. (2024), OpenVLA, arXiv preprint: 7B open-source VLA와 970k real-world demonstration 기반 학습을 제시한 핵심 anchor이다.
- Black et al. (2024/2026), π_0 , RSS 2025: VLM 위에 flow matching action architecture를 얹은 general robot control anchor이다.

Reading assignment [Basic]

OpenVLA와 SayCan을 같은 13-question template로 읽고, 하나는 VLA로, 다른 하나는 pre-VLA foundation으로 분류되는 이유를 controller interface와 action output 관점에서 1쪽으로 정리하라.

Chapter 2

VLA의 기본 구조

2.1 장의 목표

1장은 VLA를 언어 조건부 페루프 정책으로 정의했다. 2장의 목표는 그 정책이 실제 시스템에서 어떤 모듈로 구성되는지 분해하는 것이다. VLA 논문은 모델명을 앞세우는 경우가 많지만, 공학적으로 중요한 것은 모델명이 아니라 interface이다. 입력은 무엇인가, 출력은 무엇인가, action space는 어떻게 정의되는가, 학습 loss는 무엇인가, inference loop는 어느 주기로 돌아가는가가 먼저 정리되어야 한다.

이 장에서는 VLA를 다음 여섯 구성요소로 나눈다.

1. 관측 인코더: 이미지, 비디오, proprioception, force, tactile signal을 표현으로 바꾼다.
2. 언어 인코더: 자연어 지시를 task condition 또는 goal representation으로 바꾼다.
3. 멀티모달 결합부: 시각, 언어, 로봇 상태를 하나의 context로 통합한다.
4. 행동 표현부: continuous action, action token, action chunk, trajectory를 정의한다.
5. 정책 head: context로부터 행동 또는 행동 분포를 생성한다.
6. 실행 loop: 센서 입력, 모델 추론, safety filter, controller command를 시간적으로 연결한다.

이 장의 관점

VLA 구조를 읽을 때는 transformer block 수나 parameter 수보다 먼저 “어떤 변수에서 어떤 변수로 가는 함수인가”를 확인해야 한다. 같은 VLA라는 이름을 써도 high-level planner, visuomotor policy, diffusion action decoder, benchmark evaluator는 서로 다른 시스템이다.

2.2 표준 표기법

시점 t 에서 로봇이 받는 관측을 다음과 같이 쓴다.

$$\mathbf{o}_t = \{I_t^{1:C}, q_t, f_t, m_t\}. \quad (2.1)$$

$I_t^{1:C}$ 는 C 개의 카메라 영상, q_t 는 joint angle, velocity, gripper state 같은 proprioception, f_t 는

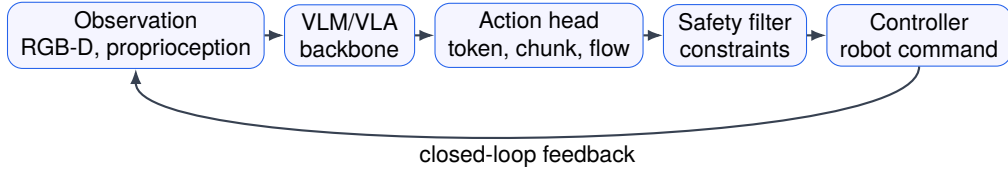


Figure 2.1: Author-created schematic. VLA system stack. 공개용 원고에서는 VLA를 backbone 하나가 아니라 observation, action head, safety filter, controller interface가 묶인 closed-loop system으로 읽는다.

force/torque 또는 tactile signal, m_t 는 선택적으로 들어가는 memory state나 map representation이다. 언어 지시는 ℓ 으로 쓴다. 정책이 사용하는 이력은

$$\mathbf{h}_t = \{\mathbf{o}_{t-k:t}, \mathbf{a}_{t-k:t-1}\} \quad (2.2)$$

처럼 최근 k step 관측과 행동을 포함한다. 그러면 VLA 정책은 일반적으로 다음 조건부 분포로 쓸 수 있다.

$$\pi_{\theta}(\mathbf{A}_t \mid \mathbf{h}_t, \ell), \quad (2.3)$$

여기서 $\mathbf{A}_t$ 는 반드시 단일 행동일 필요가 없다. 단일 step 행동 $\mathbf{a}_t$ 일 수도 있고, horizon H 의 action chunk $\mathbf{a}_{t:t+H-1}$ 일 수도 있으며, discretized token sequence $u_{1:L}$ 일 수도 있다. 많은 논문의 차이는 결국 $\mathbf{A}_t$ 를 어떻게 정의하는가에서 나온다.

2.3 관측 인코더

VLA의 첫 모듈은 관측 인코더이다. 가장 단순한 경우에는 RGB 이미지 한 장을 vision encoder에 넣는다.

$$z_t^{\text{vis}} = E_{\text{vis}}(I_t). \quad (2.4)$$

그러나 실제 로봇에서는 한 장의 이미지가 충분하지 않은 경우가 많다. manipulation에서는 wrist camera와 external camera가 서로 다른 정보를 제공한다. wrist camera는 end-effector 주변의 접촉과 물체 상대 위치를 잘 보고, external camera는 전체 장면과 goal object 위치를 잘 본다. multi-view VLA는 이를 다음처럼 결합한다.

$$z_t^{\text{vis}} = \text{Fuse} \left(E_{\text{vis}}(I_t^1), E_{\text{vis}}(I_t^2), \dots, E_{\text{vis}}(I_t^C) \right). \quad (2.5)$$

이때 Fuse는 단순 concatenation일 수도 있고, cross-attention일 수도 있으며, view token을 transformer에 넣는 방식일 수도 있다. 논문을 읽을 때는 multi-view를 썼다는 사실만 볼 것이 아니라 camera calibration, view ordering, missing view 처리, inference cost를 함께 확인해야 한다.

Proprioception도 중요하다. 로봇의 현재 joint state를 모르면 같은 이미지와 같은 언어 지시에서도 필요한 action이 달라진다. 예를 들어 gripper가 이미 컵 가까이에 있는 상태와 테이블 바깥에 있는 상태는 같은 “컵을 잡아라” 명령에 대해 다른 행동을 요구한다. 따라서 상태 인코더는 보통 다음과 같이 둔다.

$$z_t^{\text{state}} = E_{\text{state}}(q_t, \dot{q}_t, g_t), \quad (2.6)$$

여기서 g_t 는 gripper state이다. Force-aware VLA나 contact-rich manipulation 논문에서는 f_t 를 별도 인코더에 넣는다.

$$z_t^{\text{force}} = E_{\text{force}}(f_{t-r:t}). \quad (2.7)$$

force signal은 이미지보다 낮은 차원이지만, 접촉 상태를 직접 반영한다는 점에서 중요하다. 다만 force를 넣으면 sensor calibration, noise filtering, contact timing alignment가 새 문제가 된다.

2.4 언어 인코더와 goal representation

언어 지시 ℓ 는 단순한 문자열이 아니다. VLA에서 언어는 task identity, object reference, spatial relation, temporal order, constraint를 모두 포함할 수 있다. 언어 인코더는 이를 embedding으로 바꾼다.

$$z^{\text{lang}} = E_{\text{lang}}(\ell). \quad (2.8)$$

문제는 z^{lang} 가 어떤 수준의 goal을 표현하는가이다. “빨간 컵을 접시 위에 놓아라”라는 지시는 최소한 다음 정보를 포함한다.

1. target object: 빨간 컵
2. destination object 또는 region: 접시
3. relation: 위에 놓기
4. implicit constraints: 컵을 떨어뜨리지 않기, 장애물과 충돌하지 않기
5. terminal condition: 컵이 접시 위에 안정적으로 놓인 상태

VLA 논문이 언어 지시를 다룬다고 말하더라도, 실제로는 template instruction만 쓰는 경우가 많다. template instruction은 일반화 평가를 단순하게 만들지만, 자연어 ambiguity나 compositional instruction을 충분히 검증하지 못한다. 따라서 언어 일반화를 주장하는 논문은 seen template, unseen template, unseen object, unseen composition을 분리해서 보여 주어야 한다.

2.5 멀티모달 결합

관측과 언어를 얻은 뒤에는 이를 결합해야 한다. 가장 단순한 구조는 모든 embedding을 concatenate한 뒤 MLP 또는 transformer에 넣는 것이다.

$$c_t = F_{\theta}(z_t^{\text{vis}}, z_t^{\text{state}}, z^{\text{lang}}). \quad (2.9)$$

c_t 는 policy head가 사용할 context이다. 큰 VLA 모델에서는 F_{θ} 가 VLM backbone 또는 LLM decoder일 수 있다. Adapter 기반 모델에서는 backbone은 고정하고, visual token과 action token

Table 2.1: 멀티모달 결합 방식의 차이

결합 방식	공학적 의미
Early fusion	언어가 시각 feature 추출부터 영향을 주어 target object grounding에 유리할 수 있다.
Late fusion	각 modality를 독립적으로 인코딩한 뒤 합쳐 구현이 단순하지만 세밀한 grounding이 약할 수 있다.
Cross-attention Adapter fusion	언어 token과 시각 token 사이의 대응을 명시적으로 학습할 수 있다.
Routed fusion	큰 backbone을 고정하고 작은 모듈만 학습해 비용을 줄인다. token이나 expert를 조건부로 선택해 계산량을 줄이고 task별 specialization을 유도한다.

사이에 작은 학습 가능 모듈을 삽입한다. Routing이나 sparsification 기반 모델에서는 모든 token이 같은 계산 경로를 지나지 않도록 선택적으로 처리한다.

멀티모달 결합에서 중요한 질문은 세 가지이다. 첫째, 언어가 시각 token에 언제 영향을 주는가? early fusion이면 vision feature 추출 단계부터 언어가 개입하고, late fusion이면 이미 추출된 feature를 나중에 합친다. 둘째, 로봇 상태가 transformer context 안에 들어가는가, 아니면 마지막 policy head에만 들어가는가? 셋째, action history가 포함되는가? 이 질문에 따라 같은 backbone이라도 페루프 성질이 달라진다.

2.6 행동 공간

VLA에서 가장 중요한 설계 선택은 행동 공간이다. 일반적으로 행동은 다음 중 하나로 표현된다.

1. joint position 또는 joint velocity
2. end-effector delta pose
3. gripper open/close를 포함한 Cartesian command
4. waypoint sequence
5. discretized action token
6. diffusion 또는 flow model이 생성한 action trajectory

End-effector delta pose는 다음처럼 쓸 수 있다.

$$\mathbf{a}_t = (\Delta x_t, \Delta y_t, \Delta z_t, \Delta r_t, \Delta p_t, \Delta y_t^{\text{rot}}, g_t). \quad (2.10)$$

여기서 앞의 세 항은 위치 변화, 다음 세 항은 회전 변화, 마지막 g_t 는 gripper command이다. 이 표현은 조작 과제에서 자주 쓰이지만, controller가 end-effector command를 안정적으로 joint command로 변환한다는 가정이 필요하다.

반대로 action token 방식은 continuous action을 codebook index로 바꾼다.

$$u_t = Q(\mathbf{a}_t), \quad u_t \in \{1, 2, \dots, K\}. \quad (2.11)$$

Q 는 vector quantizer이고 K 는 codebook size이다. 이 방식은 LLM/VLM의 token prediction

구조와 잘 맞지만, quantization error가 생긴다.

$$\epsilon_Q = \mathbb{E}_{\mathbf{a} \sim \mathcal{D}} [\|\mathbf{a} - Q^{-1}(Q(\mathbf{a}))\|_2^2]. \quad (2.12)$$

좋은 action tokenizer 논문은 단순히 token을 썼다고 말하지 않고, ϵ_Q 가 task success에 어떤 영향을 주는지 보여 주어야 한다. codebook이 너무 작으면 행동이 거칠어지고, 너무 크면 token prediction이 어려워질 수 있다.

2.7 정책 head

정책 head는 context c_t 에서 행동을 생성한다. 가장 단순한 회귀 head는 평균 행동을 직접 예측한다.

$$\hat{\mathbf{a}}_t = Wc_t + b. \quad (2.13)$$

이 방식은 빠르고 구현이 단순하지만 multi-modal action distribution을 표현하기 어렵다. 같은 장면에서 컵을 왼쪽으로 돌아 잡을 수도 있고 오른쪽으로 돌아 잡을 수도 있다. 평균을 내면 두 행동 사이의 부적절한 trajectory가 나올 수 있다.

분포 기반 head는 평균과 분산을 예측한다.

$$p_\theta(\mathbf{a}_t | c_t) = \mathcal{N}(\mathbf{a}_t; \mu_\theta(c_t), \Sigma_\theta(c_t)). \quad (2.14)$$

Diffusion policy 계열은 행동 trajectory를 denoising 과정으로 생성한다. 단순화하면 noise가 섞인 trajectory $\mathbf{A}^k$ 에서 더 깨끗한 trajectory $\mathbf{A}^{k-1}$ 를 예측한다.

$$p_\theta(\mathbf{A}^{k-1} | \mathbf{A}^k, c_t). \quad (2.15)$$

Diffusion 방식은 multi-modal trajectory를 표현하는 데 강하지만, denoising step 수가 많으면 latency가 커진다. 그래서 2025–2026년에는 diffusion의 표현력과 real-time constraint를 동시에 맞추려는 연구가 늘었다. Flow policy도 비슷한 문제의식에서 등장한다. 핵심은 action distribution을 풍부하게 만들수록 inference cost가 증가하기 쉽다는 점이다.

2.8 학습 목적

가장 기본적인 VLA 학습은 demonstration action을 맞추는 behavior cloning이다. continuous action이면 평균제곱오차를 쓸 수 있다.

$$\mathcal{L}_{\text{MSE}} = \sum_{i,t} \|\mathbf{a}_t^{(i)} - \hat{\mathbf{a}}_t^{(i)}\|_2^2. \quad (2.16)$$

action token이면 cross entropy를 쓴다.

$$\mathcal{L}_{\text{CE}} = - \sum_{i,t} \log p_\theta(u_t^{(i)} | \mathbf{h}_t^{(i)}, \ell^{(i)}). \quad (2.17)$$

Diffusion이면 noise prediction loss를 쓴다.

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{\mathbf{A}, k, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{A}^k, k, c_t)\|_2^2 \right]. \quad (2.18)$$

세 loss는 서로 다른 action representation을 전제한다. 따라서 논문 비교에서 loss만 보거나 action representation만 보면 안 된다. 예를 들어 token CE loss가 낮아도 복원된 continuous action이 부드럽지 않으면 로봇 제어에 좋지 않을 수 있다. MSE loss가 낮아도 multi-modal demonstration에서는 평균 행동 문제가 생길 수 있다. Diffusion loss가 좋아도 inference latency가 제어 주기를 넘으면 배포가 어렵다.

2.9 Inference loop

VLA는 offline model이 아니라 실행 loop 안에 들어가는 정책이다. 한 step의 실행은 다음 순서로 볼 수 있다.

1. sensor에서 이미지와 로봇 상태를 읽는다.
2. 전처리와 normalization을 수행한다.
3. VLA가 행동 또는 action chunk를 예측한다.
4. safety filter가 collision, joint limit, force limit을 검사한다.
5. low-level controller가 command를 실행한다.
6. 다음 관측을 받아 다시 반복한다.

이를 시간 budget으로 쓰면 다음과 같다.

$$\tau_{\text{sense}} + \tau_{\text{pre}} + \tau_{\text{model}} + \tau_{\text{safe}} + \tau_{\text{ctrl}} \leq \tau_{\text{loop}}. \quad (2.19)$$

여기서 τ_{loop} 는 원하는 페루프 주기이다. VLA 논문이 real-time을 주장한다면 τ_{model} 만 보고해서는 부족하다. sensor I/O, image preprocessing, action decoding, safety filter까지 포함한 end-to-end latency가 필요하다.

Action chunk를 쓰면 모델 호출 주기는 느려질 수 있지만 controller 실행은 빠르게 유지할 수 있다. H step chunk를 r Hz controller에서 실행하면, 모델 호출 주기는 대략 r/H 가 된다. 하지만 chunk가 길수록 중간에 환경이 변했을 때 반응이 늦어진다. 따라서 chunk length H 는 latency와 reactivity 사이의 hyperparameter이다.

2.10 메모리와 장기 과제

짧은 조작 과제에서는 현재 이미지와 언어 지시만으로도 충분해 보일 수 있다. 그러나 long-horizon task에서는 과거에 무엇을 했는지 기억해야 한다. 예를 들어 “서랍을 열고, 컵을 넣고, 다시 닫아라”라는 과제에서는 현재 이미지가 같아도 현재 단계가 다르면 행동이 달라진다.

메모리 상태를 m_t 라고 하면 업데이트는 다음처럼 쓸 수 있다.

$$m_t = U_{\theta}(m_{t-1}, \mathbf{o}_t, \mathbf{a}_{t-1}, \ell). \quad (2.20)$$

Table 2.2: VLA 구조 분석을 위한 최소 표

항목	확인할 내용
관측 입력	카메라 수, image resolution, proprioception, force/tactile 포함 여부
언어 입력	template instruction, free-form instruction, multi-step instruction 여부
backbone	VLM, LLM, transformer, diffusion model, frozen/fine-tuned 여부
결합 방식	early fusion, late fusion, cross-attention, adapter, routing
action space	joint, end-effector, waypoint, token, trajectory, action chunk
loss	MSE, cross entropy, diffusion loss, auxiliary loss, reward-based objective
inference	closed-loop 주기, chunk length, latency, safety filter
평가	dataset, robot platform, success criterion, baseline, train-test split
공개 자원	code, checkpoint, dataset, benchmark script

정책은 m_t 를 조건으로 행동을 낸다.

$$\pi_{\theta}(\mathbf{a}_t \mid \mathbf{o}_t, m_t, \ell). \quad (2.21)$$

MemoryVLA류 논문은 이 구조를 명시적으로 강화하려는 방향으로 볼 수 있다. 다만 메모리가 있다고 해서 자동으로 reasoning이 생기는 것은 아니다. 메모리가 실제로 task progress, object state, failure state를 저장하는지 ablation으로 확인해야 한다.

2.11 Safety filter와 controller interface

많은 VLA 논문은 policy output까지만 설명하고, 실제 controller와 safety layer는 간단히 처리한다. 그러나 실제 로봇에서는 이 부분이 중요하다. VLA가 낸 행동 $\hat{\mathbf{a}}_t$ 는 그대로 실행되지 않고, 보통 safety filter를 지난다.

$$\mathbf{a}_t^{\text{exec}} = S(\hat{\mathbf{a}}_t, q_t, \mathcal{C}), \quad (2.22)$$

여기서 S 는 safety filter이고, $\mathcal{C}$ 는 collision constraint, joint limit, velocity limit, force limit 같은 제약 집합이다. 만약 논문에서 safety filter가 강하게 개입한다면, 성공률이 VLA 자체의 능력인지 classical controller와 constraint checker의 도움인지 구분해야 한다.

Controller interface도 마찬가지이다. End-effector command를 쓰는 정책은 inverse kinematics와 low-level impedance controller에 의존한다. Joint command를 직접 내는 정책은 더 직접적이지만 학습이 어려울 수 있다. Torque command까지 내려가면 제어 자유도는 커지지만 안전성과 데이터 요구량이 크게 증가한다.

2.12 구조 비교표

VLA 논문을 읽을 때는 다음 표를 채우면 구조가 명확해진다.

이 표는 이후 모든 장에서 반복해서 쓸 수 있다. 특히 action representation 논문, efficient VLA 논문, benchmark 논문은 contribution 위치가 다르기 때문에 같은 표로 정리해야 비교가 가능하다.

2.13 요약

VLA의 기본 구조는 관측 인코더, 언어 인코더, 멀티모달 결합부, 행동 표현부, 정책 head, 실행 loop로 나눌 수 있다. 이 분해를 사용하면 모델 이름이 달라도 논문이 실제로 어떤 병목을 해결하려는지 볼 수 있다. VLA에서 가장 중요한 설계 선택은 action representation과 inference loop이다. 언어와 시각을 잘 이해해도, 행동 표현이 부적절하거나 latency budget을 만족하지 못하면 실제로 로봇 정책으로는 약하다.

다음 장에서는 이 책 전체의 읽기 도구가 될 13개 질문 프레임을 정리한다. 13개 질문은 논문을 소개하는 양식이 아니라, VLA 연구가 충분히 공학적으로 설계되고 검증되었는지 평가하는 체크리스트로 사용할 것이다.

Key papers

- Driess et al. (2023), PaLM-E: embodied multimodal representation이 VLA 구조의 semantic side를 어떻게 준비했는지 보여 준다.
- Brohan et al. (2023), RT-2: VLM token interface를 robot action으로 확장한 구조적 anchor이다.
- Octo Model Team et al. (2024), Octo, arXiv preprint: 800k Open X-Embodiment trajectories 기반 open-source generalist policy이다.
- Kim et al. (2024), OpenVLA: Llama 2, DINOv2, SigLIP 기반 open VLA architecture와 action head 연결을 확인할 수 있다.
- NVIDIA et al. (2025), GR00T N1: vision-language module과 diffusion transformer action module을 분리한 humanoid VLA 사례이다.

Reading assignment [Basic]

OpenVLA, Octo, GR00T N1 중 하나를 골라 [figure 2.1](#)의 다섯 블록에 대응되는 구체 모듈, 입력, 출력, 공개 여부를 표로 채워라.

Chapter 3

VLA 연구를 읽는 13개 질문

3.1 장의 목표

앞의 두 장에서는 VLA를 언어 조건부 페루프 정책으로 정의하고, 관측 인코더, 언어 인코더, 멀티모달 결합, 행동 표현, 정책 head, 실행 loop로 구조를 분해했다. 이제 필요한 것은 논문을 읽을 때 같은 기준으로 비교하는 방법이다. VLA 논문은 빠르게 늘고 있으며, 제목만 보면 모두 “generalist robot policy”, “efficient VLA”, “reasoning VLA”처럼 보일 수 있다. 그러나 실제 기여는 서로 다르다. 어떤 논문은 행동 표현을 바꾸고, 어떤 논문은 fine-tuning recipe를 바꾸며, 어떤 논문은 벤치마크를 만든다.

이 장의 목적은 VLA 논문을 읽는 13개 질문을 공학적 평가 도구로 정식화하는 것이다. 여기서 13개 질문은 요약 양식이 아니다. 논문이 실제로 무엇을 주장하며, 그 주장이 어떤 실험으로 검증되었고, 어디까지 믿을 수 있는지를 확인하는 audit checklist이다.

3.2 13개 질문의 전체 구조

이 책에서 사용하는 13개 질문은 다음과 같다.

1. 배경: 이 연구는 어떤 분야와 임무 맥락에 놓이는가?
2. 문제: 구체적으로 어떤 기능 또는 과제를 풀려 하는가?
3. 기존 한계: 이전 접근의 어떤 병목 때문에 새 방법이 필요한가?
4. 목표: 논문이 달성하려는 목표는 측정 가능한가?
5. 방법: 제안 시스템은 어떤 입력, 출력, 모듈, 학습 절차를 갖는가?
6. 핵심 아이디어: 기술적으로 무엇이 새롭거나 중요한가?
7. 검증: 어떤 데이터셋, 시뮬레이터, 실제 로봇, 시나리오로 평가했는가?
8. 결과: 어떤 지표에서 얼마나 좋아졌는가?
9. 비교: baseline은 공정하고 충분히 강한가?
10. 의의: 결과가 VLA와 로봇 학습에서 무엇을 가능하게 하는가?
11. 한계: 남은 제약과 실패 모드는 무엇인가?

12. 향후 과제: 다음 연구 단계는 구체적으로 무엇인가?
13. 자원 공개: 코드, 데이터, checkpoint, benchmark가 공개되었는가?

이 순서는 중요하다. 많은 논문 요약은 방법과 결과부터 시작하지만, 그렇게 읽으면 왜 그 방법이 필요한지 놓치기 쉽다. 반대로 배경과 문제만 길게 읽으면 논문이 실제로 무엇을 검증했는지 흐려진다. 13개 질문은 논문의 주장 흐름을 배경에서 공개 자원까지 닫힌 형태로 추적한다.

3.3 질문 1–4: 문제 설정을 검증하기

첫 네 질문은 논문의 문제 설정을 검사한다. 좋은 VLA 논문은 배경, 문제, 기존 한계, 목표가 서로 직접 연결된다. 예를 들어 “VLA가 느리다”는 배경이 있다면 문제는 추론 latency여야 하고, 기존 한계는 autoregressive decoding 또는 큰 backbone의 계산량이어야 하며, 목표는 latency를 줄이면서 success rate를 유지하는 것이어야 한다. 만약 배경은 real-world deployment인데 실험은 작은 offline dataset에만 머문다면 문제 설정과 검증이 분리된 것이다.

3.3.1 배경

배경은 연구의 큰 위치를 정한다. VLA 논문에서 배경은 보통 다음 중 하나이다.

- manipulation: 언어 지시를 실제 조작 행동으로 바꾸는 문제
- efficient VLA: 큰 VLA를 실제 제어 주기에 맞추는 문제
- action representation: 연속 행동을 모델이 다루기 쉬운 표현으로 바꾸는 문제
- benchmark: 모델 성능을 공정하게 비교하는 문제
- reasoning and planning: 장기 과제와 단계적 의사결정을 다루는 문제
- robustness and safety: 분포 변화와 실패 모드를 다루는 문제

배경을 쓸 때는 “로봇이 중요하다” 같은 일반론으로 끝내면 안 된다. 어떤 로봇 과제, 어떤 데이터 흐름, 어떤 deployment constraint가 문제인지 지정해야 한다.

3.3.2 문제

문제는 구체적이어야 한다. “VLA의 성능을 높인다”는 문제 정의가 아니다. 다음처럼 변수와 평가 기준으로 바꿀 수 있어야 한다.

$$\max_{\theta} \text{SR}(\theta) \quad \text{s.t.} \quad \tau_{\text{model}}(\theta) \leq \tau_{\text{max}}. \quad (3.1)$$

이 식은 efficient VLA 논문의 문제 정의에 가깝다. 성공률을 최대화하되 모델 추론 시간이 허용 범위를 넘지 않아야 한다. Action representation 논문이라면 문제는 quantization error와 policy loss 사이의 균형이 될 수 있다.

$$\min_{Q, \theta} \mathcal{L}_{\text{policy}}(\theta; Q) + \lambda \epsilon_Q. \quad (3.2)$$

문제 정의가 이런 식으로 쓰이지 않더라도, 독자는 논문을 읽으며 암묵적 objective를 복원해야 한다.

3.3.3 기존 한계

기존 한계는 논문의 필요성을 만드는 부분이다. 약한 논문은 기존 연구를 단순히 “비싸다”, “느리다”, “일반화가 약하다”라고 말하지만, 어떤 구조적 원인 때문에 그런지 설명하지 않는다. 강한 논문은 한계를 모듈 수준으로 지정한다. 예를 들어 action tokenization이 약하다면 codebook 설계가 문제인지, temporal consistency가 문제인지, high-frequency control과 token prediction의 mismatch가 문제인지 밝혀야 한다.

3.3.4 목표

목표는 측정 가능해야 한다. 목표가 “더 일반적인 VLA”라면 어떤 test split에서 일반화를 측정하는지 필요하다. 목표가 “real-time VLA”라면 loop frequency와 latency budget이 필요하다. 목표가 “robust VLA”라면 어떤 perturbation, distractor, unseen object, unseen instruction을 넣는지 필요하다. 목표가 측정 불가능하면 실험이 좋아 보여도 주장이 닫히지 않는다.

3.4 질문 5–6: 방법의 기술적 중심 찾기

질문 5와 6은 방법을 읽는 단계이다. 여기서 중요한 것은 방법 설명과 핵심 아이디어를 분리하는 것이다. 방법은 시스템 전체의 구성이고, 핵심 아이디어는 그중 논문이 실제로 새로 만든 부분이다.

3.4.1 방법

VLA 방법 설명은 최소한 다음 다섯 항목을 포함해야 한다.

1. 입력: 이미지, 비디오, proprioception, force, 언어 지시
2. 출력: action token, end-effector command, joint command, trajectory, action chunk
3. backbone: VLM, LLM, transformer, diffusion/flow model
4. 학습: behavior cloning, instruction tuning, reinforcement fine-tuning, adapter tuning
5. 추론: closed-loop frequency, chunk length, decoding step, safety filter

논문이 모델 그림만 보여 주고 위 항목을 명확히 하지 않으면 재현성이 약하다. 특히 action space와 controller interface는 반드시 확인해야 한다.

3.4.2 핵심 아이디어

핵심 아이디어는 한 문장으로 줄일 수 있어야 한다. 예를 들어 다음과 같이 쓸 수 있다.

- Action tokenization 논문: 연속 로봇 행동을 VLM/LLM이 예측하기 쉬운 discrete token sequence로 바꾼다.

Table 3.1: VLA 검증 수준

수준	의미
Offline dataset	저장된 trajectory와 observation에서 action prediction 성능을 평가한다.
Simulation bench- mark	시뮬레이터에서 policy를 실행해 success rate와 robustness를 평가한다.
Real robot	실제 센서, controller, latency, 물리 접촉을 포함해 task success를 평가한다.

- Efficient VLA 논문: 모든 token을 같은 계산량으로 처리하지 않고 task-relevant token만 선택적으로 계산한다.
- Post-training 논문: pretrained VLA를 simulator reward 또는 verified feedback으로 다시 보정한다.
- Benchmark 논문: 다양한 과제와 domain randomization으로 VLA의 robustness를 같은 조건에서 비교한다.

핵심 아이디어가 “새로운 framework를 제안한다”로만 남으면 부족하다. 어떤 병목을 어떤 mechanism으로 줄이는지가 보여야 한다.

3.5 질문 7–9: 검증의 공정성 보기

질문 7–9는 논문의 실험을 평가한다. VLA 분야에서 실험은 특히 조심해서 봐야 한다. 로봇 플랫폼, reset 방식, instruction template, object set, camera view, demonstration quality가 조금만 달라도 성공률이 달라진다.

3.5.1 검증

검증은 다음 세 수준으로 나눌 수 있다.

세 수준은 서로 대체되지 않는다. offline action prediction이 좋아도 closed-loop success가 낮을 수 있고, simulation success가 좋아도 real robot에서는 접촉과 센서 노이즈 때문에 실패할 수 있다. 논문이 어떤 수준의 검증을 했는지 먼저 분리해야 한다.

3.5.2 결과

결과는 숫자로 읽되, 숫자의 조건을 함께 읽어야 한다. 성공률은 다음처럼 조건부 지표이다.

$$SR = SR(\mathcal{T}, \mathcal{E}, \mathcal{O}, \mathcal{L}, \mathcal{R}), \quad (3.3)$$

여기서 $\mathcal{T}$ 는 task set, $\mathcal{E}$ 는 environment distribution, $\mathcal{O}$ 는 object set, $\mathcal{L}$ 은 language distribution, $\mathcal{R}$ 은 robot platform이다. 단일 숫자로 쓰인 success rate는 이 조건들을 압축한 결과일 뿐이다.

Latency도 마찬가지이다. 모델 forward time만 보고하면 실제 deployment latency를 알 수 없다. end-to-end latency는 센서 입력, 전처리, 모델 추론, decoding, safety filter, controller interface를 포함해야 한다.

3.5.3 비교

비교에서 가장 중요한 것은 baseline의 강도이다. 약한 baseline만 이기면 논문 주장이 과장될 수 있다. VLA 논문에서 baseline은 다음 그룹을 고려해야 한다.

- 같은 데이터로 학습한 기존 VLA
- 같은 action space를 쓰는 diffusion policy 또는 visuomotor policy
- frozen VLM + planner + controller pipeline
- ablation: backbone 고정, action head 변경, data size 변경, chunk length 변경

특히 action representation 논문은 같은 backbone과 같은 데이터에서 action representation만 바꾸는 ablation이 필요하다. Efficient VLA 논문은 같은 success rate에서 latency가 줄었는지, 같은 latency에서 success rate가 올랐는지 분리해서 보여야 한다.

3.6 질문 10–12: 의미와 한계를 분리하기

질문 10–12는 논문 결과를 해석하는 단계이다. 여기서 흔한 오류는 의의를 과장하거나 한계를 future work 한 줄로 처리하는 것이다.

3.6.1 의의

의의는 결과가 가능하게 하는 것을 말한다. VLA 논문에서 의의는 보통 다음 중 하나이다.

1. 더 작은 GPU 또는 더 빠른 제어 주기에서 VLA를 실행할 수 있다.
2. 더 다양한 조작 과제에서 language-conditioned policy를 쓸 수 있다.
3. action representation을 바꾸어 VLM과 로봇 제어 사이의 mismatch를 줄인다.
4. benchmark를 통해 모델 간 비교와 failure diagnosis가 가능해진다.
5. reasoning, memory, world model을 통해 long-horizon task로 확장한다.

의의는 논문이 실제로 검증한 범위를 넘으면 안 된다. simulation만 했다면 real-world deployment 의의는 제한적으로 써야 한다. offline dataset만 했다면 closed-loop robot policy로서의 의의는 보수적으로 써야 한다.

3.6.2 한계

한계는 두 종류로 나누어야 한다. 첫째는 논문이 명시한 한계이다. 둘째는 독자가 근거로부터 도출하는 한계이다. 예를 들어 논문이 “실제 로봇 실험은 하지 않았다”고 말하면 명시적 한계이다. 논문이 말하지 않았지만 benchmark가 template instruction만 사용했다면, 자연어 일반화가 제한적이라는 도출 한계를 쓸 수 있다.

한계 작성 원칙

한계는 논문을 깎아내리는 문장이 아니라 주장 범위를 정확히 닫는 문장이다. 좋은 한계 분석은 다음 실험을 설계할 수 있게 만든다.

Table 3.2: VLA 공개 자원의 수준

수준	확인할 내용
Paper only	논문과 PDF만 있고 재현 코드는 없다.
Code	학습 또는 추론 코드가 공개되어 있다.
Checkpoint	pretrained 또는 fine-tuned model weight가 공개되어 있다.
Dataset	trajectory, language instruction, split metadata가 공개되어 있다.
Benchmark	evaluation script, success checker, environment setup이 공개되어 있다.
Full stack	code, checkpoint, data, benchmark, robot/simulator setup이 함께 제공된다.

3.6.3 향후 과제

향후 과제는 “더 연구해야 한다”가 아니어야 한다. VLA 논문에서 의미 있는 향후 과제는 구체적인 실험이나 시스템 확장으로 써야 한다.

- unseen object와 unseen instruction을 분리한 generalization test
- real robot latency와 safety filter를 포함한 deployment benchmark
- 실패 trajectory와 recovery data를 포함한 post-training
- multi-embodiment action representation의 통일
- force/tactile signal을 포함한 contact-rich VLA

3.7 질문 13: 공개 자원 확인

마지막 질문은 공개 자원이다. 공개 자원은 단순 부록이 아니다. VLA는 로봇 플랫폼, controller, dataset split, evaluation script가 복잡하기 때문에 공개 여부가 연구의 검증 가능성을 크게 좌우한다.

공개 자원은 다음 수준으로 나눈다.

논문 요약에서 “코드 공개”라고 쓸 때는 실제 URL을 확인해야 한다. GitHub repository가 있어도 empty repository이거나 inference만 있고 training code가 없을 수 있다. Dataset page가 있어도 license가 제한적일 수 있다. 이 책에서는 공개 자원을 평가할 때 링크 존재와 사용 가능성을 구분한다.

3.8 13개 질문을 표로 적용하기

실제 논문을 읽을 때는 다음 표를 채우면 된다.

이 표는 간단하지만 강력하다. 한 논문을 이 표로 채우지 못하면 보통 두 가지 중 하나이다. 논문 자체가 필요한 정보를 충분히 제공하지 않았거나, 독자가 아직 논문의 핵심을 잡지 못한 것이다.

3.9 나쁜 요약과 좋은 요약

13-question framework는 문장을 길게 만드는 도구가 아니라 근거 없는 요약을 줄이는 도구이다. 예를 들어 OpenVLA-OFT를 다음처럼 요약하면 부족하다.

Table 3.3: 13-question reading template

항목	작성 내용
배경	분야, task family, deployment context
문제	해결하려는 기능과 평가 가능한 objective
기존 한계	이전 접근의 모듈 수준 병목
목표	측정 가능한 target metric 또는 system property
방법	입력, 출력, backbone, loss, inference loop
핵심 아이디어	contribution mechanism 한 문장
검증	dataset, simulator, robot, scenario
결과	metric과 수치, 조건
비교	baseline, ablation, fairness
의의	검증 범위 안에서 가능한 것
한계	명시 한계와 도출 한계
향후 과제	다음 실험 또는 시스템 확장
자원 공개	code, data, checkpoint, benchmark 링크

Bad summary

OpenVLA-OFT는 OpenVLA를 fine-tuning해서 성능을 크게 올린 논문이다. 최신 VLA fine-tuning 방법을 제안했고 LIBERO에서 좋은 결과를 보였다.

같은 내용을 더 좋은 요약으로 바꾸면 다음과 같다.

Good summary

OpenVLA-OFT는 같은 OpenVLA base model을 유지한 상태에서 parallel decoding, action chunking, continuous action representation, L1 objective를 결합해 LIBERO 평균 success와 action generation throughput을 함께 개선한 fine-tuning recipe이며, 따라서 contribution은 새 backbone보다 action interface와 adaptation procedure에 있다 [11].

좋은 요약은 짧아도 입력, 출력, 바뀐 mechanism, 검증 지표, 주장의 범위를 포함한다. 나쁜 요약은 “좋다”, “최신”, “성능 향상”처럼 검증 조건을 지운다.

3.10 리뷰와 연구 기획에서의 사용

13개 질문은 읽기 도구이지만, 동시에 연구 기획 도구이다. 새 VLA 논문을 쓰기 전에 각 항목을 질문으로 뒤집으면 된다.

- 배경: 독자가 왜 이 문제가 중요한지 바로 납득하는가?
- 문제: 과제가 너무 넓거나 모호하지 않은가?
- 기존 한계: prior work의 한계와 제안 방법이 직접 연결되는가?
- 목표: 목표가 측정 가능하고 검증 가능한가?
- 방법: 입력, 출력, 모듈, 학습/추론 절차가 재현 가능한가?
- 검증: 실험이 contribution을 직접 검증하는가?

- 비교: baseline이 충분히 강한가?
- 한계: reviewer가 찌를 약점을 선제적으로 다루는가?

이 방식은 논문 초안 평가에도 유용하다. 어떤 항목이 비어 있으면 그 부분이 reviewer의 공격 지점이 된다. 특히 VLA 분야에서는 benchmark와 baseline이 약하면 모델 구조가 좋아도 설득력이 급격히 떨어진다.

3.11 요약

VLA 논문을 읽을 때 13개 질문은 배경에서 공개 자원까지 주장을 담은 도구이다. 이 프레임의 핵심은 방법을 멋지게 요약하는 것이 아니라, 입력과 출력, 학습 신호, 검증 조건, baseline, 실패 모드, 공개 자원을 같은 기준으로 확인하는 데 있다. 이후 장에서는 이 13개 질문을 각 주제별로 적용한다. Action representation 장에서는 질문 5와 6이 중심이 되고, benchmark 장에서는 질문 7-9가 중심이 되며, robustness 장에서는 질문 11과 12가 특히 중요해진다.

Key papers for practicing the template

- Kim et al. (2024), OpenVLA: architecture, dataset mixture, fine-tuning, 공개 자원을 한 번에 점검하기 좋다.
- Kim, Finn, and Liang (2025), OpenVLA-OFT: 같은 base model에서 fine-tuning recipe가 claim을 어떻게 바꾸는지 보기 좋다.
- Pertsch et al. (2025), FAST: action tokenization claim과 closed-loop evidence를 분리해서 읽는 연습에 적합하다.
- Open X-Embodiment Collaboration (2023), Open X-Embodiment: dataset paper에서 claim, split, embodiment, source 공개를 확인하는 연습에 적합하다.
- Google DeepMind (2025/2026), Gemini Robotics page/report: closed industrial VLA에서 공개되지 않은 정보와 확인 가능한 정보를 구분하는 연습에 적합하다.

Reading assignment [Basic]

같은 논문에 대해 13-question summary와 6-line quick note를 각각 작성하고, 두 형식 중 연구 주제 발굴에 더 도움이 된 항목이 무엇인지 설명하라.

Part II

Core Bottlenecks

Chapter 4

Action Representation: VLA의 진짜 병목

4.1 장의 목표

VLA에서 가장 과소평가되기 쉬운 부분은 행동 표현(action representation)이다. 비전과 언어 backbone이 아무리 강해도, 로봇 행동을 잘못 표현하면 정책은 실제 제어에서 불안정해진다. 2024–2026년 VLA 논문군에서 action tokenization, action chunking, trajectory autoregressive modeling, diffusion action decoder가 계속 등장한 이유는 명확하다. VLA의 병목은 단순히 “무엇을 이해하는가”가 아니라 “이해한 것을 어떤 제어 변수로 바꾸는가”에 있다.

이 장은 2장에서 정의한 action space를 더 깊게 다룬다. 목표는 다음 네 질문에 답하는 것이다.

1. continuous action과 discrete action token은 각각 어떤 장단점이 있는가?
2. action chunking은 왜 속도 문제와 안정성 문제를 동시에 건드리는가?
3. diffusion 또는 flow 기반 행동 생성은 VLA에 왜 적합한가?
4. 좋은 action representation 논문을 평가하려면 어떤 실험이 필요한가?

4.2 행동 표현이 어려운 이유

언어는 이산 token sequence이다. 이미지는 patch token sequence로 바꿀 수 있다. 그러나 로봇 행동은 대개 연속값이며, 시간적으로 부드러워야 하고, robot embodiment와 controller에 의존한다. 이 차이가 VLA의 핵심 mismatch이다.

언어 모델 관점에서는 다음 예측이 자연스럽다.

$$p_{\theta}(w_{1:L} \mid I, x) = \prod_{\ell=1}^L p_{\theta}(w_{\ell} \mid w_{<\ell}, I, x), \quad (4.1)$$

여기서 w_{ℓ} 은 단어 또는 subword token이다. 반면 로봇 정책은 다음과 같은 연속 제어 문제에 가깝다.

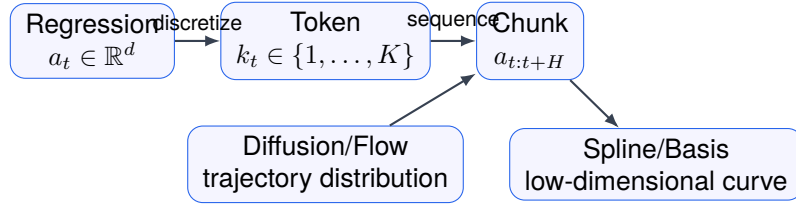


Figure 4.1: Author-created schematic. Action representation map. Regression, token, chunk, diffusion/flow, spline은 서로 배타적 분류가 아니라 VLA가 행동을 압축하고 복원하는 서로 다른 interface이다.

$$\mathbf{a}_t \in \mathbb{R}^{d_a}, \quad \mathbf{a}_{t+1} - \mathbf{a}_t \text{ is usually small.} \quad (4.2)$$

즉, 로봇 행동은 값의 정확도와 시간적 smoothness가 모두 중요하다. 이산 token 예측을 그대로 가져오면 LLM/VLM 구조와는 잘 맞지만 quantization error가 생긴다. 연속 회귀를 쓰면 제어 변수에는 직접 맞지만 multi-modal 행동을 표현하기 어렵다.

4.3 연속 행동 회귀

가장 직접적인 방식은 정책이 연속 행동을 회귀하는 것이다.

$$\hat{\mathbf{a}}_t = \mu_\theta(\mathbf{h}_t, \ell), \quad \mathbf{a}_t \in \mathbb{R}^{d_a}. \quad (4.3)$$

학습은 평균제곱오차로 할 수 있다.

$$\mathcal{L}_{\text{reg}} = \sum_{i,t} \left\| \mathbf{a}_t^{(i)} - \mu_\theta(\mathbf{h}_t^{(i)}, \ell^{(i)}) \right\|_2^2. \quad (4.4)$$

장점은 단순성과 속도이다. policy head가 한 번의 forward pass로 행동을 내므로 real-time control에 유리하다. 또한 end-effector delta pose, joint velocity, gripper command 같은 기존 controller interface와 바로 연결된다.

약점은 multi-modality이다. 같은 관측과 같은 명령에서도 여러 행동이 모두 타당할 수 있다. 컵을 왼쪽으로 돌아 잡을 수도 있고 오른쪽으로 돌아 잡을 수도 있다. MSE loss는 그 평균을 선호하므로 물리적으로 어색한 중간 행동이 나올 수 있다. 이를 평균 행동 문제라고 부를 수 있다.

평균 행동 문제

MSE 기반 연속 회귀는 demonstration distribution이 multi-modal일 때 각 mode 사이의 평균을 예측할 수 있다. 로봇에서는 이 평균이 충돌, 미끄러짐, 불안정한 grasp로 이어질 수 있다.

4.4 이산 action token

이산 action token 방식은 연속 행동을 codebook index로 바꾼다.

$$u_t = Q(\mathbf{a}_t), \quad u_t \in \{1, \dots, K\}. \quad (4.5)$$

정책은 action token을 autoregressive하게 예측한다.

$$p_\theta(u_{1:L} \mid \mathbf{h}_t, \ell) = \prod_{\ell=1}^L p_\theta(u_\ell \mid u_{<\ell}, \mathbf{h}_t, \ell). \quad (4.6)$$

이 방식의 매력은 VLM/LLM 구조와 잘 맞는다는 것이다. 언어 token, visual token, action token을 같은 sequence modeling 문제로 만들 수 있다. VQ-VLA류 논문은 이 지점에서 출발한다. 행동을 token으로 바꾸면 pretrained language model의 autoregressive machinery를 활용하기 쉽다.

그러나 action token에는 세 가지 문제가 있다. 첫째, codebook이 실제 로봇 행동의 geometry를 잘 보존해야 한다. 둘째, token sequence가 길어지면 inference latency가 증가한다. 셋째, token prediction error가 continuous control error로 어떻게 전파되는지 분석해야 한다.

Quantization error는 다음처럼 정의할 수 있다.

$$\epsilon_Q = \mathbb{E} \left[\|\mathbf{a} - Q^{-1}(Q(\mathbf{a}))\|_2^2 \right]. \quad (4.7)$$

하지만 ϵ_Q 가 낮다고 항상 좋은 것은 아니다. 중요한 것은 task-relevant error이다. 예를 들어 gripper command의 작은 오류는 치명적일 수 있지만, 이동 중 회전 각도의 작은 차이는 큰 문제가 아닐 수 있다. 따라서 action tokenizer는 단순 reconstruction error뿐 아니라 closed-loop success로 평가해야 한다.

이를 engineering diagnostic으로 쓰면 task-weighted action error를 다음처럼 볼 수 있다.

$$\epsilon_{\text{task}} = \mathbb{E}_{a \sim D} \left[(a - \hat{a})^\top W_{\text{task}} (a - \hat{a}) \right]. \quad (4.8)$$

여기서 W_{task} 는 실제 논문 objective가 아니라 action error의 task relevance를 설명하기 위한 conceptual weight이다. Contact phase, gripper timing, end-effector height error처럼 실패 비용이 큰 차원에는 더 큰 weight가 들어간다.

4.5 Action chunking

Action chunking은 한 번에 여러 step의 행동을 생성하는 방식이다.

$$\mathbf{A}_t = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}) \sim \pi_\theta(\cdot \mid \mathbf{h}_t, \ell). \quad (4.9)$$

이 방식은 VLA에서 특히 중요하다. 큰 VLA 모델은 매 제어 step마다 호출하기 어렵다. H step chunk를 내면 모델 호출 빈도를 줄일 수 있다. controller frequency가 r Hz이고 chunk length가 H 이면 모델 호출 frequency는 대략 r/H 가 된다.

$$f_{\text{model}} \approx \frac{r}{H}. \quad (4.10)$$

그러나 H 를 크게 잡으면 환경 변화에 반응이 늦어진다. 따라서 chunking은 latency와 reactivity의 trade-off이다.

$$J(H) = \text{SR}(H) - \lambda_1 \tau_{\text{model}}(H) - \lambda_2 \text{Staleness}(H). \quad (4.11)$$

$\text{Staleness}(H)$ 는 chunk가 길어질수록 현재 관측과 실행 중 action이 어긋나는 정도를 나타낸다. 좋은 action chunking 논문은 H 를 고정하지 않고, task 난이도와 환경 변화 속도에 따른 민감도를 보여야 한다.

4.6 Trajectory autoregressive modeling

Trajectory autoregressive modeling은 단일 행동보다 긴 trajectory를 순차적으로 생성한다. action chunking과 비슷하지만, chunk 내부의 구조를 sequence model로 더 명시적으로 다룬다.

$$p_{\theta}(\mathbf{A}_t \mid \mathbf{h}_t, \ell) = \prod_{h=0}^{H-1} p_{\theta}(\mathbf{a}_{t+h} \mid \mathbf{a}_{t:t+h-1}, \mathbf{h}_t, \ell). \quad (4.12)$$

이 방식은 행동 사이의 시간적 의존성을 표현할 수 있다. 예를 들어 grasp 접근, gripper close, lift, place는 순서가 중요하다. 단일 step 정책은 이 순서를 암묵적으로만 학습하지만 trajectory model은 chunk 내부 순서를 직접 모델링한다.

문제는 compounding error이다. 초반 행동이 틀리면 뒤 행동 조건도 틀어진다. 따라서 trajectory autoregressive 모델은 teacher forcing과 closed-loop rollout 사이의 gap을 확인해야 한다.

$$\Delta_{\text{rollout}} = \mathbb{E}[\ell_{\text{closed}}] - \mathbb{E}[\ell_{\text{teacher}}]. \quad (4.13)$$

Δ_{rollout} 가 크면 offline training loss가 낮아도 실제 실행에서 무너질 수 있다.

4.7 Diffusion action decoder

Diffusion 방식은 행동 trajectory를 확률적으로 생성한다. 초기 noise trajectory $\mathbf{A}^K$ 에서 시작해 denoising 과정을 거쳐 실행할 trajectory $\mathbf{A}^0$ 를 얻는다.

$$\mathbf{A}^{k-1} = D_{\theta}(\mathbf{A}^k, k, c_t), \quad k = K, \dots, 1. \quad (4.14)$$

여기서 c_t 는 시각, 언어, 로봇 상태를 결합한 context이다. Diffusion action decoder의 장점은 multi-modal trajectory distribution을 표현하기 쉽다는 점이다. 같은 목표를 여러 방식으로 달성할 수 있을 때, 하나의 평균 trajectory가 아니라 가능한 trajectory sample을 생성할 수 있다.

단점은 inference cost이다. K 번 denoising하면 모델 호출이 많아진다. 따라서 VLA에서 diffusion을 쓰려면 step 수를 줄이거나, flow matching, consistency model, distillation 같은 방법으로 빠르게 만들어야 한다. 2025–2026년 flow-based policy와 real-time diffusion policy가 중요해진 이유가 여기에 있다.

4.8 B-spline과 저차원 행동 기저

행동 trajectory를 모든 시점의 action으로 표현하면 차원이 커진다. B-spline 같은 기저 함수를 쓰면 trajectory를 몇 개의 control point로 표현할 수 있다.

$$\mathbf{a}(t) = \sum_{j=1}^M \mathbf{c}_j B_j(t), \quad (4.15)$$

여기서 $\mathbf{c}_j$ 는 control point이고 $B_j(t)$ 는 basis function이다. 이 방식은 행동을 부드럽게 만들고 token 수를 줄일 수 있다. BEAST류 접근은 이런 저차원 trajectory encoding의 장점을 활용한다.

그러나 spline 표현은 모든 과제에 맞지는 않는다. 접촉이 급격히 바뀌는 과제에서는 너무 부드러운 trajectory가 오히려 불리할 수 있다. 예를 들어 물체를 밀거나 삽입하는 과제에서는 순간적인 force change가 필요할 수 있다. 따라서 trajectory basis는 task dynamics와 함께 평가해야 한다.

4.9 행동 표현과 embodiment

행동 표현은 robot embodiment와 분리할 수 없다. 같은 end-effector delta pose라도 Franka arm, UR5, xArm에서 controller와 workspace가 다르다. Humanoid나 mobile manipulator로 가면 action space는 더 복잡해진다. Multi-embodiment VLA의 핵심 문제는 서로 다른 로봇의 action을 공통 표현으로 바꾸는 것이다.

이를 추상적으로 쓰면 각 robot r 에 대해 action mapping이 있다.

$$\mathbf{a}_t^{(r)} = M_r(z_t), \quad (4.16)$$

여기서 z_t 는 embodiment-independent latent action이고 M_r 은 robot-specific decoder이다. 좋은 multi-embodiment VLA는 z_t 가 task semantics를 담고, M_r 이 embodiment constraint를 처리하도록 설계해야 한다. 그렇지 않으면 단순히 여러 로봇 데이터를 섞은 policy가 되어 일반화가 약해질 수 있다.

4.10 평가 기준

Action representation 논문은 다음 실험을 갖추어야 한다.

1. 같은 backbone과 같은 데이터에서 action representation만 바꾼 비교
2. reconstruction error와 task success의 관계 분석
3. chunk length, codebook size, trajectory horizon 민감도
4. inference latency와 memory 사용량
5. closed-loop rollout에서의 failure mode
6. unseen task와 unseen object에서의 generalization

특히 reconstruction error만으로는 부족하다. 행동 표현이 좋아졌다는 주장은 실제 task success와 안정성으로 달혀야 한다.

Table 4.1: 행동 표현 방식의 비교

표현	장점	약점	확인할 실험
Continuous regression	빠르고 controller와 직접 연결됨	multi-modal 행동에 약함	MSE와 closed-loop success 비교
Action token	VLM/LLM 구조와 잘 맞음	quantization error와 긴 sequence 문제	codebook size, reconstruction, success
Action chunk	모델 호출 빈도 감소	반응성 저하 가능	chunk length ablation
Diffusion trajectory	multi-modal trajectory 표현력	denoising latency	step 수와 success/latency trade-off
Spline basis	부드럽고 compact 한 표현	급격한 접촉 변화에 약함	contact-rich task 평가

4.11 요약

VLA의 행동 표현은 단순 구현 세부사항이 아니라 모델의 성공률, 속도, 일반화, 안전성을 동시에 결정하는 핵심 설계이다. Continuous regression은 빠르지만 multi-modality에 약하고, action token은 VLM 구조와 잘 맞지만 quantization 문제가 있으며, action chunking은 latency를 줄이지만 반응성을 희생할 수 있다. Diffusion과 flow 기반 정책은 trajectory distribution을 풍부하게 표현하지만 real-time constraint를 해결해야 한다.

다음 장에서는 이런 행동 표현을 가진 VLA를 실제 과제에 맞추는 fine-tuning과 post-training 문제를 다룬다. Action representation이 “무엇을 출력할 것인가”의 문제라면, fine-tuning은 “그 출력을 특정 데이터와 과제에 어떻게 맞출 것인가”의 문제이다.

Case study: FAST와 VQ-VLA

FAST는 per-dimension, per-timestep binning이 high-frequency dexterous data에서 약하다는 문제를 지적하고 DCT 기반 Frequency-space Action Sequence Tokenization을 제안한다. 핵심은 language model에 맞는 discrete token을 쓰되, continuous trajectory의 주파수 구조를 보존하려는 것이다. VQ-VLA는 vector-quantized action tokenizer를 대규모 synthetic/real trajectory로 scaling하여 smoother action과 real-world long-horizon success 개선을 주장한다. 두 논문 모두 “token이 좋다”가 아니라 “tokenization이 closed-loop execution을 얼마나 덜 망가뜨리는가”로 읽어야 한다.

Table 4.2: Claim-source-evidence-caution map for action representation

Claim	Source	Evidence	Caution
Action tokenizer는 독립 병목이다	FAST [16], VQ-VLA [21]	DCT tokenization, VQ action tokenizer scaling, closed-loop 또는 long-horizon 결과	Tokenizer-only effect가 backbone, data, controller 변화와 분리되었는지 확인한다.
Diffusion은 multi-modal action에 강하다	Diffusion Policy [5]	Visuomotor trajectory denoising과 real-robot manipulation evidence	VLA backbone 효과가 아니라 action decoder 효과라는 점을 분리한다.
Chunking은 long-horizon imitation에 유용하다	ACT/ALOHA [23]	Bimanual manipulation에서 action chunk prediction과 hardware demonstration	Language grounding이 약한 predecessor이므로 VLA 대표 논문으로 과대분류하지 않는다.
Reconstruction error 만으로는 부족하다	FAST/VQ-VLA plus task results	Token error와 closed-loop success를 함께 봐야 함	Low reconstruction error가 contact-rich task success를 보장하지 않는다.

Key papers

- Chi et al. (2023), Diffusion Policy: multimodal continuous action distribution을 trajectory denoising으로 다루는 핵심 선행 논문이다.
- Zhao et al. (2023), ACT/ALOHA: action chunking이 long-horizon bimanual manipulation에서 왜 유용한지 보여 준다.
- Pertsch et al. (2025), FAST, arXiv preprint: DCT 기반 action tokenization과 FAST+ universal tokenizer를 제안한다.
- Wang et al. (2025), VQ-VLA, ICCV 2025: vector-quantized action tokenizer scaling과 real-world long-horizon 개선을 주장한다.
- Kim, Finn, and Liang (2025), OpenVLA-OFT, RSS 2025: continuous action representation, action chunking, L1 objective가 fine-tuning 성능에 주는 영향을 연결한다.

Reading assignment [Intermediate]

FAST와 VQ-VLA를 비교해 codebook/tokenizer가 줄이는 error가 reconstruction error인지, closed-loop task error인지, latency인지 구분하라.

Chapter 5

Fine-Tuning과 Post-Training: VLA를 과제에 맞추는 방법

5.1 장의 목표

4장에서는 VLA가 무엇을 출력해야 하는지, 즉 행동 표현을 다루었다. 5장은 그 출력이 실제 과제에서 잘 작동하도록 모델을 어떻게 맞추는지 다룬다. VLA의 성능은 pretrained backbone만으로 결정되지 않는다. 같은 backbone이라도 어떤 데이터로 fine-tuning했는지, action head를 어떻게 초기화했는지, adapter를 어디에 넣었는지, post-training feedback을 어떻게 주었는지에 따라 success rate와 latency가 크게 달라진다.

이 장에서는 VLA 학습 과정을 다섯 단계로 구분한다.

1. pretraining: 일반 시각-언어 또는 로봇 prior를 얻는 단계
2. robot adaptation: action space와 embodiment에 맞추는 단계
3. supervised fine-tuning: demonstration을 이용해 task policy를 맞추는 단계
4. parameter-efficient tuning: adapter, LoRA류 모듈만 학습하는 단계
5. post-training: reward, verification, interaction, simulator feedback으로 실행 품질을 보정하는 단계

5.2 Pretraining과 robot adaptation

Pretraining은 VLA가 세상의 객체, 언어 표현, 장면 구조, 일부 행동 prior를 배우는 단계이다. VLM 기반 VLA에서는 대규모 image-text data가 semantic prior를 제공한다. Robot foundation model 계열에서는 여러 로봇 trajectory를 함께 학습해 행동 prior를 얻는다.

그러나 pretraining만으로는 특정 로봇의 action space를 바로 맞추기 어렵다. 로봇 r 의 action space를 $\mathcal{A}^{(r)}$ 라고 하면, pretrained model이 사용하는 latent representation z_t 를 실제 행동으로 바꾸는 decoder가 필요하다.

$$\mathbf{a}_t^{(r)} = D_{\phi}^{(r)}(z_t). \quad (5.1)$$

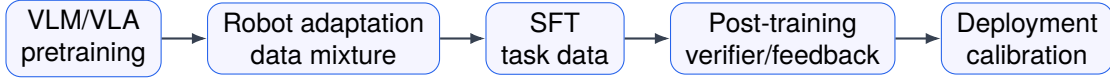


Figure 5.1: Author-created schematic. VLA fine-tuning pipeline. 공개용 원고에서는 pretraining, robot adaptation, supervised fine-tuning, post-training, deployment calibration을 명확히 구분한다.

Robot adaptation은 $D_{\phi}^{(r)}$ 를 특정 embodiment에 맞추는 단계이다. 이 단계가 약하면 backbone은 장면을 잘 이해하지만 행동이 robot controller와 맞지 않는다. OpenVLA류 모델을 실제 과제에 fine-tuning할 때 action head와 normalization, controller interface가 중요한 이유가 여기에 있다.

5.3 Supervised fine-tuning

가장 기본적인 fine-tuning은 demonstration dataset을 이용한 supervised learning이다.

$$\mathcal{D}_{\text{task}} = \{(\mathbf{o}_{1:T_i}^{(i)}, \ell^{(i)}, \mathbf{a}_{1:T_i}^{(i)})\}_{i=1}^N. \quad (5.2)$$

continuous action이면 다음 loss를 쓴다.

$$\mathcal{L}_{\text{SFT}} = \sum_{i,t} \left\| \mathbf{a}_t^{(i)} - \pi_{\theta}(\mathbf{h}_t^{(i)}, \ell^{(i)}) \right\|_2^2. \quad (5.3)$$

action token이면 cross entropy가 된다.

$$\mathcal{L}_{\text{SFT-token}} = - \sum_{i,t} \log p_{\theta}(u_t^{(i)} | \mathbf{h}_t^{(i)}, \ell^{(i)}). \quad (5.4)$$

이 단계의 핵심은 data split이다. train task와 test task가 너무 비슷하면 fine-tuning 성능은 높아 보이지만 일반화 주장은 약해진다. 따라서 VLA fine-tuning 논문은 seen task, unseen object, unseen scene, unseen instruction을 분리해야 한다.

5.4 Full fine-tuning과 frozen backbone

Full fine-tuning은 backbone까지 모두 업데이트한다. 장점은 task adaptation 능력이 크다는 점이다. 약점은 계산 비용과 catastrophic forgetting이다. VLM backbone이 가진 넓은 semantic prior가 작은 robot dataset에 과적합될 수 있다.

Frozen backbone 방식은 pretrained encoder 또는 language model을 고정하고 action head만 학습한다.

$$\theta = (\theta_{\text{frozen}}, \phi_{\text{head}}), \quad \nabla_{\theta_{\text{frozen}}} \mathcal{L} = 0. \quad (5.5)$$

이 방식은 안정적이고 비용이 낮지만, pretrained representation이 robot action에 충분히 맞지 않으면 성능이 제한된다. 실제 연구에서는 full fine-tuning과 frozen backbone 사이에 adapter, LoRA, partial fine-tuning이 위치한다.

Table 5.1: VLA fine-tuning 방식의 비교

방식	장점	약점	확인할 실험
Full fine-tuning	task adaptation 강함	비용과 forgetting 위험	data size ablation
Frozen backbone	안정적이고 저비용	action mismatch 가능	head-only baseline
Adapter tuning	비용과 적응력 균형	adapter 위치에 민감	adapter placement ablation
LoRA 류 tuning	parameter-efficient	rank 선택에 민감	rank/latency/성능 비교
Post-training	실행 품질 보정 가능	reward 나 verifier bias 가능	real rollout 검증

5.5 Adapter와 LoRA류 접근

Adapter 기반 VLA는 큰 backbone을 대부분 고정하고 작은 학습 가능 모듈을 삽입한다. 이를 단순화하면 한 transformer layer에서 다음과 같이 쓸 수 있다.

$$h_{\ell+1} = F_{\ell}(h_{\ell}) + A_{\psi}(h_{\ell}), \quad (5.6)$$

여기서 F_{ℓ} 은 pretrained layer이고 A_{ψ} 는 작은 adapter이다. Adapter는 robot-specific signal을 넣기 좋은 위치를 제공한다. VLA-Adapter류 논문은 이 관점에서 읽을 수 있다.

LoRA류 방법은 weight update를 low-rank로 제한한다.

$$W' = W + BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}. \quad (5.7)$$

r 이 작으면 학습 파라미터 수가 줄어든다. 그러나 rank가 너무 작으면 robot action adaptation이 부족할 수 있다. 따라서 parameter-efficient tuning 논문은 parameter count만 보고해서는 부족하고, success rate, latency, memory, forgetting을 함께 보여야 한다.

5.6 Instruction tuning from understanding to manipulation

VLA instruction tuning의 어려움은 언어 이해와 행동 실행이 같은 것이 아니라는 점이다. “서랍을 열어라”라는 지시는 텍스트 응답으로는 간단하지만, 로봇에게는 handle detection, approach, grasp, pull trajectory, force control을 요구한다.

Instruction tuning dataset은 다음처럼 생각할 수 있다.

$$\mathcal{D}_{\text{inst}} = \{(\ell^{(i)}, \mathbf{o}_{1:T_i}^{(i)}, \mathbf{a}_{1:T_i}^{(i)}, y^{(i)})\}, \quad (5.8)$$

여기서 $y^{(i)}$ 는 optional language rationale, subgoal, success label일 수 있다. 중요한 것은 y 가 실제 행동 개선에 도움을 주는가이다. 많은 reasoning-style VLA는 explanation을 생성하지만, explanation이 closed-loop success를 올리는지는 별도 검증이 필요하다.

5.7 Post-training

Post-training은 supervised fine-tuning 이후 실행 품질을 보정하는 단계이다. VLA에서는 다음 유형이 중요하다.

1. interaction-based correction: 실행 중 실패를 보고 다시 학습한다.
2. simulator verified reward: 시뮬레이터에서 task success를 확인해 reward를 준다.
3. human feedback: 사람이 trajectory 또는 행동 결과를 평가한다.
4. preference optimization: 더 좋은 trajectory를 선호하도록 학습한다.
5. safety-aware correction: 위험 행동을 penalize한다.

이를 reward 기반 objective로 쓰면 다음과 같다.

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau, \ell) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})]. \quad (5.9)$$

π_{ref} 는 supervised fine-tuned reference policy이다. KL term은 post-training이 기존 policy를 너무 멀리 벗어나지 않게 한다. VLA-RFT류 접근은 이 식의 실제 구현으로 볼 수 있다.

5.8 Verifier와 reward의 위험

Post-training에서 가장 중요한 위험은 reward hacking이다. Verifier가 성공을 잘못 판단하면 policy는 실제 과제 성공이 아니라 verifier를 속이는 방향으로 학습할 수 있다. 시뮬레이터 reward도 마찬가지이다. 시뮬레이터의 물리, object model, contact dynamics가 실제와 다르면 post-training된 policy가 real robot에서 약할 수 있다.

따라서 reward 기반 VLA 논문은 다음 질문에 답해야 한다.

1. reward 또는 verifier가 무엇을 기준으로 success를 판단하는가?
2. reward가 language instruction의 모든 제약을 반영하는가?
3. simulator와 real robot 사이의 gap을 측정했는가?
4. post-training 후 unsafe behavior가 늘지 않았는가?
5. reference policy 대비 어떤 failure mode가 줄었는가?

5.9 데이터 효율과 scaling

Fine-tuning 논문은 data efficiency를 보여야 한다. 단순히 많은 데이터를 넣어 좋아졌다면 방법의 contribution인지 데이터 규모의 효과인지 구분하기 어렵다. 데이터 개수 N 에 따른 성능을 다음 처럼 볼 수 있다.

$$\text{SR}(N) = a - bN^{-\alpha}. \quad (5.10)$$

α 가 클수록 데이터 증가에 따른 성능 개선이 빠르다. 좋은 fine-tuning 방법은 같은 N 에서 더 높은 성공률을 보이거나, 같은 성공률을 더 작은 N 으로 달성해야 한다. 이를 sample efficiency라고 한다.

또한 fine-tuning은 speed와도 연결된다. 작은 adapter만 학습하면 학습 비용은 줄지만 inference 구조에 따라 latency가 늘 수도 있다. 따라서 training efficiency와 inference efficiency를 구분해야 한다.

5.10 평가 기준

Fine-tuning과 post-training 논문을 평가할 때는 다음 항목을 확인한다.

1. 어떤 파라미터가 학습되고 어떤 파라미터가 고정되는가?
2. action head와 action normalization은 어떻게 정의되는가?
3. train/test split은 task, object, scene, instruction 기준으로 분리되는가?
4. data size ablation이 있는가?
5. baseline은 full fine-tuning, frozen backbone, adapter-only를 포함하는가?
6. post-training reward나 verifier의 정확도는 검증되는가?
7. real robot rollout에서 improvement가 유지되는가?

5.11 요약

VLA fine-tuning은 pretrained model을 특정 로봇과 과제에 맞추는 과정이다. Full fine-tuning은 강하지만 비용과 forgetting 위험이 있고, frozen backbone은 안정적이지만 action mismatch가 남을 수 있다. Adapter와 LoRA류 방법은 비용과 적응력 사이의 절충을 제공한다. Post-training은 reward, verifier, interaction feedback을 통해 실행 품질을 보정하지만, reward hacking과 sim-to-real gap을 조심해야 한다.

다음 장에서는 fine-tuned VLA를 실제 제어 주기 안에서 실행하기 위한 효율성 문제를 다룬다. Action representation과 fine-tuning이 성능을 만든다면, efficient VLA는 그 성능을 실제 로봇 시간 안에 집어넣는 문제이다.

Formal objective와 engineering score의 구분

이 장의 수식은 많은 경우 논문이 실제로 최적화한 objective라기보다 fine-tuning design choice를 비교하기 위한 conceptual scoring function이다. $\mathcal{L}_{BC}$, cross entropy, diffusion loss처럼 실제 학습 loss와, success-latency-memory-risk를 묶은 engineering utility를 계속 구분해야 한다.

Table 5.2: Claim-source-evidence-caution map for fine-tuning

Claim	Source	Evidence	Caution
Open VLA backbone은 adaptation base가 될 수 있다	OpenVLA [10]	7B open VLA, 970k demonstrations, public model artifacts	Public checkpoint가 있어도 target robot controller adaptation은 별도 문제이다.
Fine-tuning recipe는 success와 speed를 함께 바꾼다	OpenVLA-OFT [11]	LIBERO 76.5%→97.1%, throughput 26× 보고	Throughput 정의, hardware, baseline protocol이 같은지 확인한다.
Generalist policy는 small-data adaptation의 출발점이다	Octo [14]	Open X-Embodiment 기반 800k trajectory pretraining과 fine-tuning interface	In-domain data mixture와 test split이 일반화 claim을 좌우한다.
Open-world generalization은 data mixture와 semantic prediction을 요구한다	$\pi_{0.5}$ [17]	Heterogeneous co-training, web data, multiple robot evidence	Emerging-front preprint이므로 source status와 artifact completeness를 표시한다.

Case study: OpenVLA-OFT

Kim, Finn, and Liang (2025)의 OpenVLA-OFT는 OpenVLA를 대표 base model로 두고 action decoding, action representation, learning objective를 체계적으로 바꾼다. OFT recipe는 parallel decoding, action chunking, continuous action representation, L1 regression objective를 결합해 LIBERO 평균 success를 76.5%에서 97.1%로 올리고 action generation throughput을 26× 높였다고 보고한다. 이 사례는 fine-tuning 장과 efficient VLA 장을 동시에 읽어야 하는 이유를 보여 준다.

Key papers

- Kim et al. (2024), OpenVLA: open VLA의 base model과 consumer GPU fine-tuning 가능성을 확인할 수 있다.
- Kim, Finn, and Liang (2025), OpenVLA-OFT, RSS 2025: fine-tuning recipe가 speed와 success를 동시에 바꾸는 대표 사례이다.
- Octo Model Team et al. (2024), Octo: in-domain data가 적을 때 generalist policy를 어떻게 fine-tuning하는지 볼 수 있다.
- Physical Intelligence et al. (2025), $\pi_{0.5}$: heterogeneous co-training, semantic prediction, web data를 결합한 open-world generalization 사례이다.
- NVIDIA et al. (2025), GR00T N1: humanoid VLA에서 real, synthetic, human video data mixture를 쓰는 adaptation 사례이다.

Reading assignment [Advanced]

OpenVLA-OFT의 improvement를 success, throughput, action representation, objective 네 축으로 분해하고, 어떤 변화가 model-centric이고 어떤 변화가 system-centric인지 표시하라.

Chapter 6

Efficient VLA: 빠르고 작은 VLA 만들기

6.1 장의 목표

VLA가 실제 로봇 정책이 되려면 성공률뿐 아니라 시간 안에 행동을 내야 한다. 큰 VLM backbone을 쓰는 VLA는 semantic prior가 강하지만, 제어 주기 관점에서는 느릴 수 있다. 로봇은 모델이 생각을 끝낼 때까지 기다려 주지 않는다. 센서 입력, 전처리, 모델 추론, action decoding, safety filter, controller command가 모두 한 loop 안에 들어와야 한다.

이 장은 VLA 효율성을 네 층으로 나누어 다룬다.

1. system-level latency: 전체 제어 loop의 시간 예산
2. model-level efficiency: backbone, token 수, expert routing, pruning
3. decoding-level efficiency: speculative decoding, action chunking, diffusion step 감소
4. deployment-level efficiency: GPU memory, batch size, controller interface

6.2 Latency budget

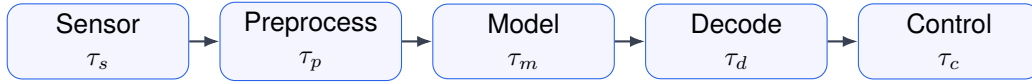
VLA의 효율성은 단순한 FPS가 아니다. 실제 제어 loop의 시간 예산은 다음과 같다.

$$\tau_{\text{loop}} = \tau_{\text{sense}} + \tau_{\text{pre}} + \tau_{\text{model}} + \tau_{\text{decode}} + \tau_{\text{safe}} + \tau_{\text{ctrl}}. \quad (6.1)$$

Real-time VLA를 주장하려면 τ_{loop} 가 controller 요구 주기보다 작아야 한다.

$$\tau_{\text{loop}} \leq \frac{1}{f_{\text{ctrl}}}. \quad (6.2)$$

예를 들어 10 Hz closed-loop policy를 원하면 전체 loop가 100 ms 안에 끝나야 한다. 모델 forward pass만 80 ms라면 sensor와 safety filter를 포함했을 때 이미 위험하다. 따라서 효율성 논문은 τ_{model} 뿐 아니라 end-to-end latency를 보고해야 한다.



$$\tau_s + \tau_p + \tau_m + \tau_d + \tau_c \leq T_{\text{loop}}$$

Figure 6.1: Author-created schematic. Latency budget for VLA deployment. 효율성 주장은 model forward time만이 아니라 sensor, preprocessing, decoding, safety/controller overhead를 포함해 평가해야 한다.

6.3 Token 수가 왜 중요한가

Transformer 기반 VLA에서 계산량은 token 수에 크게 의존한다. self-attention의 기본 계산량은 token 수 n 에 대해 대략 $O(n^2)$ 이다.

$$C_{\text{attn}} \propto n^2 d. \quad (6.3)$$

VLA는 이미지 patch token, 언어 token, state token, action token을 모두 다룰 수 있다. Multi-view image를 쓰면 visual token 수가 급격히 늘어난다. 따라서 token pruning이나 token merging은 단순 최적화가 아니라 VLA 실시간성의 핵심 전략이다.

중요한 점은 모든 token이 행동에 똑같이 중요하지 않다는 것이다. “빨간 컵을 잡아라”라는 명령에서 배경, 벽, 관련 없는 객체 token은 action 결정에 덜 중요할 수 있다. 효율적 VLA는 task-relevant token을 남기고 나머지 계산을 줄이려 한다.

6.4 Sparsification과 routing

Sparsification은 계산에 참여하는 token, channel, layer, expert를 줄이는 방법이다. 가장 단순하게는 mask $m_i \in \{0, 1\}$ 를 두고 token representation h_i 를 선택한다.

$$\tilde{h}_i = m_i h_i. \quad (6.4)$$

선택된 token 수가 $n_s = \sum_i m_i$ 이면 attention cost는 n_s^2 에 비례한다. 만약 $n_s \ll n$ 이면 계산량이 크게 줄 수 있다. 그러나 token을 잘못 버리면 목표 객체나 중요한 spatial relation을 잃는다. 따라서 sparsification은 success rate와 함께 평가해야 한다.

Routing은 입력 조건에 따라 expert나 path를 선택한다.

$$y = \sum_{e=1}^E g_e(x) F_e(x), \quad (6.5)$$

여기서 $g_e(x)$ 는 routing weight이고 F_e 는 expert이다. CogVLA류 논문은 cognition-aligned routing과 sparsification을 통해 필요한 계산만 쓰려는 방향으로 볼 수 있다. 핵심 질문은 routing이 단순히 빠른가가 아니라, task-relevant reasoning path를 실제로 선택하는가이다.

6.5 Adapter로 작게 맞추기

VLA-Adapter류 접근은 큰 backbone을 그대로 두고 작은 모듈만 학습한다. 학습 비용을 줄이는 효과가 크지만, inference 비용은 adapter 구조에 따라 달라진다. Adapter가 모든 layer에 들어가면 parameter 수는 작아도 latency는 늘 수 있다.

따라서 adapter 효율성을 평가할 때는 다음 네 지표를 분리해야 한다.

1. trainable parameter ratio
2. training GPU memory
3. inference latency
4. task success rate

작은 adapter가 좋다는 주장은 이 네 항목을 함께 보여야 달한다. parameter-efficient가 반드시 deployment-efficient는 아니다.

6.6 Speculative decoding

Speculative decoding은 작은 draft model이 후보 action token을 먼저 만들고, 큰 target model이 이를 검증하는 방식이다. 언어 모델에서 쓰이던 아이디어를 VLA action decoding에 적용할 수 있다.

작은 모델 q 가 후보 sequence $\tilde{u}_{1:L}$ 를 생성하고, 큰 모델 p 가 acceptance를 결정한다고 하자.

$$\tilde{u}_{1:L} \sim q(u_{1:L} \mid c_t), \quad a_\ell = \min \left(1, \frac{p(\tilde{u}_\ell \mid \tilde{u}_{<\ell}, c_t)}{q(\tilde{u}_\ell \mid \tilde{u}_{<\ell}, c_t)} \right). \quad (6.6)$$

VLA에서 문제는 relaxed acceptance이다. 언어 token은 틀리면 문장이 조금 이상해질 수 있지만, action token은 작은 차이가 물리적으로 큰 실패를 만들 수 있다. 따라서 Spec-VLA류 접근은 token-level acceptance뿐 아니라 continuous action error와 task success를 함께 봐야 한다.

6.7 Diffusion step 줄이기

Diffusion action decoder는 표현력이 좋지만 step 수가 많으면 느리다. K step denoising을 하면 대략 K 번의 model call이 필요하다.

$$\tau_{\text{diff}} \approx K \tau_{\text{denoise}}. \quad (6.7)$$

이를 줄이는 방법은 세 가지이다.

1. denoising step 수를 줄인다.
2. consistency distillation으로 one-step 또는 few-step generation을 만든다.
3. flow matching으로 더 직접적인 trajectory mapping을 학습한다.

FreqPolicy나 efficient flow-based policy 계열은 이 문제의식과 연결된다. 좋은 논문은 step 수를 줄였을 때 success rate가 얼마나 떨어지는지, 그리고 real-time loop를 만족하는지 보여야 한다.

Table 6.1: Efficient VLA 평가 기준

항목	확인할 내용
End-to-end latency	sensor, preprocessing, model, decoding, safety, controller 포함 여부
Model latency	backbone forward, action head, decoding step 별 시간
Success-latency curve	같은 성공률에서 더 빠르지, 같은 latency에서 더 성공적인지
Memory	parameter, activation, cache, buffer 포함 여부
Ablation	token 수, chunk length, expert 수, diffusion step 수 변화
Hardware	GPU/CPU 종류, batch size, precision, robot control frequency
Failure mode	빨라지면서 어떤 실패가 늘었는지

6.8 Action chunking과 효율성

4장에서 action chunking은 행동 표현 관점에서 다루었다. 여기서는 효율성 관점에서 본다. H step action chunk를 생성하면 모델 호출 빈도는 줄어든다.

$$N_{\text{calls}} = \left\lceil \frac{T}{H} \right\rceil. \quad (6.8)$$

그러나 chunk를 길게 만들면 policy가 현재 관측을 덜 자주 반영한다. 따라서 efficient VLA는 단순히 큰 H 를 쓰는 것이 아니라, task phase에 따라 adaptive chunking을 고려할 수 있다. 접근 구간에서는 큰 chunk를 쓰고, 접촉 구간에서는 작은 chunk를 쓰는 방식이 더 합리적일 수 있다.

6.9 메모리와 quantization

실제 배포에서는 GPU memory도 중요하다. 모델 parameter, activation, KV cache, visual token buffer가 모두 memory를 사용한다. 특히 autoregressive action token decoding에서는 KV cache가 길이에 따라 증가한다.

$$M_{\text{total}} = M_{\text{param}} + M_{\text{act}} + M_{\text{cache}} + M_{\text{buffer}}. \quad (6.9)$$

Quantization은 M_{param} 를 줄이는 대표 방법이다. 그러나 로봇 정책에서 quantization은 action accuracy에 영향을 줄 수 있다. 언어 응답에서는 약간의 logit 변화가 큰 문제가 아닐 수 있지만, 로봇 action head에서는 작은 변화가 trajectory를 흔들 수 있다. 따라서 model quantization은 action error와 success rate로 검증해야 한다.

6.10 효율성 평가표

효율성 논문에서 가장 중요한 것은 trade-off curve이다. 단일 점으로 “빠르고 좋다”고 주장하면 부족하다. 성공률과 latency를 함께 그려야 한다.

Table 6.2: End-to-end latency measurement template

Stage	Time (ms)	Notes
Camera read		resolution, view count, synchronization
Preprocess		resize, crop, normalize, tokenization
VLA forward		precision, GPU/CPU, batch size, visual token count
Action decoding		autoregressive steps, diffusion/flow steps, chunk length
Safety filter		collision check, workspace limit, joint limit
Controller dispatch		ROS/control stack, network and actuator delay
Total		compare with target loop rate

6.11 요약

Efficient VLA는 모델을 작게 만드는 문제가 아니라, 로봇 제어 loop 안에 VLA를 넣는 문제이다. Token pruning, sparsification, routing, adapter, speculative decoding, diffusion step reduction, action chunking은 모두 latency budget을 맞추기 위한 서로 다른 도구이다. 그러나 속도만 빨라지면 충분하지 않다. 효율성 연구는 success rate, latency, memory, failure mode를 함께 보고해야 한다.

다음 장에서는 VLA가 가장 많이 검증되는 응용 영역인 manipulation을 다룬다. Efficient VLA가 시간 제약의 문제라면, manipulation은 물리 접촉과 행동 안정성의 문제이다.

Case study: SmolVLA와 real-time action policies

SmolVLA는 billion-scale VLA가 아닌 compact VLA의 가능성을 보여 주는 사례이다. 논문은 consumer GPU/CPU deployment를 목표로 single-GPU training, asynchronous inference stack, chunked action generation을 제시한다. 다만 이 주장을 검증할 때는 task coverage, hardware setting, benchmark comparability를 함께 확인해야 하며, 작은 모델이 좋다는 결론보다 perception/action prediction과 execution을 decouple하는 system design 문제로 읽는 편이 안전하다. Real-time action chunking flow policy 계열은 같은 질문을 flow/chunk representation 관점에서 다룬다.

Key papers

- Kim, Finn, and Liang (2025), OpenVLA-OFT: throughput 26× 개선과 success 개선을 함께 보고해 success-latency curve 논의의 좋은 사례이다.
- Shukor et al. (2025), SmolVLA, arXiv preprint: compact VLA, consumer hardware, asynchronous inference stack을 효율성 장의 중심 사례로 제공한다.
- Black et al. (2024/2026), π_0 : flow matching architecture와 dexterous control을 연결하는 general robot control anchor이다.
- Pertsch et al. (2025), FAST: tokenization을 통해 autoregressive VLA의 학습/추론 비용을 줄이는 접근이다.
- CogVLA 계열 sparsification 논문: instruction-conditioned routing으로 계산량과 성능을 조절하는 model/system 교차 사례이다.

Table 6.3: Claim-source-evidence-caution map for efficient VLA

Claim	Source	Evidence	Caution
Speed improvement must preserve success	OpenVLA-OFT [11]	Success and throughput are reported together	Hardware and decoding protocol must be comparable.
Small VLA is a system design question	SmolVLA [19]	Compact model, asynchronous inference, consumer hardware claims	Task coverage and benchmark comparability must be checked.
Tokenization affects both training and inference cost	FAST [16]	DCT-based action compression and tokenizer efficiency	Compression gain must be evaluated against physical execution error.
Flow policies trade sampling cost and control quality	π_0 [2]	Flow matching architecture for general robot control	Real-time claims depend on action horizon, hardware, and controller interface.

Reading assignment [Intermediate]

SmolVLA와 OpenVLA-OFT를 비교해 효율성 개선이 model size, decoding, action chunking, asynchronous system 중 어디에서 오는지 분해하라.

Part III

Evaluation in the Physical World

Chapter 7

Manipulation이 VLA의 주전장이 된 이유

6장에서는 VLA를 제어 loop 안에 넣기 위해 필요한 latency, memory, decoding step, action chunking의 문제를 다루었다. 이 장에서는 그 다음 병목인 물리 접촉을 다룬다. VLA 연구가 manipulation에 집중되는 이유는 단순히 실험실에 로봇팔이 많아서가 아니다. Manipulation은 언어 지시, 시각 인식, 공간 추론, 접촉 동역학, 페루프 보정이 한 과제 안에서 동시에 드러나는 영역이다. 따라서 VLA가 정말 robot policy인지, 아니면 그럴듯한 multimodal sequence model인지 가장 빨리 드러나는 시험장이다.

7.1 Manipulation 문제의 구조

Manipulation은 환경 안의 물체 상태를 바꾸는 행동이다. 이동 로봇이 자신의 상태 x_t 를 바꾸는 문제에 가깝다면, manipulation은 로봇 상태와 물체 상태를 함께 바꾸는 문제이다.

$$s_t = [x_t^{\text{robot}}, x_t^{\text{object}}, x_t^{\text{env}}], \quad a_t = [\Delta p_t, \Delta R_t, g_t]. \quad (7.1)$$

여기서 Δp_t 와 ΔR_t 는 end-effector의 병진 및 회전 명령이고, g_t 는 gripper 또는 hand command이다. 실제 시스템에서는 joint velocity, torque, impedance parameter, suction command, multi-finger command 등으로 더 세분화된다. 중요한 점은 action이 언어 token처럼 의미 단위로만 평가되지 않는다는 것이다. 같은 “컵을 집어라”라는 지시라도, 손끝 위치가 몇 cm 어긋나면 grasp가 실패하고, 회전이 잘못되면 충돌이 발생한다.

VLA 관점에서 manipulation task는 다음 mapping으로 볼 수 있다.

$$\pi_\theta : (I_{1:t}, q, h_{1:t}, r_t) \mapsto a_{t:t+H-1}. \quad (7.2)$$

$I_{1:t}$ 는 RGB 또는 RGB-D 관측, q 는 언어 지시, $h_{1:t}$ 는 proprioception과 과거 행동, r_t 는 optional retrieval 또는 memory context이다. 출력은 single action일 수도 있고, H 길이의 action chunk일 수도 있다. Manipulation VLA에서 핵심은 이 mapping이 물체, 관계, 접촉 가능성, 시간적 진행 상태를 함께 반영해야 한다는 점이다.

Table 7.1: Manipulation 과제 유형과 VLA가 요구받는 능력

과제 유형	대표 예시	VLA에 요구되는 능력
Pick-and-place	컵 집기, 블록 옮기기	물체 grounding, grasp pose 추론, 짧은 horizon 제어
Articulated object	서랍 열기, 문 손잡이 돌리기	joint 구조 추론, 접촉 유지, force 방향 제어
Tool use	주걱, 집게, 걸이 사용	도구 affordance, object-tool relation, 긴 horizon 계획
Deformable object	천 접기, 케이블 정리	형태 변화 예측, partial observability, 실패 복구
Dexterous manipulation	손가락 회전, in-hand regrasp	고차원 action, 접촉 안정성, tactile feedback
Bimanual manipulation	양손 조립, 큰 물체 운반	역할 분담, synchronization, collision avoidance

7.2 왜 pick-and-place에서 시작했는가

많은 VLA benchmark와 robot learning dataset은 pick-and-place, drawer opening, button pressing, object rearrangement에서 시작한다. 이유는 명확하다. 이 과제들은 언어로 명시하기 쉽고, 성공 여부를 비교적 명확히 측정할 수 있으며, 카메라와 로봇팔로 반복 수집하기 쉽다.

Pick-and-place가 쉬운 과제라는 뜻은 아니다. 다만 성공 조건이 비교적 분명하다. 물체가 목표 영역 안에 놓였는지, gripper가 물체를 떨어뜨리지 않았는지, 충돌이 없었는지를 판정할 수 있다. 그래서 VLA 논문은 초기에 “언어 지시를 보고 물체를 골라 실제 로봇이 움직일 수 있는가”를 보여주기 위해 pick-and-place를 자주 사용한다.

하지만 pick-and-place만으로 VLA의 실력을 판단하면 위험하다. 이 과제는 많은 경우 scene이 정리되어 있고, 물체 종류가 제한되어 있으며, 접촉 시간이 짧다. 실제 manipulation은 물체가 가려지고, 목표가 중간에 바뀌며, 접촉이 길게 유지되고, 실패 후 복구가 필요하다. 따라서 2024–2026년의 흐름은 단순 pick-and-place 성공률에서 long-horizon, contact-rich, dexterous, bimanual manipulation으로 확장되는 방향으로 읽어야 한다.

7.3 Language grounding과 affordance

Manipulation에서 언어는 목표를 지정한다. 그러나 로봇은 문장을 그대로 실행할 수 없다. 문장은 물체와 행동 가능성으로 grounding되어야 한다.

$$q \rightarrow (o^*, r^*, \phi^*), \quad (7.3)$$

여기서 o^* 는 대상 물체, r^* 는 다른 물체 또는 장소와의 관계, ϕ^* 는 필요한 조작 primitive 또는 affordance이다. 예를 들어 “빨간 컵을 집시 오른쪽에 놓아라”라는 지시는 빨간 컵의 segmentation, 집시의 위치, 오른쪽이라는 관계, 컵을 잡고 옮기는 행동으로 분해되어야 한다.

Affordance는 물체가 어떤 행동을 허용하는지에 대한 함수로 볼 수 있다.

$$A(o, a, c) = \Pr(\text{success} \mid o, a, c), \quad (7.4)$$

여기서 c 는 scene context이다. VLA가 강한 이유는 pretrained VLM이 물체명, 속성, 관계, 상식적 affordance에 대한 prior를 제공한다는 점이다. 하지만 이 prior는 물리적으로 검증된 지식이 아니다. VLM은 “손잡이를 잡는다”는 말의 의미를 알 수 있지만, 현재 카메라 각도에서 손잡이의 실제 6D pose, 마찰, 충돌 여유, 로봇의 reachability를 자동으로 보장하지 않는다.

그래서 manipulation VLA는 language grounding과 geometry grounding을 분리해서 읽어야 한다. 좋은 모델은 instruction-object alignment만 잘하는 것이 아니라, 그 alignment가 실행 가능한 grasp, push, pull, rotate action으로 이어진다는 것을 보여야 한다.

7.4 Contact dynamics가 만드는 난점

Manipulation이 navigation이나 visual question answering보다 어려운 결정적 이유는 접촉이다. 접촉이 없을 때 물체 상태는 거의 변하지 않는다. 접촉이 발생하는 순간부터 동역학은 불연속적이고 민감해진다.

$$M(q_r)\ddot{q}_r + C(q_r, \dot{q}_r)\dot{q}_r + g(q_r) = \tau + J_c(q_r)^\top \lambda, \quad (7.5)$$

q_r 는 로봇 joint configuration, τ 는 actuator torque, J_c 는 contact Jacobian, λ 는 contact force이다. VLA policy가 직접 torque를 내지 않더라도, end-effector command가 low-level controller를 통해 결국 이 식의 접촉항을 만든다.

Grasp 안정성도 단순한 위치 맞추기가 아니다. 점 접촉 모델에서는 접촉력이 마찰 원뿔 안에 있어야 한다.

$$\|f_t^{\text{tan}}\| \leq \mu f_t^{\text{normal}}. \quad (7.6)$$

VLA가 카메라와 언어만으로 행동을 생성하면 μ , 물체 질량, compliance, 손가락 접촉 위치 같은 변수를 정확히 알기 어렵다. 따라서 manipulation VLA의 실패는 종종 semantic error가 아니라 contact error이다. 모델은 올바른 물체를 고르고도 너무 빨리 닫거나, 잡는 위치가 낮거나, 목표점 근처에서 충돌하거나, 물체를 놓는 순간 안정성을 잃는다.

7.5 Closed-loop policy가 필요한 이유

Open-loop action chunking은 효율성을 높인다. 그러나 manipulation에서는 관측을 자주 다시 보는 것이 중요하다. 특히 접촉 전후에는 작은 오차가 누적된다.

$$a_{t:t+H-1} = \pi_\theta(o_t, q), \quad o_{t+k} \notin \pi_\theta \quad (1 \leq k < H) \quad (7.7)$$

위 식은 긴 chunk의 위험을 보여준다. H 가 길수록 중간 관측 o_{t+k} 가 policy에 반영되지 않는다. 물체가 미끄러지거나, gripper가 예상보다 늦게 닫히거나, 사람이 물체를 조금 움직이면 open-loop chunk는 실패할 수 있다.

Closed-loop VLA는 다음과 같이 재계획한다.

$$a_{t:t+H-1} = \pi_{\theta}(o_t, q), \quad a_{t+\Delta:t+\Delta+H-1} = \pi_{\theta}(o_{t+\Delta}, q, h_{1:t+\Delta}). \quad (7.8)$$

문제는 재계획 빈도와 latency의 trade-off이다. 6장에서 보았듯이 VLA inference가 느리면 closed-loop라고 불려도 실제 제어 loop에는 늦게 들어온다. 그래서 manipulation에서는 efficient VLA와 closed-loop policy가 함께 필요하다. 접근 단계에서는 큰 chunk를 쓰고, 접촉 단계에서는 작은 chunk 또는 low-level reflex를 쓰는 hybrid 구조가 실용적이다.

7.6 Low-level controller와 VLA의 역할 분리

VLA가 모든 제어를 직접 담당해야 하는 것은 아니다. 실제 manipulation 시스템은 high-level policy, motion planner, impedance controller, gripper controller, safety monitor를 조합한다.

$$q, I_t \xrightarrow{\text{VLA}} z_t \xrightarrow{\text{planner/controller}} u_t \xrightarrow{\text{robot}} s_{t+1}. \quad (7.9)$$

z_t 는 target pose, waypoint, skill id, affordance map, action chunk일 수 있다. u_t 는 joint command 또는 torque command이다. 공학적으로 좋은 VLA 논문은 VLA가 어느 수준의 결정을 내리는지 명확히 해야 한다. End-to-end라고 주장하면서 실제로는 hand-coded motion primitive가 대부분을 해결한다면, 모델의 기여를 과대평가할 수 있다.

역으로, 모든 것을 neural policy로 처리하는 것도 항상 좋은 방향은 아니다. 알려진 기하 제약, collision checking, joint limit, force limit은 classical controller가 더 안정적으로 처리할 수 있다. VLA의 강점은 언어와 시각에서 목표와 상황을 해석하는 데 있고, low-level controller의 강점은 물리 제약을 빠르고 안정적으로 만족시키는 데 있다.

7.7 Data 관점: manipulation dataset의 편향

Manipulation VLA는 대규모 robot dataset에 의존한다. 그러나 데이터 수집은 비싸고 편향이 강하다. 같은 “pick up the cup”이라도 로봇 종류, gripper 형태, 카메라 위치, 물체 set, table height, demonstration policy가 다르면 action distribution이 달라진다.

$$p_{\mathcal{D}}(a \mid o, q) \neq p_{\text{deploy}}(a \mid o, q). \quad (7.10)$$

이 distribution shift는 VLA가 deployment에서 실패하는 주요 이유이다. Open X-Embodiment 계열 데이터는 다양한 robot embodiment를 모으지만, embodiment heterogeneity 자체가 또 다른 문제를 만든다. 같은 action token이 다른 로봇에서는 다른 물리적 의미를 가진다. 따라서 manipulation dataset을 읽을 때는 다음을 확인해야 한다.

- 어떤 robot platform에서 수집되었는가.
- action space가 joint, end-effector, discrete skill 중 무엇인가.
- language instruction이 사람이 직접 쓴 것인지, template 또는 자동 생성인지.
- 실패 trajectory가 포함되는가.
- force, tactile, depth, proprioception이 포함되는가.

- train task와 test task가 물체, 장면, 지시, embodiment 중 무엇에서 분리되는가.

Manipulation VLA의 일반화 주장은 이 항목 없이는 해석하기 어렵다. “새 물체에 일반화”와 “새 task에 일반화”와 “새 robot에 일반화”는 서로 다른 주장이다.

7.8 Evaluation: 성공률만으로 충분하지 않다

Manipulation benchmark는 success rate를 자주 사용한다.

$$SR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{task}_i \text{ success}]. \quad (7.11)$$

Success rate는 필요하지만 충분하지 않다. 특히 VLA는 언어 지시를 받기 때문에 성공 여부를 더 세분화해야 한다.

$$\text{Failure} = F_{\text{semantic}} \cup F_{\text{geometric}} \cup F_{\text{contact}} \cup F_{\text{temporal}} \cup F_{\text{safety}}. \quad (7.12)$$

Semantic failure는 잘못된 물체를 고르는 경우이고, geometric failure는 올바른 물체를 고르지만 pose가 틀리는 경우이다. Contact failure는 잡거나 누르거나 끌 때 힘과 접촉이 실패하는 경우이다. Temporal failure는 순서가 틀리거나 중간 상태를 복구하지 못하는 경우이다. Safety failure는 충돌, 과도한 force, workspace violation이다.

좋은 평가표는 성공률과 함께 episode length, intervention count, collision rate, drop rate, recovery rate, latency, human instruction complexity를 보고해야 한다. 특히 “실패 후 다시 시도해서 성공하는가”는 VLA가 단순 imitation을 넘어 interactive policy가 되는지 판단하는 중요한 지표이다.

7.9 Long-horizon manipulation

Long-horizon task에서는 하나의 instruction이 여러 subgoal을 포함한다.

$$q \rightarrow (g_1, g_2, \dots, g_K), \quad \pi_{\theta}(o_t, q, g_k) \rightarrow a_t. \quad (7.13)$$

예를 들어 “서랍을 열고 손가락을 꺼내 컵 옆에 놓아라”는 drawer opening, object localization, grasping, placing을 포함한다. 여기서는 VLA가 바로 action을 내는 것보다, subgoal 또는 skill sequence를 구성하는 능력이 중요해진다.

Long-horizon manipulation은 두 가지 오류가 누적된다. 첫째, planning error이다. 필요한 subgoal을 빠뜨리거나 순서를 잘못 잡는다. 둘째, execution error이다. 각 subgoal을 수행하는 동안 작은 실패가 다음 단계의 초기 조건을 바꾼다. 따라서 long-horizon VLA는 hierarchical policy와 memory가 필요하다.

$$\pi_{\theta}(a_t | o_t, q, h_t) = \sum_{z_t} \pi_{\theta}(a_t | o_t, z_t, h_t) p_{\theta}(z_t | o_t, q, h_t). \quad (7.14)$$

z_t 는 latent subtask, skill, option, waypoint가 될 수 있다. 이 분해는 모델이 task-level reasoning과 control-level action generation을 동시에 처리하지 않도록 돕는다.

7.10 Tactile과 force feedback

대부분의 VLA는 RGB 또는 RGB-D 중심이다. 그러나 manipulation에서 접촉 상태는 시각만으로 충분히 관측되지 않는다. 물체가 손가락 안에 들어온 뒤에는 카메라에서 가려지고, 미끄러짐은 작은 움직임으로만 나타난다. Force-torque sensor와 tactile sensor는 이 빈칸을 채운다.

$$o_t = [I_t, d_t, q_t^{\text{joint}}, f_t, \tau_t, T_t^{\text{tactile}}]. \quad (7.15)$$

문제는 tactile data가 대규모 web pretraining과 잘 연결되어 있지 않다는 점이다. 언어-이미지 모델은 인터넷 이미지와 텍스트로 pretraining할 수 있지만, tactile-language-action 데이터는 규모가 작고 센서마다 형식이 다르다. 그래서 tactile VLA는 2024–2026년 이후에도 중요한 연구 기회로 남아 있다. 특히 slip detection, grasp force adaptation, insertion task, deformable object manipulation에서는 tactile feedback이 성능 차이를 만들 가능성이 크다.

7.11 Manipulation 논문을 읽는 기준

Manipulation VLA 논문은 화려한 demonstration video만으로 평가하면 안 된다. 다음 질문을 반드시 확인해야 한다.

1. 실제 로봇 실험인가, simulation인가, offline dataset 평가인가.
2. 성공률을 몇 episode, 몇 seed, 몇 object, 몇 task에서 측정했는가.
3. baseline이 같은 perception, 같은 controller, 같은 data budget을 쓰는가.
4. action space가 무엇이고, low-level controller가 얼마나 많은 일을 하는가.
5. failure case를 semantic, geometric, contact, temporal, safety로 분리했는가.
6. novel object, novel instruction, novel scene, novel embodiment 중 무엇을 일반화로 주장하는가.
7. real-time loop와 latency를 보고했는가.

이 기준은 13-question framework와 연결된다. 특히 “방법”, “검증”, “비교”, “한계”, “자원 공개” 항목에서 manipulation 논문은 숨은 engineering detail이 많다. Controller와 dataset을 빼고 모델 구조만 보면 논문을 잘못 읽게 된다.

7.12 요약

Manipulation은 VLA가 언어와 시각을 실제 물리 행동으로 바꾸는 능력을 가장 직접적으로 검증하는 영역이다. VLM prior는 물체와 affordance를 해석하는 데 유용하지만, grasp 안정성, 접촉 동역학, force control, failure recovery는 별도의 공학적 문제로 남는다. 따라서 manipulation VLA의 핵심 질문은 “문장을 이해했는가”가 아니라 “이해한 내용을 real-time closed-loop contact task에서 안정적으로 실행했는가”이다.

다음 장에서는 이 질문을 실제로 어떻게 측정하는지 다룬다. VLA 성능은 논문마다 다른 dataset과 benchmark 위에서 보고되기 때문에, benchmark 설계와 dataset 편향을 이해하지 못하면 2024–2026년의 성능 향상을 정확히 비교할 수 없다.

Case study: contact-rich manipulation

ForceVLA 류 논문은 vision-language prior만으로는 접촉 상태, 미끄러짐, 과도한 힘, compliant motion을 충분히 다루기 어렵다는 점을 드러낸다. Manipulation VLA를 읽을 때 force, tactile, proprioception이 들어가면 단순히 modality가 늘어난 것이 아니라 failure taxonomy가 바뀐다. 성공률이 같아도 contact force violation이 줄었다면 safety와 deployment 관점에서 더 중요한 개선일 수 있다.

Key papers

- Zhao et al. (2023), ACT/ALOHA: action chunking과 bimanual manipulation을 연결하는 선행 anchor이다.
- Chi et al. (2023), Diffusion Policy: contact-rich continuous manipulation에서 action distribution modeling의 중요성을 보여 준다.
- Kim et al. (2024), OpenVLA: multiple robot embodiments와 29 tasks 평가를 통해 generalist manipulation 논의를 연다.
- Kim, Finn, and Liang (2025), OpenVLA-OFT: LIBERO와 real-world bimanual ALOHA에서 fine-tuning recipe를 평가한다.
- ForceVLA 계열 논문: force-aware module이 contact failure를 줄이는지 확인하기 위한 최신 case study로 읽는다.

Reading assignment [Intermediate]

하나의 manipulation VLA 논문을 골라 semantic failure, geometric failure, contact failure, temporal failure, safety failure로 실패 사례를 재분류하라.

Chapter 8

Benchmarks and Datasets: VLA 성능은 무엇으로 재는가

7장에서는 manipulation VLA를 평가할 때 성공률만 보면 안 된다고 했다. 이 장의 핵심은 그 주장을 더 엄밀하게 만드는 것이다. VLA 논문에서 성능 숫자는 dataset, benchmark, robot platform, action space, evaluation protocol에 묶여 있다. 같은 모델이라도 offline imitation loss에서는 좋아 보이고, simulation에서는 중간이고, 실제 로봇에서는 실패할 수 있다. 따라서 2024–2026년 VLA 트렌드를 읽으려면 모델 구조만큼이나 평가 체계를 읽어야 한다.

8.1 Benchmark가 정의하는 문제

Benchmark는 단순한 시험지가 아니다. 어떤 관측을 주고, 어떤 행동을 허용하고, 어떤 성공 조건을 쓰는지에 따라 연구 문제가 달라진다. VLA benchmark는 일반적으로 다음 tuple로 볼 수 있다.

$$B = (\mathcal{T}, \mathcal{E}, \mathcal{O}, \mathcal{A}, \mathcal{M}, \mathcal{P}). \quad (8.1)$$

$\mathcal{T}$ 는 task set, $\mathcal{E}$ 는 environment distribution, $\mathcal{O}$ 는 observation space, $\mathcal{A}$ 는 action space, $\mathcal{M}$ 은 metric, $\mathcal{P}$ 는 evaluation protocol이다. 논문이 “SOTA를 달성했다”고 말할 때 실제 의미는 이 tuple 안에서의 SOTA이다. $\mathcal{A}$ 가 end-effector delta pose인지, joint action인지, discrete skill인지에 따라 같은 success rate도 난이도가 달라진다.

Benchmark는 모델의 강점뿐 아니라 연구자들이 보는 세계를 정한다. Pick-and-place 중심 benchmark에서는 object grounding과 grasp pose가 중요해지고, long-horizon household benchmark에서는 subgoal planning과 recovery가 중요해진다. Driving benchmark에서는 temporal prediction과 safety가 중요해지고, embodied QA benchmark에서는 language reasoning이 더 크게 보인다. VLA를 읽을 때는 먼저 benchmark가 어떤 능력을 보상하는지 물어야 한다.

8.2 Offline dataset과 real-robot evaluation

VLA 연구는 크게 두 종류의 평가를 사용한다. 첫째는 offline dataset 평가이다. 둘째는 online 또는 real-robot evaluation이다.

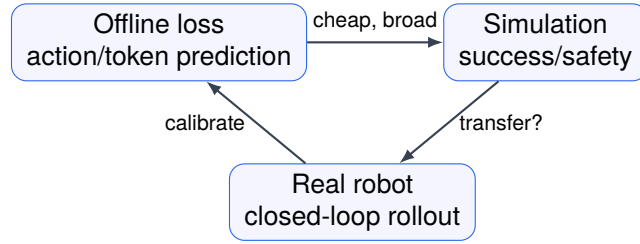


Figure 8.1: Author-created schematic. Evaluation triangle. VLA 성능은 offline loss, simulation, real robot evaluation 사이의 일치와 불일치를 함께 보아야 한다.

$$\mathcal{L}_{BC} = \mathbb{E}_{(o,q,a) \sim \mathcal{D}} [\ell(\pi_{\theta}(o, q), a)]. \quad (8.2)$$

Offline evaluation은 위와 같이 behavior cloning loss, action prediction accuracy, token accuracy, trajectory reconstruction error를 본다. 장점은 빠르고 반복 가능하다는 것이다. 단점은 실제 실행에서 distribution shift가 발생한다는 것이다. Dataset 안에서 expert action을 잘 예측해도, 모델이 한 번 실수해서 다른 state로 들어가면 dataset에는 없는 관측이 나오고 policy가 흔들린다.

Real-robot evaluation은 task success를 직접 측정한다.

$$J(\pi_{\theta}) = \mathbb{E}_{\tau \sim p(\tau|\pi_{\theta})} [R(\tau)]. \quad (8.3)$$

여기서 τ 는 policy가 실제로 만든 trajectory이고, $R(\tau)$ 는 성공, 안전성, 시간, 충돌 등을 포함한 return이다. Real-robot 평가는 더 믿을 만하지만 비용이 크고 episode 수가 제한된다. 그래서 논문은 offline 결과와 robot 결과를 함께 보여야 한다. Offline loss가 낮아졌는데 real success가 오르지 않으면, action 표현이나 evaluation target이 실제 제어 성공과 잘 맞지 않는다는 뜻일 수 있다.

8.3 대표 dataset 계열

2024–2026년 VLA 논문은 몇 가지 dataset 계열 위에서 움직인다. 이름을 외우는 것보다 각 dataset이 어떤 실험 주장을 가능하게 하는지 이해하는 것이 중요하다.

Open X-Embodiment류 dataset은 여러 로봇과 task를 묶어 generalist policy 학습을 가능하게 한다. 그러나 이 장점은 동시에 해석상의 난점을 만든다. 서로 다른 로봇의 action이 같은 좌표계와 시간 간격으로 정규화되어 있는지, gripper command가 같은 의미인지, camera viewpoint가 얼마나 다른지 확인해야 한다.

반대로 단일 실험실 dataset은 제어와 센서가 정돈되어 있어서 학습이 안정적이다. 하지만 이때 성능은 해당 lab setup에 과적합될 수 있다. 따라서 dataset의 크기보다 중요한 질문은 “어떤 종류의 변동성을 포함하는가”이다.

8.4 Generalization split

VLA에서 generalization은 하나의 단어로 쓰기에는 너무 넓다. 최소한 다음 축을 분리해야 한다.

$$\mathcal{G} = \mathcal{G}_{\text{object}} \times \mathcal{G}_{\text{instruction}} \times \mathcal{G}_{\text{scene}} \times \mathcal{G}_{\text{task}} \times \mathcal{G}_{\text{embodiment}}. \quad (8.4)$$

Table 8.1: VLA dataset을 읽는 관점

Dataset 유형	주로 검증하는 주장	주의할 점
Multi-robot manipulation	embodiment generalization, data scaling	action space 정규화와 robot별 편향을 확인해야 한다
Single-lab robot	실제 hardware success, precise control	task와 object 다양성이 제한될 수 있다
Simulation manipulation	대규모 episode, controlled ablation	sim-to-real gap과 contact realism을 확인해야 한다
Video-language dataset	world knowledge, affordance prior	action supervision이 없거나 약할 수 있다
Human demonstration	natural task strategy, rich variation	teleoperation interface와 demonstrator bias가 크다
Synthetic instruction dataset	language variation, compositionality	template bias와 실제 사용자 지시와의 차이를 확인해야 한다

Table 8.2: VLA 평가 metric의 계층

Metric 계층	예시	해석
Prediction	action MSE, token accuracy, negative log-likelihood	dataset action을 얼마나 잘 모사하는지 본다
Execution	success rate, completion rate, SPL-like score	실제 또는 simulated task가 끝까지 성공하는지 본다
Safety	collision rate, force violation, workspace violation	성공과 별개로 위험 행동이 있는지 본다
Efficiency	latency, memory, episode time, intervention count	real-time 배포 가능성을 본다
Robustness	perturbation success, recovery rate, OOD score	환경 변화와 실패 후 복구를 본다

Object generalization은 새로운 물체를 다루는 능력이다. Instruction generalization은 새로운 문장 표현을 이해하는 능력이다. Scene generalization은 배치, 조명, 배경, clutter가 달라져도 작동하는 능력이다. Task generalization은 새로운 subgoal 조합이나 새로운 manipulation skill을 수행하는 능력이다. Embodiment generalization은 다른 robot morphology에서 작동하는 능력이다.

논문에서 “unseen task”라고 쓰면 실제로 무엇이 unseen인지 확인해야 한다. 물체만 새롭고 행동 primitive는 같을 수 있다. 문장만 새롭고 scene은 같을 수 있다. Task template은 같고 색깔만 바뀔 수 있다. 반대로 embodiment까지 바뀌면 action mapping 자체가 달라지므로 난이도가 크게 올라간다.

8.5 Metric: 무엇을 숫자로 만들 것인가

VLA 평가 metric은 크게 prediction metric, execution metric, safety metric, efficiency metric으로 나눌 수 있다.

Prediction metric은 학습 과정에서는 중요하지만 최종 결론으로는 부족하다. 특히 multimodal

action distribution에서는 expert action이 하나만 정답이 아니다. 물체를 왼쪽에서 잡아도 되고 오른쪽에서 잡아도 되는데, MSE는 두 행동의 평균을 예측하게 만들 수 있다.

$$\hat{a}_{\text{MSE}} = \mathbb{E}[a \mid o, q]. \quad (8.5)$$

이 평균 action은 실제로는 어느 mode에도 속하지 않아 실패할 수 있다. 그래서 diffusion policy, discretized action token, mixture policy, flow policy가 등장한다. Metric도 이에 맞게 mode coverage와 task success를 함께 봐야 한다.

8.6 Success rate의 신뢰구간

Robot evaluation은 episode 수가 작을 때가 많다. N 번 중 k 번 성공한 success rate는 $\hat{p} = k/N$ 이다. 이 값 하나만 보고 모델 차이를 판단하면 위험하다.

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{N}}. \quad (8.6)$$

$N = 20$ 인 실험에서 14회 성공과 16회 성공은 각각 70%와 80%이다. 숫자로는 10 percentage point 차이지만, 통계적으로는 불확실성이 크다. 따라서 좋은 VLA 논문은 episode 수, seed, confidence interval, task별 breakdown을 제공해야 한다.

Episode 수가 작은 robot experiment에서는 단순 normal approximation만으로 충분하지 않을 수 있다. 따라서 binomial success rate에는 Wilson interval을 쓰거나, task별 episode가 섞인 경우에는 bootstrap confidence interval을 함께 보고하는 편이 더 안전하다. 핵심은 평균 success 하나가 아니라, 그 숫자가 얼마나 불확실한지 독자가 볼 수 있게 하는 것이다.

특히 manipulation에서는 task 난이도가 균등하지 않다. 쉬운 task 80개와 어려운 task 20개를 합쳐 평균을 내면 모델의 한계가 가려질 수 있다. 그래서 aggregate success rate와 함께 per-task, per-object, per-scene 결과가 필요하다.

8.7 Benchmark leakage와 overfitting

VLA는 pretrained VLM을 기반으로 하므로 benchmark leakage 가능성이 있다. 인터넷 이미지, caption, instruction corpus, public benchmark description이 pretraining에 들어갔을 수 있다. Robot trajectory 자체가 없더라도, 물체와 task에 대한 언어-시각 prior가 이미 학습되어 있을 수 있다.

Leakage를 완전히 배제하기는 어렵다. 대신 논문은 다음을 명확히 해야 한다.

- benchmark task와 object가 pretraining corpus에 얼마나 노출되었을 가능성이 있는가.
- evaluation instruction이 training instruction template과 얼마나 다른가.
- hyperparameter를 benchmark test set에 반복적으로 맞추지 않았는가.
- public leaderboard가 있다면 test set이 숨겨져 있는가.

VLA에서 overfitting은 단지 loss overfitting이 아니다. Demonstration style, camera pose, table layout, language template, robot speed, gripper calibration에 대한 overfitting도 포함한다. 논문을 읽

을 때 video success가 반복되는 동일 setup에서 나온 것인지, 독립적인 환경 변화에서 나온 것인지 확인해야 한다.

8.8 Simulation benchmark의 역할

Simulation은 대규모 evaluation과 ablation에 유용하다. 실제 로봇에서 1000 episode를 실행하는 것은 비싸지만 simulation에서는 가능하다. 또한 물체 위치, 조명, 질량, 마찰, distractor를 체계적으로 바꿀 수 있다.

$$\Delta_{\text{sim2real}} = J_{\text{sim}}(\pi_{\theta}) - J_{\text{real}}(\pi_{\theta}). \quad (8.7)$$

Simulation 결과를 읽을 때는 이 gap을 생각해야 한다. Visual realism이 좋아도 contact dynamics가 부정확하면 manipulation 성능이 과대평가된다. 반대로 simulation이 거칠어도 policy ranking이 real robot ranking과 잘 맞으면 benchmark로 유용할 수 있다. 핵심은 simulation이 실제 물리 세계의 절대 성능을 대신하는지, 아니면 모델 비교와 ablation을 위한 상대 평가인지 구분하는 것이다.

8.9 Leaderboard보다 중요한 ablation

2024–2026년 VLA 논문은 모델 이름과 benchmark score를 크게 보여주는 경향이 있다. 그러나 연구자로서 더 중요한 것은 ablation이다. Ablation은 성능이 어디서 오는지 밝힌다.

$$\Delta J_i = J(\pi_{\text{full}}) - J(\pi_{\setminus i}). \quad (8.8)$$

i 는 vision encoder, language backbone, action tokenizer, data mixture, fine-tuning method, temporal context, diffusion step, controller module일 수 있다. 좋은 ablation은 단일 요소를 제거했을 때 전체 성능이 얼마나 변하는지 보여준다. 나쁜 ablation은 서로 여러 요소가 동시에 바뀌어 해석이 어렵다.

VLA에서 특히 필요한 ablation은 다음과 같다.

1. language를 제거하거나 template instruction으로 바꿨을 때 성능.
2. visual pretraining backbone을 바꿨을 때 성능.
3. action representation을 continuous, discrete, diffusion으로 바꿨을 때 성능.
4. robot data mixture 비율을 바꿨을 때 성능.
5. latency를 줄였을 때 success가 얼마나 유지되는지.
6. low-level controller 또는 planner를 바꿨을 때 성능.

이 ablation이 있어야 VLA가 실제로 language와 vision을 사용해 행동을 만든 것인지, 아니면 controller와 dataset prior가 대부분을 해결했는지 판단할 수 있다.

8.10 데이터셋 문서를 읽는 체크리스트

Dataset 논문이나 benchmark 논문은 모델 논문보다 덜 화려할 수 있지만, VLA 분야에서는 매우 중요하다. 좋은 dataset 문서는 다음 정보를 포함해야 한다.

- robot platform, sensor, controller, calibration 절차.
- task taxonomy와 instruction 생성 방식.
- episode 수, 성공/실패 trajectory 비율, teleoperation 방식.
- action frequency, coordinate frame, gripper command 정의.
- train/validation/test split과 OOD split.
- license, privacy, safety, data quality filtering.
- baseline model과 재현 가능한 evaluation script.

특히 action coordinate frame은 반드시 확인해야 한다. End-effector delta가 camera frame인지, robot base frame인지, world frame인지에 따라 학습 문제가 달라진다.

$$\Delta p^{\text{base}} = R_{\text{cam}}^{\text{base}} \Delta p^{\text{cam}}. \quad (8.9)$$

Frame 정의가 애매하면 다른 연구자가 같은 모델을 재현하기 어렵다. VLA benchmark에서 재현성은 code 공개만으로 충분하지 않고, data schema와 evaluation environment가 함께 공개되어야 한다.

8.11 요약

VLA 성능은 모델 하나의 고유한 숫자가 아니라 benchmark tuple 위에서 정의되는 조건부 숫자이다. Offline loss, simulation success, real-robot success, safety, latency, robustness는 서로 다른 질문에 답한다. 2024–2026년 VLA 논문을 비교할 때는 “어느 모델이 더 높은가”보다 “무엇을 같은 조건에서 비교했는가”를 먼저 확인해야 한다.

다음 장에서는 benchmark score만으로는 포착하기 어려운 reasoning 문제를 다룬다. Manipulation과 benchmark가 실행 가능성을 묻는다면, reasoning VLA는 로봇이 지시를 분해하고 상황을 판단하며 실패를 설명하고 재계획할 수 있는지를 묻는다.

Key papers

- Open X-Embodiment Collaboration (2023), Open X-Embodiment/RT-X: 1M+ real robot trajectories와 22 robot embodiments를 통해 multi-embodiment dataset scaling을 논한다.
- Liu et al. (2024), LIBERO: language-conditioned manipulation benchmark에서 task suite와 generalization split을 읽는 기준을 제공한다.
- RoboTwin 2.0 계열 논문: simulation data generation과 bimanual manipulation evaluation을 연결하는 최신 benchmark 사례이다.
- Kim, Finn, and Liang (2025), OpenVLA-OFT: LIBERO 평균 success와 real-world evaluation을 함께 보고하므로 benchmark transfer를 토론하기 좋다.
- SimplerEnv 계열 benchmark: simulation-to-real ranking validity를 점검할 때 참고할 수 있다.

Reading assignment [Intermediate]

Open X-Embodiment, LIBERO, RoboTwin 2.0 중 하나를 골라 benchmark tuple $(\mathcal{T}, \mathcal{E}, \mathcal{O}, \mathcal{A}, \mathcal{M}, \mathcal{P})$ 을 채우고, 숨은 confound를 3개 이상 적어라.

Chapter 9

Reasoning VLA: 로봇은 생각해야 하는가

8장에서는 VLA 성능이 benchmark tuple 위에서 정의된다고 했다. 그런데 benchmark success rate가 높아도 아직 설명되지 않는 문제가 있다. 로봇이 복잡한 지시를 어떻게 분해하는가, 관측이 불충분할 때 어떻게 멈추는가, 실패했을 때 무엇을 고쳐야 하는가, 다른 사람이 볼 수 있는 언어로 자신의 상태를 설명할 수 있는가. 이 장에서 말하는 reasoning은 추상적인 철학 용어가 아니라, 행동 선택 전에 필요한 명시적 중간 계산이다.

9.1 Reasoning의 공학적 정의

VLA에서 reasoning은 action을 바로 출력하기 전에 latent 또는 explicit variable을 계산하는 과정으로 볼 수 있다.

$$\pi_{\theta}(a_t | o_t, q, h_t) = \sum_{z_t} \pi_{\theta}(a_t | o_t, q, h_t, z_t) p_{\theta}(z_t | o_t, q, h_t). \quad (9.1)$$

z_t 는 subgoal, object relation, affordance, constraint, plan, uncertainty state, failure diagnosis일 수 있다. 즉 reasoning은 모델이 “무엇을 해야 하는가”와 “어떻게 움직일 것인가”를 구분하도록 돕는 중간 표현이다.

이 정의가 중요한 이유는 reasoning을 단순히 chain-of-thought 문장 생성으로 축소하지 않기 위해서이다. 로봇에서 reasoning은 자연어 설명일 수도 있지만, waypoint graph, symbolic plan, constraint set, belief state, value estimate일 수도 있다. 핵심은 그 중간 계산이 실제 행동 품질을 개선해야 한다는 점이다.

9.2 언제 reasoning이 필요한가

모든 VLA task에 긴 reasoning이 필요한 것은 아니다. 눈앞에 있는 블록을 집어 지정된 상자에 넣는 짧은 과제에서는 direct policy가 충분할 수 있다. Reasoning이 필요해지는 상황은 대체로 다음 네 가지이다.

1. 지시가 여러 subgoal을 포함한다.
2. 관측이 부분적이어서 보이지 않는 상태를 추론해야 한다.

3. 행동 전에 safety 또는 task constraint를 검사해야 한다.
4. 실패 후 원인을 진단하고 다른 전략을 선택해야 한다.

이를 decision problem으로 쓰면 다음과 같다.

$$a_t^* = \arg \max_{a_t} \mathbb{E} [R(s_{t:t+H}, q) - \lambda C(s_{t:t+H}, a_{t:t+H}) \mid b_t], \quad (9.2)$$

b_t 는 belief state이고, C 는 collision, force violation, rule violation 같은 cost이다. Reasoning은 b_t , R , C , 가능한 subgoal을 더 잘 추정하기 위한 과정이다. 로봇은 단순히 다음 token을 맞추는 것이 아니라, 불확실한 상태에서 비용이 큰 실패를 피해야 한다.

9.3 Task decomposition

긴 자연어 지시는 바로 motor action으로 내려가기 어렵다. 일반적인 구조는 지시를 subtask sequence로 바꾸는 것이다.

$$q \xrightarrow{\text{decompose}} (g_1, \dots, g_K) \xrightarrow{\text{policy}} (a_1, \dots, a_T). \quad (9.3)$$

예를 들어 “서랍을 열고 안에 있는 스펀지를 싱크대 옆에 놓아라”는 서랍 위치 찾기, 손잡이 잡기, 당기기, 내부 물체 탐색, 스펀지 grasp, 목표 위치 배치로 나뉜다. 여기서 subgoal sequence가 틀리면 low-level policy가 아무리 좋아도 실패한다.

Task decomposition의 위험은 language prior가 물리 상태를 무시할 수 있다는 점이다. LLM은 그럴듯한 순서를 만들 수 있지만, 현재 서랍이 잠겨 있는지, 손잡이가 보이는지, 로봇팔이 닿는지 모를 수 있다. 따라서 VLA의 decomposition은 관측 기반이어야 한다.

$$p(g_{1:K} \mid q) \text{ 보다 } p(g_{1:K} \mid q, o_t, h_t) \text{ 가 필요하다.} \quad (9.4)$$

좋은 reasoning VLA는 plan을 한 번 만들고 끝내지 않는다. 실행 중 관측이 바뀌면 subgoal을 수정한다.

9.4 Symbolic plan과 neural policy

Reasoning을 구현하는 한 방법은 symbolic planner와 neural policy를 결합하는 것이다.

$$q, o_t \rightarrow \mathcal{P} = (\mathcal{S}, \mathcal{G}, \mathcal{U}, c) \rightarrow \pi_\theta(a_t \mid o_t, g_k). \quad (9.5)$$

$\mathcal{S}$ 는 symbolic state, $\mathcal{G}$ 는 goal predicates, $\mathcal{U}$ 는 operators, c 는 constraint이다. Planner는 “컵이 비어 있어야 집을 수 있다”, “문을 열기 전에 손잡이를 잡아야 한다” 같은 구조를 표현할 수 있다. Neural policy는 각 operator를 실제 행동으로 실행한다.

이 접근의 장점은 해석 가능성과 검증 가능성이다. 단점은 symbolic state를 정확히 추출하기 어렵고, 조작 세계의 연속적 접촉을 discrete predicate로 단순화해야 한다는 것이다. 그래서 2024–2026년 reasoning VLA는 완전한 symbolic planning보다, language plan, visual grounding, learned skill policy를 섞는 hybrid 구조가 많다.

9.5 Belief와 uncertainty

로봇은 모든 상태를 볼 수 없다. 물체가 가려질 수 있고, grasp가 성공했는지 카메라로 확실히 보이지 않을 수 있고, drawer 안의 물체는 열기 전까지 모른다. 따라서 VLA는 state s_t 가 아니라 belief b_t 위에서 행동해야 한다.

$$b_t(s) = p(s_t = s \mid o_{1:t}, a_{1:t-1}, q). \quad (9.6)$$

Belief update는 Bayes filter 형태로 쓸 수 있다.

$$b_t(s') \propto p(o_t \mid s') \int p(s' \mid s, a_{t-1}) b_{t-1}(s) ds. \quad (9.7)$$

대부분의 VLA는 이 식을 명시적으로 풀지 않는다. 대신 transformer hidden state, recurrent memory, visual history, retrieved context가 belief 역할을 한다. 하지만 논문을 읽을 때는 같은 질문을 해야 한다. 모델이 무엇을 모른다고 표현할 수 있는가. 불확실할 때 더 관측하려는 행동을 선택하는가. 성공 여부를 확인한 뒤 다음 단계로 넘어가는가.

Uncertainty는 score로 나타낼 수 있다.

$$U_t = H[p_\theta(a_t \mid o_t, q, h_t)] + \beta H[p_\theta(g_t \mid o_t, q, h_t)]. \quad (9.8)$$

이 값이 높을 때 로봇은 바로 행동하기보다 재관측, 질문, 느린 제어, human confirmation을 선택할 수 있다. Reasoning VLA의 중요한 특징은 답을 만드는 능력보다 모를 때 멈추는 능력이다.

9.6 Self-check와 verifier

LLM 기반 agent에서는 self-check 또는 verifier가 자주 사용된다. VLA에서도 유사한 구조를 쓸 수 있다.

$$\hat{a}_{t:t+H} = \pi_\theta(o_t, q), \quad v_\phi(o_t, q, \hat{a}_{t:t+H}) \rightarrow [\text{valid}, \text{risk}, \text{reason}]. \quad (9.9)$$

Verifier는 action이 목표와 맞는지, collision 가능성이 있는지, unsafe instruction인지, missing precondition이 있는지 확인한다. 예를 들어 “뜨거운 팬을 맨손으로 집어라” 같은 지시는 거부하거나 도구 사용을 제안해야 한다.

다만 verifier가 language-only이면 충분하지 않다. 물리적 위험은 geometry와 force에 의존한다. 따라서 VLA verifier는 가능하면 scene geometry, robot reachability, predicted trajectory, contact risk를 함께 봐야 한다. Verifier는 행동 전 safety layer가 될 수 있지만, latency를 늘릴 수 있으므로 6장의 efficient VLA 논의와도 연결된다.

9.7 Failure diagnosis와 recovery

Robot reasoning의 가장 실용적인 용도는 실패 복구이다. 실패한 후 같은 action을 반복하면 안 된다. 실패 원인을 분류하고 다음 전략을 바꿔야 한다.

$$d_t = f_\psi(o_{1:t}, a_{1:t}, q) \in \{\text{wrong object, bad grasp, collision, lost object, unreachable}\}. \quad (9.10)$$

복구 policy는 diagnosis에 조건부로 달라진다.

$$a_t \sim \pi_\theta(a_t \mid o_t, q, h_t, d_t). \quad (9.11)$$

예를 들어 bad grasp라면 gripper를 열고 접근 pose를 바꾼다. Lost object라면 camera를 움직이거나 scene을 재탐색한다. Unreachable이면 base를 이동하거나 사용자에게 목표 위치 변경을 요청한다. 이처럼 recovery는 reasoning, perception, control이 모두 필요한 영역이다.

9.8 Language as interface

VLA에서 언어는 input instruction일 뿐 아니라 interface가 될 수 있다. 로봇은 자신의 plan, uncertainty, failure reason을 사람에게 설명할 수 있다.

$$e_t = g_\eta(o_t, h_t, q, z_t, d_t), \quad (9.12)$$

e_t 는 자연어 explanation이다. 설명은 사용자가 로봇을 신뢰하게 만드는 장식이 아니다. HRI 관점에서는 사용자가 로봇의 한계를 이해하고, 지시를 수정하고, 안전한 협업을 할 수 있게 하는 제어 channel이다.

하지만 설명은 행동과 일치해야 한다. 로봇이 실제로는 scene을 확인하지 않았는데 “확인했다”고 말하면 위험하다. 따라서 explanation은 internal state와 action log에 grounded되어야 한다. Reasoning VLA 논문은 natural language explanation의 fluency보다, 설명이 실제 perception과 policy decision에 근거하는지를 보여야 한다.

9.9 Chain-of-thought 공개의 문제

LLM에서는 chain-of-thought를 출력하면 성능이 좋아 보일 수 있다. 그러나 로봇에서는 모든 reasoning trace를 공개하는 것이 항상 옳지 않다. 첫째, 긴 reasoning은 latency를 증가시킨다. 둘째, 생성된 문장이 실제 policy decision과 다를 수 있다. 셋째, 안전 관련 reasoning은 사용자가 악용할 수 있는 정보를 포함할 수 있다.

따라서 VLA에서는 internal reasoning과 external explanation을 구분하는 것이 좋다.

$$z_t^{\text{internal}} \neq e_t^{\text{external}}. \quad (9.13)$$

Internal reasoning은 action selection과 safety check를 위한 계산이고, external explanation은 사용자에게 필요한 수준으로 요약된 정보이다. 수업에서 이 차이를 강조해야 한다. Reasoning이 좋다는 말은 긴 문장을 많이 출력한다는 뜻이 아니라, 행동에 필요한 중간 상태를 정확히 계산한다는 뜻이다.

Table 9.1: Reasoning VLA 평가 항목

항목	확인할 내용
Decomposition accuracy	지시를 올바른 subgoal sequence로 분해하는가
Grounded plan validity	plan이 현재 관측과 robot reachability에 맞는가
Replanning success	실패 또는 관측 변화 후 plan을 수정하는가
Uncertainty behavior	불확실할 때 멈춤, 재관측, 질문을 선택하는가
Constraint satisfaction	safety, collision, force, workspace constraint를 지키는가
Explanation faithfulness	설명이 실제 관측, plan, action log와 일치하는가
Latency overhead	reasoning module이 control loop를 망치지 않는가

9.10 Reasoning VLA 평가

Reasoning 능력은 success rate만으로 평가하기 어렵다. 다음과 같은 평가가 필요하다.

Reasoning benchmark는 adversarial instruction과 partial observability를 포함해야 한다. 예를 들어 보이지 않는 물체를 요구하거나, 모순된 지시를 주거나, 안전하지 않은 행동을 요청하거나, 중간에 물체 위치를 바꾸는 상황이 필요하다. 단순한 household command completion만으로는 reasoning과 memorized prior를 구분하기 어렵다.

Review box: Text plan vs grounded plan vs recovery policy

Reasoning VLA 논문은 plan text를 생성했다는 사실만으로는 충분하지 않다. 다음 세 수준을 분리해야 논문 주장이 action-level evidence로 닫힌다.

수준	리뷰할 질문
Text plan	생성된 plan이 언어적으로 그럴듯한가를 넘어서, 현재 관측과 robot reachability를 실제로 반영하는가?
Grounded plan	Subgoal, object relation, precondition, constraint가 visual scene과 controller interface에 연결되어 있는가?
Recovery policy	실패를 감지한 뒤 같은 action을 반복하지 않고, diagnosis에 따라 재관측, 재접근, handoff, safe stop으로 전환하는가?
최소 baseline	No-plan direct policy, text-only plan, grounded plan without recovery를 분리해 비교했는가?
핵심 metric	Plan accuracy보다 recovery success, unsafe action reduction, intervention reduction, latency overhead를 보고했는가?

Table 9.2: Claim-source-evidence-caution map for reasoning VLA

Claim	Source	Evidence	Caution
Language planning needs affordance grounding	SayCan [1]	LLM plan combined with affordance scoring in robot tasks	Pre-VLA predecessor; do not classify as end-to-end VLA.
Planning representation should be tested, not assumed	VLA-OS [20]	Comparison of planning representations/paradigms	Check whether the comparison controls backbone and task protocol.
Memory/reasoning must improve action-level outcomes	MemoryVLA 계열, Gemini Robotics-ER [9]	Long-horizon memory or embodied reasoning claims	Closed/company or preprint evidence must be separated from reproducible benchmarks.
Explanation is not reasoning unless grounded in action logs	Reasoning VLA papers	Explanation faithfulness and recovery behavior	Fluent text can be post-hoc and unrelated to policy decisions.

9.11 요약

Reasoning VLA는 로봇이 긴 문장을 생성하는 문제가 아니라, 행동 전에 필요한 중간 상태를 계산하고 검증하는 문제이다. Task decomposition, belief update, uncertainty, verifier, failure diagnosis, recovery, explanation은 모두 reasoning의 구체적 형태이다. 좋은 reasoning은 성공률을 높이는 것뿐 아니라, 실패를 더 잘 감지하고, 위험한 행동을 피하고, 사람과 더 정확히 상호작용하게 만든다.

다음 장에서는 reasoning의 한 단계 앞에 있는 문제를 다룬다. 로봇이 행동하기 전에 세계가 어떻게 변할지 예측할 수 있다면, plan과 safety check는 더 강해진다. 이것이 world model과 simulation이 VLA에서 중요한 이유이다.

Case study: VLA-OS와 MemoryVLA

VLA-OS류 논문은 VLA가 end-to-end action generator로 충분한지, 또는 planning representation을 명시적으로 구조화해야 하는지 묻는다. 이때 핵심 평가는 plan text의 자연스러움이 아니라 planning representation이 success, recovery, failure diagnosis를 얼마나 개선하는가이다. MemoryVLA류 논문은 장기 이력과 과거 실패를 action decision에 넣는 방향으로 볼 수 있으며, reasoning을 “설명 생성”이 아니라 state/history variable 관리 문제로 읽게 만든다.

Key papers

- Ahn et al. (2022), SayCan: LLM plan과 affordance grounding의 선행 구조를 보여 준다.
- VLA-OS (2025), arXiv preprint: planning representation/paradigm을 VLA 내부 구조로 비교하는 case study이다.
- WorldVLA (2025), preprint/project report: action과 image understanding/generation을 결합해 reasoning과 world model의 경계를 흐린다.
- Google DeepMind Gemini Robotics-ER (2025/2026), company report/page: closed industrial embodied reasoning model의 사례로 읽는다.
- MemoryVLA 계열 논문: history, memory, failure recovery가 reasoning claim을 어떻게 바꾸는지 확인할 수 있다.

Reading assignment [Advanced]

Reasoning VLA 논문 하나를 골라 reasoning output이 subgoal, relation, constraint, uncertainty, recovery 중 무엇인지 표시하고, 그 변수가 closed-loop success로 검증되었는지 판단하라.

Chapter 10

World Models and Simulation: 행동 전에 세계를 예측하기

9장에서는 reasoning을 행동 전에 필요한 중간 계산으로 정의했다. 이 장은 그 중간 계산 중 가장 물리적인 부분을 다룬다. 로봇이 행동하기 전에 “이 행동을 하면 세계가 어떻게 변하는가”를 예측할 수 있다면, planning, safety check, recovery가 모두 강해진다. World model은 이 예측을 담당하는 모델이고, simulation은 이 예측을 대규모로 시험하거나 학습시키는 환경이다.

10.1 World model의 기본 형태

World model은 현재 상태와 행동이 주어졌을 때 다음 상태 또는 미래 관측을 예측하는 함수이다.

$$p_{\phi}(s_{t+1} \mid s_t, a_t), \quad p_{\phi}(o_{t+1} \mid o_{\leq t}, a_t). \quad (10.1)$$

VLA에서는 state s_t 를 직접 알기 어려우므로 관측 기반 world model이 더 자주 쓰인다.

$$\hat{o}_{t+1:t+H} = f_{\phi}(o_{1:t}, a_{t:t+H-1}, q). \quad (10.2)$$

여기서 $\hat{o}_{t+1:t+H}$ 는 미래 image, depth, object state, affordance map, contact state, success probability 등일 수 있다. 중요한 점은 world model이 VLA backbone과 완전히 별개일 필요는 없다는 것이다. Transformer의 hidden state가 implicit world model 역할을 할 수도 있고, 별도 video prediction model이나 dynamics model을 붙일 수도 있다.

10.2 왜 VLA에 world model이 필요한가

순수 reactive policy는 현재 관측에서 바로 action을 낸다.

$$a_t = \pi_{\theta}(o_t, q). \quad (10.3)$$

이 구조는 빠르지만, 행동의 결과를 미리 평가하지 않는다. 반면 model-based policy는 candidate action을 rollout하고 평가할 수 있다.

$$a_t^* = \arg \max_{a_t} \mathbb{E}_{\hat{\tau} \sim p_\phi} [R(\hat{\tau}, q) - \lambda C(\hat{\tau})]. \quad (10.4)$$

여기서 $\hat{\tau}$ 는 world model이 예측한 미래 trajectory이다. 이 식은 9장의 reasoning과 직접 연결된다. Reasoning이 subgoal과 constraint를 만들면, world model은 그 subgoal을 실행했을 때 실제 장면이 어떻게 변할지 평가한다.

VLA에서 world model이 특히 필요한 경우는 contact-rich manipulation, long-horizon task, safety-critical setting, partial observability, human interaction이다. 행동 결과가 단순하지 않고, 한 번의 실패 비용이 큰 경우일수록 예측이 중요하다.

10.3 Video prediction과 action-conditioned prediction

가장 직관적인 world model은 video prediction이다. 과거 이미지와 행동을 보고 미래 이미지를 만든다.

$$\hat{I}_{t+1:t+H} = f_\phi(I_{1:t}, a_{t:t+H-1}, q). \quad (10.5)$$

문제는 pixel-level prediction이 로봇 제어에 항상 좋은 objective는 아니라는 것이다. 배경 texture를 잘 예측하는 것보다, 물체 pose, 접촉, 성공 여부를 잘 예측하는 것이 더 중요하다. 따라서 VLA용 world model은 pixel loss뿐 아니라 task-relevant latent를 예측해야 한다.

$$z_{t+1} = e_\psi(o_{t+1}), \quad \hat{z}_{t+1} = f_\phi(z_t, a_t, q). \quad (10.6)$$

z_t 는 object-centric representation, scene graph, keypoint, affordance field, latent visual token일 수 있다. 좋은 world model은 모든 픽셀을 정확히 맞추는 모델이 아니라, 행동 선택에 필요한 미래 변수를 맞추는 모델이다.

10.4 Object-centric world model

Manipulation에서는 object state가 중요하다. 따라서 scene을 object set으로 표현하는 방식이 유용하다.

$$S_t = \{(c_i, p_i, R_i, v_i, \alpha_i)\}_{i=1}^{N_t}, \quad (10.7)$$

c_i 는 object category, p_i 와 R_i 는 pose, v_i 는 velocity, α_i 는 affordance 또는 상태이다. Object-centric model은 다음을 예측한다.

$$S_{t+1} = F_\phi(S_t, a_t, q). \quad (10.8)$$

이 표현은 planning과 해석에 좋다. “컵이 목표 영역 안에 들어갔는가”, “서랍이 열렸는가”, “장애물과 충돌했는가”를 직접 계산할 수 있다. 단점은 object detection과 pose estimation이 어려우며, deformable object나 fluid처럼 명확한 object state를 정의하기 어려운 경우가 있다는 점이다.

10.5 Language-conditioned world model

VLA의 world model은 언어와 분리될 수 없다. 같은 장면에서도 지시가 다르다면 예측해야 할 미래가 달라진다. 예를 들어 같은 컵을 보더라도 “컵을 집어라”와 “컵을 밀어라”는 다른 미래 상태를 요구한다.

$$p_\phi(s_{t+1:t+H} \mid s_t, a_{t:t+H-1}, q) \neq p_\phi(s_{t+1:t+H} \mid s_t, a_{t:t+H-1}). \quad (10.9)$$

언어는 reward와 constraint도 정의한다.

$$R(\tau, q) = r_{\text{task}}(\tau, q) - \lambda_{\text{safety}} c_{\text{safety}}(\tau) - \lambda_{\text{effort}} c_{\text{effort}}(\tau). \quad (10.10)$$

따라서 language-conditioned world model은 미래 상태 예측뿐 아니라, 그 미래가 지시를 만족하는지 판단하는 model도 포함한다. VLM 기반 reward model이나 success classifier가 여기에 들어갈 수 있다.

10.6 Simulation as data engine

Simulation은 world model을 직접 쓰는 방식이기도 하고, world model을 학습하기 위한 data engine이기도 하다. Simulation에서 다양한 scene과 action을 생성하면 real robot보다 훨씬 많은 transition을 얻을 수 있다.

$$\mathcal{D}_{\text{sim}} = \{(o_t, a_t, o_{t+1}, q, r_t)\}_{t=1}^M. \quad (10.11)$$

Simulation의 강점은 scale과 control이다. 물체 위치, 조명, 마찰, 질량, 카메라 pose, distractor를 체계적으로 바꿀 수 있다. 또한 unsafe action도 실제 위험 없이 시험할 수 있다. Safety-critical VLA에서는 simulation이 거의 필수적이다.

그러나 8장에서 본 것처럼 sim-to-real gap이 있다.

$$p_{\text{sim}}(o_{t+1} \mid o_t, a_t) \neq p_{\text{real}}(o_{t+1} \mid o_t, a_t). \quad (10.12)$$

이 gap을 줄이기 위해 domain randomization, system identification, real-to-sim calibration, residual dynamics learning을 사용한다. 중요한 것은 simulation 결과를 실제 성능으로 과장하지 않고, 어떤 역할로 사용했는지 명확히 하는 것이다.

10.7 Model predictive control과 VLA

World model이 있으면 model predictive control(MPC)을 사용할 수 있다.

$$a_{t:t+H-1}^* = \arg \max_{a_{t:t+H-1}} \sum_{k=0}^{H-1} R(\hat{s}_{t+k}, a_{t+k}, q), \quad (10.13)$$

$$\hat{s}_{t+k+1} = f_\phi(\hat{s}_{t+k}, a_{t+k}). \quad (10.14)$$

MPC는 첫 action만 실행하고 다음 관측에서 다시 최적화한다. 이는 7장의 closed-loop manipulation과 잘 맞는다. VLA는 candidate action이나 subgoal을 제안하고, world model과 MPC는 그 후보를 평가하여 더 안전한 선택을 할 수 있다.

다만 VLA에서 MPC는 계산 비용이 크다. Action space가 고차원이고 world model이 neural network이면 많은 rollout이 필요하다. 따라서 practical VLA는 다음 중 하나를 선택한다.

- VLA가 소수의 candidate subgoal을 만들고 world model이 ranking한다.
- Low-level continuous control은 classical MPC에 맡기고 VLA는 goal을 준다.
- Learned value function으로 rollout 수를 줄인다.
- Diffusion 또는 sampling policy를 proposal distribution으로 사용한다.

10.8 Counterfactual reasoning

World model이 있으면 “만약 이 행동을 하면 어떻게 되는가”를 비교할 수 있다.

$$\Delta R = R(\hat{\tau}^{(1)}, q) - R(\hat{\tau}^{(2)}, q). \quad (10.15)$$

예를 들어 컵을 왼쪽에서 잡는 후보와 오른쪽에서 잡는 후보를 비교할 수 있다. 서랍을 당기기 전에 물체를 치울지, 바로 손잡이를 잡을지 비교할 수 있다. 이런 counterfactual reasoning은 단순 imitation dataset에는 직접 들어 있지 않은 판단을 가능하게 한다.

하지만 예측이 틀리면 counterfactual 비교도 틀린다. 특히 접촉, 마찰, deformable object에서는 world model error가 빠르게 누적된다.

$$\epsilon_H = \|s_{t+H} - \hat{s}_{t+H}\| \leq L^H \epsilon_0 + \sum_{k=0}^{H-1} L^{H-1-k} \delta_k. \quad (10.16)$$

L 은 dynamics의 Lipschitz-like factor이고, δ_k 는 step-wise model error이다. Horizon이 길어지면 오차가 커질 수 있으므로, world model은 장기 예측보다 short-horizon verification과 replanning에 더 실용적일 때가 많다.

10.9 World model 평가

World model을 pixel MSE로만 평가하면 부족하다. VLA에서 필요한 평가는 다음과 같다.

가장 중요한 평가는 planning utility이다. 미래 이미지를 예쁘게 만드는 world model보다, 성공 후보와 실패 후보를 구분하는 world model이 로봇에는 더 유용하다.

Table 10.1: VLA world model 평가 기준

항목	확인할 내용
Prediction accuracy	미래 image, pose, affordance, contact state를 얼마나 잘 예측하는가
Task relevance	예측 오차가 실제 success와 상관되는가
Planning utility	world model을 쓰면 policy success가 오르는가
Safety utility	collision, force, unsafe state를 사전에 감지하는가
Horizon robustness	예측 horizon이 길어질 때 error가 어떻게 증가하는가
Sim-to-real validity	simulation 또는 learned dynamics가 real robot ranking과 맞는가
Latency	rollout과 scoring이 control loop에 들어갈 수 있는가

Review box: Pixel prediction, object-state prediction, candidate ranking

World model 논문은 무엇을 예측하는지와 그 예측이 행동 선택에 어떻게 쓰이는지를 분리해서 읽어야 한다.

예측 수준	리뷰할 질문
Pixel prediction	미래 image/video가 그럴듯한가보다, object pose, contact, occlusion, failure precursor를 보존하는가가 중요하다.
Object-state prediction	Object pose, relation, affordance, contact state가 task success와 직접 연결되는 변수로 예측되는가?
Candidate ranking	여러 action 후보 중 성공 가능성이 높은 후보를 더 잘 고르는가, 그리고 ranking accuracy가 rollout success와 상관되는가?
Safety utility	Collision, force violation, unreachable state, human proximity 같은 위험을 실행 전에 걸러내는가?
Cost	Rollout과 scoring latency가 control loop 안에 들어오며, reactive baseline 대비 net utility가 남는가?

10.10 요약

World model은 VLA가 행동하기 전에 미래를 평가하게 만드는 장치이다. Video prediction, object-centric dynamics, language-conditioned reward, simulation, MPC는 모두 이 목적을 위한 서로 다른 구현이다. 그러나 world model은 만능이 아니다. 접촉 동역학과 장기 horizon에서는 예측 오차가 누적되고, simulation은 real world와 다르며, rollout은 latency를 늘린다. 따라서 VLA에서 world model은 reactive policy를 대체하기보다, safety check, candidate ranking, short-horizon replanning, failure recovery를 보강하는 도구로 보는 것이 현실적이다.

다음 장에서는 예측과 planning이 실패했을 때 발생하는 문제를 정리한다. VLA가 실제 환경에 배포되려면 평균 성공률보다 robustness, safety, failure mode가 더 중요해지는 순간이 온다.

Table 10.2: Claim-source-evidence-caution map for world models

Claim	Source	Evidence	Caution
Action world models should be judged by planning utility	WorldVLA [22]	Autoregressive action/world modeling report	Pixel or latent prediction quality alone is insufficient.
Planning representation affects VLA behavior	VLA-OS [20]	Structured comparison of planning paradigms	Need direct action-level success and recovery evidence.
Embodied reasoning can be a separate module from action VLA	Gemini Robotics-ER [9]	Company report on embodied reasoning	Closed-stack claim; mark as evidence level C unless reproducible artifacts exist.
Simulation is useful when ranking transfers	RoboTwin 2.0 [18], LIBERO [12]	Controlled simulation benchmarks	Sim-to-real validity must not be assumed.

Case study: WorldVLA

WorldVLA류 논문은 action prediction과 image understanding/generation을 unified autoregressive action world model로 묶으려는 방향으로 읽을 수 있다. 이때 중요한 검증은 생성 이미지의 시각 품질보다, rollout이나 imagined transition이 candidate action ranking, short-horizon replanning, failure avoidance에 실제 이득을 주는지이다. 즉 WorldVLA는 10장의 핵심 문장인 “planning utility가 pixel score보다 중요하다”를 적용해 읽어야 한다.

Key papers

- Ha and Schmidhuber (2018), World Models: learned dynamics와 policy를 분리해서 생각하는 기본 관점을 제공한다.
- Du et al. 계열 video/world model 논문: action-conditioned prediction과 planning utility를 비교할 때 참고할 수 있다.
- WorldVLA (2025), preprint/project report: autoregressive action world model을 VLA 흐름 안에 넣는 최신 사례이다.
- RoboTwin 2.0 계열 논문: simulation이 data engine과 evaluation environment로 동시에 쓰이는 사례이다.
- Gemini Robotics-ER (2025/2026): world understanding, planning, embodied reasoning을 closed industrial stack으로 제시하는 사례이다.

Reading assignment [Advanced]

World model 논문 하나를 골라 prediction metric, task metric, planning utility metric을 분리하고, 세 지표가 같은 결론을 주는지 확인하라.

Chapter 11

Robustness and Safety

10장에서는 world model이 행동 후보를 미리 평가하는 도구가 될 수 있다고 했다. 그러나 실제 배포에서는 예측 모델도 틀리고, perception도 흔들리고, 사용자의 지시도 모호하며, 환경은 training data와 다르다. 따라서 VLA를 연구용 demonstration에서 실제 robot system으로 옮기려면 평균 성공률보다 failure mode와 safety envelope가 더 중요해진다.

11.1 Robustness의 정의

Robustness는 distribution이 바뀌어도 성능이 유지되는 정도이다. 이를 간단히 쓰면 다음과 같다.

$$\Delta J = J_{\mathcal{D}_{\text{train}}}(\pi_{\theta}) - J_{\mathcal{D}_{\text{test}}}(\pi_{\theta}). \quad (11.1)$$

ΔJ 가 작을수록 robust하다. 하지만 VLA에서 test distribution은 하나가 아니다. 물체, 조명, 배경, 카메라 위치, 언어 표현, task composition, robot calibration, human intervention이 모두 바뀔 수 있다.

$$\mathcal{D}_{\text{test}} = \mathcal{D}(o, q, a, e, r, u), \quad (11.2)$$

o 는 관측, q 는 지시, a 는 action, e 는 environment, r 은 robot embodiment, u 는 user behavior이다. Robustness 논문은 이 중 어느 축을 흔들었는지 명확히 해야 한다.

11.2 Failure taxonomy

VLA failure는 하나의 “실패”로 묶으면 분석이 불가능하다. 최소한 다음 계층으로 나누어야 한다.

이 taxonomy는 앞 장들과 연결된다. 7장의 manipulation은 contact failure를 강조했고, 8장은 benchmark가 failure를 얼마나 잘 분리하는지 물었으며, 9장은 reasoning과 recovery failure를 다루었고, 10장은 world model error가 planning과 safety failure로 이어질 수 있음을 보였다.

Table 11.1: VLA failure mode 분류

Failure 유형	설명
Perception failure	물체, pose, depth, scene relation을 잘못 인식한다
Language failure	지시의 대상, 순서, 제약, 금지를 잘못 해석한다
Grounding failure	언어 대상과 실제 장면의 물체를 잘못 연결한다
Planning failure	subgoal 순서, precondition, constraint를 잘못 설정한다
Control failure	올바른 계획이 있어도 trajectory, grasp, force 제어가 실패한다
Contact failure	미끄러짐, 충돌, 과도한 힘, 불안정 grasp가 발생한다
Recovery failure	실패를 감지하지 못하거나 같은 실패를 반복한다
Safety failure	사람, 로봇, 환경, 물체에 위험한 상태를 만든다

11.3 OOD detection

Out-of-distribution(OOD) 상황에서 VLA는 자신감 있게 틀릴 수 있다. 로봇에서는 이것이 위험하다. OOD score는 관측 또는 hidden representation이 training distribution에서 얼마나 벗어났는지 나타낸다.

$$d_{\text{OOD}}(o_t) = \min_{z_i \in \mathcal{Z}_{\text{train}}} \|e_{\psi}(o_t) - z_i\|_2. \quad (11.3)$$

또는 policy uncertainty를 사용할 수 있다.

$$U(a_t) = - \sum_a p_{\theta}(a \mid o_t, q) \log p_{\theta}(a \mid o_t, q). \quad (11.4)$$

Continuous action에서는 ensemble disagreement나 diffusion sample variance를 쓸 수 있다. 중요한 것은 OOD를 감지한 뒤의 행동이다. 감지만 하고 계속 움직이면 의미가 없다. Threshold를 넘으면 속도를 줄이거나, 재관측하거나, human confirmation을 요청하거나, safe pose로 돌아가야 한다.

11.4 Safety envelope

Safety는 모델의 선의에 맡길 수 없다. VLA 위에는 명시적 safety envelope가 필요하다.

$$a_t^{\text{exec}} = \Pi_{\mathcal{S}}(a_t^{\text{VLA}}), \quad (11.5)$$

$\Pi_{\mathcal{S}}$ 는 action을 안전 집합 $\mathcal{S}$ 안으로 projection하는 operator이다. 안전 집합은 joint limit, velocity limit, force limit, collision-free workspace, forbidden region, human distance constraint를 포함한다.

$$\mathcal{S} = \{a : c_i(s_t, a) \leq 0, \quad i = 1, \dots, m\}. \quad (11.6)$$

이 구조는 VLA를 약하게 만드는 것이 아니라 실제 시스템에 넣기 위한 조건이다. End-to-end policy가 아무리 강해도 hardware limit과 human safety rule을 위반하면 배포할 수 없다.

11.5 Calibration

VLA는 confidence를 출력할 수 있지만, confidence가 실제 성공 확률과 맞는지는 별개이다. Calibration은 예측 확률과 경험적 성공률의 일치 정도이다.

$$\Pr(\text{success} \mid \hat{p} = p) \approx p. \quad (11.7)$$

Expected calibration error(ECE)는 다음처럼 쓸 수 있다.

$$\text{ECE} = \sum_{b=1}^B \frac{|S_b|}{N} |\text{acc}(S_b) - \text{conf}(S_b)|. \quad (11.8)$$

Robot VLA에서 calibration은 중요하다. 모델이 90% 성공한다고 생각하지만 실제로 50%만 성공한다면, human handoff나 safety slowdown을 잘못 결정한다. 따라서 success predictor, verifier, reward model은 calibration curve를 보고해야 한다.

11.6 Adversarial and ambiguous instructions

VLA는 언어를 입력으로 받으므로 instruction safety가 필요하다. 위험한 지시, 모순된 지시, 애매한 지시, task 범위를 벗어난 지시를 구분해야 한다.

$$q \rightarrow y \in \{\text{execute}, \text{clarify}, \text{refuse}, \text{handoff}\}. \quad (11.9)$$

예를 들어 “칼을 사람 쪽으로 밀어라”는 refuse 또는 handoff가 필요하다. “저것을 치워라”는 clarify가 필요할 수 있다. “컵을 접시에 넣고 컵을 그대로 두어라”처럼 모순된 지시는 plan 생성 전에 감지해야 한다.

여기서 language safety와 physical safety는 다르다. 언어상 무해한 지시도 현재 장면에서는 위험할 수 있다. “컵을 들어라”는 말은 일반적으로 안전하지만, 컵이 뜨겁거나 깨져 있거나 사람 손 옆에 있으면 unsafe할 수 있다. 따라서 instruction filter는 scene-aware해야 한다.

11.7 Human intervention

실제 로봇 시스템은 완전 자율이 아니라 human intervention을 포함할 수 있다. Intervention은 실패가 아니라 안전 설계의 일부이다.

$$\pi(a_t \mid o_t, q) = \begin{cases} \pi_{\text{VLA}}(a_t \mid o_t, q), & U_t < \tau, \\ \pi_{\text{assist}}(a_t \mid o_t, q, h), & U_t \geq \tau. \end{cases} \quad (11.10)$$

π_{assist} 는 사람이 직접 조종하거나, 선택지를 승인하거나, 지시를 수정하는 interface일 수 있다. VLA benchmark가 human intervention count를 보고하면 실용성이 더 잘 드러난다.

Intervention data는 학습에도 쓸 수 있다. 실패 직전의 state와 human correction은 hard negative와 recovery demonstration을 제공한다. 따라서 robust VLA 연구는 실패를 숨기기보다 실패를 수집하고 재학습하는 loop를 설계해야 한다.

Table 11.2: VLA red-team 축

축	예시
Language	모순 지시, 금지 행동 유도, ambiguous reference
Vision	가림, 반사, 비슷한 물체, 조명 변화, motion blur
Geometry	좁은 공간, reachability 한계, clutter, thin object
Contact	미끄러운 물체, 무거운 물체, deformable object, fragile object
Human	손 끼어들기, instruction 변경, 협업자 위치 변화
System	latency spike, calibration drift, gripper fault, sensor dropout

11.8 Red teaming for robots

LLM에서 red teaming은 모델의 취약 지시를 찾는다. VLA에서는 red teaming이 더 넓다. 언어 지시, 장면 구성, 물체 속성, 조명, 센서 가림, 사람이 끼어드는 상황, hardware fault를 포함한다.

Red-team 결과는 단순 실패 영상 모음이 아니라 taxonomy와 mitigation으로 이어져야 한다. 어떤 failure가 safety envelope로 막히는지, 어떤 failure가 dataset 추가로 줄어드는지, 어떤 failure가 architecture 변경을 요구하는지 구분해야 한다.

11.9 Robustness 평가 설계

Robustness benchmark는 평균 success만 보고하면 부족하다. Perturbation 축별 성능 곡선을 보여야 한다.

$$J(\alpha) = \mathbb{E}[R(\tau) \mid \text{perturbation level} = \alpha]. \quad (11.11)$$

예를 들어 조명 변화 강도, object clutter 수, language paraphrase 정도, action latency, camera noise를 α 로 둘 수 있다. Robust model은 α 가 증가할 때 성능이 천천히 떨어진다.

또한 safety metric은 success와 별도로 보고해야 한다.

$$\text{Risk} = w_c \text{Collision} + w_f \text{ForceViolation} + w_h \text{HumanProximity} + w_o \text{ObjectDamage}. \quad (11.12)$$

성공률이 높은 모델이라도 risk가 높으면 좋은 모델이 아니다. Robot policy 평가는 reward maximization이 아니라 constrained optimization에 가깝다.

Deployment readiness checklist

VLA 논문이 실제 배포 가능성을 주장한다면, 성공률 표와 별도로 다음 checklist를 채워야 한다.

항목	질문
Latency	End-to-end loop latency를 sensor, preprocessing, model, decoding, safety filter, controller dispatch까지 포함해 측정했는가?
Safety	Joint, velocity, force, collision, workspace envelope가 있고 VLA output이 그 안으로 projection되는가?
OOD	OOD instruction, unseen object, changed scene, sensor dropout에서 confidence 있게 움직이지 않고 stop, clarify, handoff를 선택할 수 있는가?
Recovery	실패 후 같은 action을 반복하지 않고 diagnosis에 따라 재관측, pose 변경, regrasp, safe retreat를 수행하는가?
Intervention	Human handoff 기준, timeout 기준, unsafe instruction refusal 기준이 명시되어 있는가?
Logging	Failure trajectory, intervention, OOD trigger, safety-filter override가 저장되어 post-training data로 돌아가는가?
Audit	Model claim, controller contribution, safety layer contribution, benchmark protocol effect를 ablation으로 분리했는가?

11.10 요약

VLA의 robustness와 safety는 모델 크기를 키우면 자동으로 해결되는 문제가 아니다. Failure taxonomy, OOD detection, safety envelope, calibration, instruction safety, human intervention, red teaming이 함께 필요하다. 연구 논문을 읽을 때는 평균 success rate보다 어떤 실패가 남았고, 그 실패가 safety-critical한지, 논문이 mitigation을 실제로 검증했는지를 봐야 한다.

다음 장에서는 manipulation을 넘어 VLA가 어디까지 확장될 수 있는지 살핀다. Navigation, driving, games, general embodiment는 같은 VLA 원리를 쓰지만 observation, action, safety constraint가 크게 다르다.

Key papers

- OpenVLA (2024): open model의 fine-tuning과 quantization이 success를 해치지 않는지 확인하는 robustness anchor이다.
- OpenVLA-OFT (2025): success, throughput, real-world ALOHA evaluation을 함께 보고해 deployment risk를 토론하기 좋다.
- Gemini Robotics (2025/2026), Google DeepMind page/report: VLA, embodied reasoning, on-device variant, safety governance를 closed industrial stack으로 분류할 수 있다.
- GR00T N1 (2025), NVIDIA arXiv preprint: humanoid VLA에서 real-time motor action과 simulation benchmark를 함께 다루는 safety-relevant 사례이다.
- Driving VLA survey 계열 논문: manipulation보다 safety envelope와 intervention cost가 훨씬 큰 domain transfer 사례로 읽는다.

Reading assignment [Advanced]

VLA 논문 하나를 골라 성공률 표와 별도로 collision, force violation, OOD instruction, human intervention, latency timeout을 포함하는 safety table을 새로 설계하라.

Part IV

Generalist and Cross-Domain VLA

Chapter 12

Beyond Manipulation

앞 장들에서는 manipulation을 중심으로 VLA를 설명했다. 이유는 manipulation이 VLA의 핵심 병목인 object grounding, action representation, contact, real-time control, safety를 한꺼번에 드러내기 때문이다. 그러나 VLA의 아이디어는 로봇팔에만 머무르지 않는다. Navigation, autonomous driving, game agents, web/GUI agents, mobile robots, humanoids까지 모두 vision, language, action을 결합한다는 점에서는 같은 계열이다. 이 장의 목표는 응용 영역이 바뀔 때 무엇이 유지되고 무엇이 바뀌는지 정리하는 것이다.

12.1 공통 구조

응용이 달라져도 VLA의 기본 구조는 동일하다.

$$\pi_{\theta} : (o_{1:t}, q, h_{1:t}) \mapsto a_{t:t+H-1}. \quad (12.1)$$

차이는 o_t , q , a_t , reward, safety constraint의 의미이다. Manipulation에서 a_t 는 end-effector pose 또는 joint command였지만, navigation에서는 waypoint나 velocity command이고, driving에서는 steering/throttle/brake 또는 trajectory이며, game에서는 keyboard/mouse action 또는 discrete skill이다.

따라서 VLA를 일반화한다는 말은 단일 architecture를 모든 영역에 그대로 쓰는 것이 아니다. 공통 interface를 유지하면서 domain-specific action parameterization과 safety layer를 바꾸는 것이다.

12.2 Navigation VLA

Navigation에서 언어는 goal과 constraint를 지정한다. “복도 끝 회의실 앞까지 가라”, “사람을 피해 엘리베이터 쪽으로 이동하라”, “빨간 소파 옆에서 멈춰라” 같은 지시가 예시이다.

$$a_t = (\Delta x_t, \Delta y_t, \Delta \psi_t) \quad \text{or} \quad a_t = w_t. \quad (12.2)$$

w_t 는 waypoint이다. Manipulation과 비교하면 contact는 적지만, localization, map consistency, dynamic obstacle, long-range goal grounding이 더 중요하다. VLA navigation은 VLM의 semantic grounding을 이용해 “소파”, “엘리베이터”, “창문 근처” 같은 landmark를 이해할 수 있다.

Table 12.1: 응용 영역별 VLA 구성 요소

영역	Observation	Action	주요 constraint
Manipulation	RGB-D, proprioception, tactile	pose, gripper, joint, force	contact, collision, grasp stability
Navigation	egocentric image, map, pose	waypoint, velocity, turn command	obstacle, localization, path feasibility
Driving	multi-camera, LiDAR, map, route	trajectory, steering, brake	traffic rule, safety distance, comfort
Games	screen, state, inventory	key, mouse, discrete action	game rule, long-horizon reward
Humanoid	multi-camera, whole-body state	whole-body trajectory, skill	balance, contact, human safety

Navigation의 핵심 문제는 언어-지도 grounding이다.

$$q \rightarrow g^* \in \mathcal{M}, \quad (12.3)$$

$\mathcal{M}$ 은 metric map, topological map, semantic map일 수 있다. VLA가 장면을 이해해도 robot pose가 틀리면 목표를 찾을 수 없다. 따라서 navigation VLA는 SLAM, localization, mapping과 분리해서 읽으면 안 된다.

12.3 Embodied QA와 embodied instruction following

Embodied QA는 질문에 답하기 위해 환경 안에서 움직인다. 예를 들어 “부엌에 컵이 몇 개 있는가”라는 질문은 camera image 하나로 답할 수 없고, 탐색이 필요하다.

$$a_t = \arg \max_a I(y; o_{t+1} \mid o_{1:t}, a, q), \quad (12.4)$$

여기서 y 는 답이다. 행동의 목적은 물체를 조작하는 것이 아니라 정보를 얻는 것이다. 따라서 embodied QA에서 좋은 action은 goal에 가까워지는 action이 아니라 uncertainty를 줄이는 action일 수 있다.

이 영역은 9장의 reasoning과 강하게 연결된다. 로봇은 자신이 무엇을 모르는지 판단하고, 어느 위치로 가면 답을 얻을 수 있는지 계획해야 한다. VLA가 단순히 image captioning을 잘한다고 embodied QA를 잘하는 것은 아니다.

12.4 Autonomous driving과 VLA

Driving은 VLA와 유사한 구조를 오래전부터 가지고 있었다. Multi-camera와 LiDAR를 보고, route instruction을 받고, trajectory를 출력한다. 최근에는 language-conditioned driving, VLM-based scene reasoning, instruction following driving agent가 VLA 관점과 연결된다.

$$\tau_{t:t+H} = \pi_\theta(I_t^{1:N}, L_t, m_t, q), \quad (12.5)$$

$I_t^{1:N}$ 은 여러 camera, L_t 는 LiDAR, m_t 는 map 또는 route, q 는 high-level instruction이다. Driving의 action은 직접 steering일 수도 있지만, 보통 trajectory로 표현하는 것이 안전하다.

Driving에서 safety constraint는 manipulation보다 엄격하다.

$$d_{\text{safe}}(t) \geq d_0 + T_h v_t. \quad (12.6)$$

여기서 d_{safe} 는 앞차와의 거리, T_h 는 time headway이다. VLA가 언어와 장면을 잘 이해해도 traffic rule, comfort, liability, rare event safety를 만족해야 한다. 그래서 driving VLA는 end-to-end 행동 생성보다 perception-reasoning-planning stack 안의 한 모듈로 들어갈 가능성이 크다.

12.5 Games and interactive agents

Games는 VLA 연구에 유용한 testbed이다. Minecraft, 3D embodied games, browser/GUI control은 visual observation, language goal, action sequence를 제공한다. 실제 물리 위험은 낮고, long-horizon planning과 exploration을 평가하기 좋다.

$$a_t \in \{\text{move, turn, click, use, craft, } \dots\}. \quad (12.7)$$

게임과 GUI agent에서 action은 discrete인 경우가 많다. 따라서 manipulation의 contact control 문제는 줄어들지만, long-horizon credit assignment와 memory 문제가 커진다. “나무를 캐서 도구를 만들고 집을 지어라” 같은 목표는 수십 또는 수백 step의 plan을 요구한다.

이 영역의 장점은 scalable evaluation이다. 그러나 실제 로봇으로 바로 이전되는 것은 아니다. Game VLA에서 배운 planning과 memory 구조가 real robot에 유용할 수 있지만, physics, sensing, safety는 다시 검증해야 한다.

12.6 Humanoid와 general embodiment

Humanoid는 VLA의 가장 넓은 응용처럼 보인다. 언어 지시를 이해하고, 걸어가고, 물체를 집고, 사람과 상호작용한다. 그러나 기술적으로는 가장 어려운 문제 중 하나이다. Whole-body control, balance, contact switching, bimanual manipulation, human safety가 모두 필요하다.

$$a_t = \left[q_{t+1}^{\text{wholebody}}, \dot{q}_{t+1}^{\text{wholebody}}, f_t^{\text{contact}} \right]. \quad (12.8)$$

Humanoid VLA에서 high-level language understanding과 low-level motor control을 하나의 모델이 직접 처리하는 것은 비효율적일 수 있다. 더 현실적인 구조는 VLA가 task, subgoal, skill selection을 담당하고, whole-body controller가 balance와 contact를 처리하는 것이다.

Case study: GR00T N1, Helix, Gemini Robotics

GR00T N1은 humanoid용 open foundation model로 vision-language module과 diffusion transformer action module을 결합한 dual-system VLA를 제시한다. 이 주장은 humanoid VLA의 중요한 anchor이지만, real-robot trajectories, human videos, synthetic data mixture가 어떤 검증 조건에서 효과를 냈는지 별도로 확인해야 한다. Figure의 Helix는 company report 성격이 강하므로 full-upper-body humanoid control claim을 industrial signal로 읽되 peer-reviewed evidence와 분리한다. Gemini Robotics 1.5는 visual information과 instruction을 motor command로 바꾸는 closed industrial VLA로, Gemini Robotics-ER는 embodied reasoning model로 분리해 읽는다.

Evidence split: open model vs closed industrial model

Humanoid와 industrial VLA는 public evidence와 company claim이 섞이기 쉬우므로, 다음처럼 분리해 읽는다.

구분	확인할 내용
Open model evidence	Paper, code, checkpoint, training data summary, benchmark protocol, robot/sim setup이 공개되어 독립 검증이 가능한가?
Closed industrial evidence	Demo, product page, technical report가 무엇을 보여 주며, model weight, controller, data, safety layer 중 무엇이 비공개인가?
Action interface	Motor command가 end-effector, joint, whole-body trajectory, skill selection 중 무엇인지 명시되어 있는가?
Reasoning/ action split	Vision-language reasoning module과 motor action module이 분리되어 있는지, 그 분리가 ablation으로 검증되었는가?
Evidence level	Open arXiv/project evidence는 B, company demo 중심 evidence는 C로 표시하고 같은 benchmark score처럼 비교하지 않는다.

12.7 Generalist policy의 의미

Generalist VLA는 여러 task와 embodiment에서 동작하는 policy를 뜻한다. 그러나 generalist라는 말은 세 수준으로 나눌 수 있다.

1. 같은 로봇에서 여러 task를 수행한다.
2. 여러 로봇에서 비슷한 task를 수행한다.
3. 여러 domain에서 서로 다른 action space를 다룬다.

세 번째가 가장 어렵다. 서로 다른 action space를 하나의 token vocabulary로 묶을 수는 있지만, token의 물리적 의미는 domain마다 다르다.

$$a_t^{(d)} = D_d(u_t), \quad (12.9)$$

u_t 는 domain-agnostic latent action이고, D_d 는 domain-specific decoder이다. 이 구조는 generalist

Table 12.2: Claim-source-evidence-caution map for cross-domain and humanoid VLA

Claim	Source	Evidence	Caution
Humanoid VLA requires reasoning/action separation	GR00T N1 [13]	Vision-language module plus diffusion transformer action module	Check which components and datasets are open.
Industrial humanoid VLA is emerging	Helix [7]	Company report and demo-style evidence	Treat as company claim, not peer-reviewed benchmark evidence.
Closed VLA stacks need versioned naming	Gemini Robotics 1.5 [8]	Official page/report describes instruction and visual information to motor commands	Publicly verifiable evidence is limited.
Embodied reasoning and action generation are not identical	Gemini Robotics-ER 1.6 [9]	Official page/report frames ER as embodied reasoning	Do not merge ER claims with action policy benchmark claims.
Open-world generalization is a cross-domain target	$\pi_0, \pi_{0.5}$ [2, 17]	Flow VLA and heterogeneous co-training evidence	Emerging-front wording is safer than consensus wording.

backbone과 domain-specific controller를 분리하는 한 방법이다.

12.8 Transfer의 조건

Manipulation에서 배운 것이 navigation이나 driving으로 옮겨가려면 무엇이 transfer되는지 구분해야 한다. Transfer 가능한 것은 언어 이해, visual grounding, scene relation, temporal memory, planning pattern이다. Transfer가 어려운 것은 low-level action dynamics, safety constraint, sensor calibration, reward definition이다.

$$\theta = \theta_{\text{shared}} \cup \theta_{\text{domain}}. \quad (12.10)$$

좋은 multi-domain VLA 연구는 shared parameter가 어떤 능력을 담당하고, domain-specific parameter가 무엇을 담당하는지 ablation으로 보여야 한다. 모든 domain을 하나의 benchmark score로 합치면 해석이 흐려진다.

12.9 요약

VLA는 manipulation을 넘어 navigation, embodied QA, driving, games, humanoid로 확장될 수 있다. 그러나 확장의 핵심은 “같은 거대 모델을 어디에나 붙인다”가 아니라, vision-language grounding과 action generation의 공통 구조를 유지하면서 domain별 action space, state estimator, controller, safety constraint를 정확히 설계하는 것이다. Manipulation은 VLA의 대표 전장이지만, VLA의 최종 형태는 여러 embodiment가 공유하는 backbone과 domain-specific execution layer의 결합에 가까울 가능성이 크다.

다음 장에서는 2024–2026년 VLA 흐름을 연구 트렌드 맵으로 정리한다. 지금까지 다룬 action representation, fine-tuning, efficiency, manipulation, benchmark, reasoning, world model, safety,

embodiment 확장을 하나의 지형도로 묶는다.

Key papers

- Physical Intelligence et al. (2025), $\pi_{0.5}$: heterogeneous co-training, semantic prediction, web data를 결합해 open-world generalization을 전면에 놓는다.
- NVIDIA et al. (2025), GR00T N1: humanoid VLA와 dual-system architecture의 핵심 anchor이다.
- Figure AI (2025), Helix company report: industrial humanoid VLA claim을 공개 논문과 구분해 읽는 사례이다.
- Google DeepMind (2025/2026), Gemini Robotics: closed VLA, embodied reasoning, on-device model을 한 stack으로 분류할 수 있다.
- Autonomous driving VLA survey 계열 논문: action space와 safety constraint가 manipulation과 어떻게 달라지는지 확인하기 좋다.

Reading assignment [Intermediate]

GR00T N1, Helix, Gemini Robotics 중 하나를 골라 공개 논문, project/company report, closed product claim을 구분하고, 확인 가능한 evidence와 확인 불가능한 claim을 표로 나누라.

Chapter 13

2024–2026 VLA Trend Map

12장까지는 VLA를 구성 요소별로 보았다. 이 장에서는 같은 내용을 시간 축과 연구 축으로 다시 묶는다. 2024–2026년 VLA 흐름은 “VLM에 action head를 붙였다”는 단순한 이야기로 설명되지 않는다. 더 정확한 흐름은 pretrained VLM prior를 robot policy로 바꾸기 위해 action representation, robot data mixture, fine-tuning, efficient inference, manipulation evaluation, reasoning, world model, safety가 동시에 발전한 것이다.

13.1 큰 흐름

VLA 트렌드는 세 단계로 요약할 수 있다. 단, 2026년 항목은 확정된 합의라기보다 2025–2026년에 관측되는 emerging research fronts로 읽어야 한다.

1. 2024년: VLM 기반 robot policy가 실제 manipulation에서 작동할 수 있음을 보이는 단계.
2. 2025년: OpenVLA, Octo, RT 계열 이후 fine-tuning, action chunking, post-training, efficient VLA가 본격화되는 단계.
3. 2025–2026년 emerging fronts: reasoning, world model, safety, multi-embodiment, deployment constraint가 핵심 쟁점으로 떠오르는 단계.

이 구분은 엄밀한 연도 경계라기보다 연구 질문의 이동을 나타낸다. 초기 질문은 “된다/안 된다”였고, 중간 질문은 “더 잘/더 빨리/더 싸게 되는가”였고, 이후 질문은 “실제 환경에서 신뢰할 수 있는가”로 바뀐다.

13.2 Trend axes

VLA 연구는 다음 여덟 축으로 정리할 수 있다.

이 표의 축은 독립적이지 않다. 예를 들어 action chunking은 action representation이면서 efficiency 기법이고, reasoning은 benchmark 설계와 safety를 동시에 바꾼다. 좋은 trend analysis는 논문을 하나의 축에만 넣지 않고, 어느 축의 조합을 다루는지 본다.

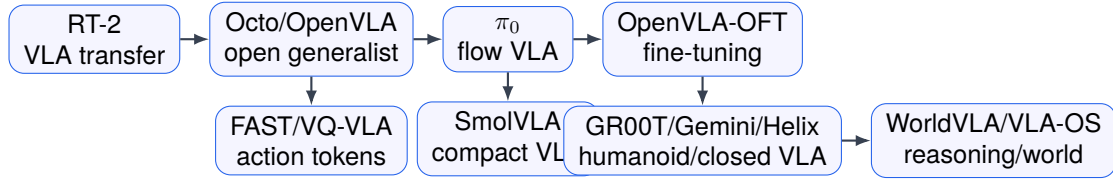


Figure 13.1: Author-created schematic. 2024–2026 VLA trend timeline. 이 도식은 publication priority가 아니라 연구 질문의 이동을 나타낸다: open generalist policy, flow/action representation, optimized fine-tuning, compact deployment, humanoid/closed industrial VLA, reasoning/world model.

Table 13.1: 2024–2026 VLA 주요 트렌드 축

축	핵심 질문
Scaling	더 큰 VLM과 더 큰 robot data가 policy 성능을 얼마나 올리는가
Action representation	continuous, discrete, diffusion, chunk 중 무엇이 실행 안정성을 높이는가
Post-training	imitation 이후 task-specific adaptation과 preference/success feedback을 어떻게 쓰는가
Efficiency	latency, memory, decoding step을 줄이면서 success를 유지할 수 있는가
Manipulation	contact-rich, long-horizon, dexterous task에서 실제로 작동하는가
Benchmark	offline, simulation, real robot metric이 같은 결론을 주는가
Reasoning/ world model	subgoal, uncertainty, rollout, recovery를 행동에 연결하는가
Safety/ deployment	OOD, failure, human intervention, safety envelope를 검증하는가

13.3 2024: generalist robot policy의 가능성

2024년 전후의 핵심 분위기는 robot foundation model의 가능성을 확인하는 것이었다. 대규모 vision-language pretraining이 물체, 속성, 관계, instruction understanding에 강한 prior를 제공하고, 여기에 robot trajectory를 fine-tuning하면 다양한 manipulation task를 수행할 수 있다는 방향이다.

$$\theta_{\text{VLA}} = \theta_{\text{VLM}} + \Delta\theta_{\text{robot}}. \quad (13.1)$$

이 시기 논문을 읽을 때 중요한 질문은 “VLM prior가 실제로 무엇을 도왔는가”이다. 언어 다양성인가, object recognition인가, unseen object grounding인가, 아니면 단순히 큰 backbone으로 인한 representation gain인가. 초기 성공은 중요하지만, evaluation이 제한된 경우도 많으므로 benchmark 조건을 함께 봐야 한다.

13.4 2025: fine-tuning과 효율성의 해

2025년의 큰 흐름은 VLA를 실제 과제에 맞게 조정하는 방법으로 이동한다. OpenVLA-style 모델을 특정 robot, 특정 task, 특정 action space에 맞게 fine-tuning하고, LoRA, adapter, OFT, action head 교체, data mixture tuning이 중요해진다. OpenVLA-OFT는 이 흐름의 명확한 anchor이다. 논문은 parallel decoding, action chunking, continuous action representation, L1 objective를 결합해 LIBERO 평균 success와 action generation throughput을 동시에 개선했다고 보고한다.

$$\min_{\Delta\theta} \mathcal{L}_{\text{robot}}(\theta_0 + \Delta\theta) \quad \text{s.t.} \quad \|\Delta\theta\| \leq \epsilon. \quad (13.2)$$

이 식은 full fine-tuning보다 parameter-efficient adaptation의 문제의식을 보여준다. Robot data는 비싸고 domain-specific하기 때문에, pretrained capability를 망가뜨리지 않으면서 필요한 행동만 조정하는 것이 중요하다.

동시에 efficient VLA가 부상한다. 실제 robot loop에서는 큰 모델이 느리면 사용할 수 없다.

$$\tau_{\text{sensor}} + \tau_{\text{model}} + \tau_{\text{decode}} + \tau_{\text{control}} \leq T_{\text{loop}}. \quad (13.3)$$

따라서 2025년 흐름은 “큰 모델이 된다”에서 “큰 모델을 어떻게 빠르게 쓰는가”로 이동한다. Token pruning, sparse expert, speculative decoding, diffusion step reduction, action chunking은 모두 이 방향의 도구이다. SmolVLA는 compact model과 asynchronous inference stack을 통해 이 문제를 system design 관점에서 다루고, FAST/VQ-VLA는 action tokenization 자체를 효율성과 실행 품질의 병목으로 다룬다.

13.5 Manipulation 중심성

Trend map에서 manipulation은 중앙에 놓인다. 이유는 단순하다. VLA의 주장 대부분이 manipulation에서 실제로 검증된다. Object grounding, affordance, action chunk, contact, safety, recovery가 한꺼번에 나타난다.

$$\text{VLA maturity} \not\approx \text{language score}; \quad \text{VLA maturity} \approx \text{closed-loop task success}. \quad (13.4)$$

Manipulation 논문을 볼 때는 task complexity를 반드시 구분해야 한다. Pick-and-place, articulated object, tool use, deformable object, dexterous hand, bimanual manipulation은 서로 다른 난이도이다. 같은 success rate라도 task class가 다르면 의미가 다르다.

13.6 Benchmark와 데이터셋의 정치성

VLA 트렌드에서 benchmark는 중립적인 측정 도구가 아니다. 어떤 benchmark가 유행하는지에 따라 연구 방향이 바뀐다. Offline action prediction benchmark가 중심이면 action token과 loss가 중요해지고, real-robot success benchmark가 중심이면 controller와 robustness가 중요해진다.

$$\text{Score} = f(\text{model}, \text{data}, \text{controller}, \text{protocol}). \quad (13.5)$$

따라서 trend를 읽을 때는 점수의 상승이 모델 때문인지, 데이터 때문인지, evaluation protocol 때문인지 분리해야 한다. 8장의 message를 다시 쓰면, VLA 분야의 SOTA는 항상 benchmark 조건부 SOTA이다.

Table 13.2: VLA 연구 지형의 네 quadrant

Quadrant	대표 질문
Model-centric VLA	backbone, architecture, action head, tokenization을 어떻게 설계하는가
Data-centric VLA	robot dataset, data mixture, annotation, synthetic data를 어떻게 구성하는가
System-centric VLA	latency, controller, safety layer, deployment stack을 어떻게 통합하는가
Evaluation-centric VLA	benchmark, OOD split, failure taxonomy, reproducibility를 어떻게 설계하는가

13.7 Reasoning과 world model의 재등장

초기 end-to-end VLA는 perception-to-action mapping을 강조했다. 그러나 long-horizon task, failure recovery, safety를 다루기 시작하면서 reasoning과 world model이 다시 중요해진다.

$$\pi(a_t \mid o_t, q) \rightarrow \pi(a_t \mid o_t, q, z_t, \hat{\tau}_{t:t+H}). \quad (13.6)$$

z_t 는 plan, belief, diagnosis이고, $\hat{\tau}$ 는 world model rollout이다. 이 전환은 classical robotics로의 회귀가 아니다. 오히려 VLM/VLA의 semantic prior와 robotics의 planning/control 구조를 결합하려는 움직임이다.

2025–2026년 emerging front에서 중요한 연구 질문은 reasoning을 어떻게 action-level improvement로 검증할 것인가이다. 좋은 reasoning 문장을 생성하는 것과 실제 recovery success를 높이는 것은 다르다.

13.8 Safety as first-class objective

VLA가 lab demo를 넘어가려면 safety가 부가 항목이 아니라 objective가 되어야 한다.

$$\max_{\pi} \mathbb{E}[R(\tau)] \quad \text{s.t.} \quad \Pr(C_i(\tau) > 0) \leq \delta_i. \quad (13.7)$$

이 constrained formulation은 2024년의 demo 중심 논문보다 2026년 이후 배포 지향 연구에서 더 중요하다. 특히 human-robot interaction, household robot, humanoid, driving에서는 safety envelope와 intervention protocol 없이는 VLA를 평가할 수 없다.

13.9 연구 지형도

VLA 연구 지형은 다음처럼 네 quadrant로 볼 수 있다.

좋은 연구 주제는 이 중 하나만 고르는 것이 아니라 두 축의 교차점에서 나온다. 예를 들어 efficient action tokenizer는 model-centric이면서 system-centric이고, failure-aware benchmark는 evaluation-centric이면서 safety-centric이다.

Table 13.3: Evidence-level legend for trend claims

Level	의미
A	Peer-reviewed 또는 accepted paper 이고 code, benchmark, real-robot evidence 중 일부가 재현 가능하다.
B	arXiv preprint와 project artifact가 있고 실험 protocol을 어느 정도 확인할 수 있다.
C	Project page, company report, closed demo 중심 source이다.
D	Survey-only 또는 claim-level source로, 핵심 성능 주장에는 보조 근거로만 쓴다.

Table 13.4: Open and academic VLA trend claims

Trend claim	Anchor sources	Level	읽을 때 확인할 것
Open generalist VLA	Octo, OpenVLA, Open X-Embodiment	B	Dataset scale, embodiment split, action interface, checkpoint/code 공개 범위.
Fine-tuning recipe	OpenVLA-OFT	B	같은 base model 통제, LIBERO/real robot 결과, throughput 정의.
Flow/action chunking	π_0 , OpenVLA-OFT	B	Flow objective, chunk length, latency-success curve, controller frequency.
Action tokenization	FAST, VQ-VLA	A/B	Token reconstruction, closed-loop success, high-frequency dexterity, tokenizer-only ablation.

13.10 요약

2024–2026년 VLA 트렌드는 VLM을 robot action으로 연결하는 초기 가능성에서 출발해, fine-tuning, efficiency, manipulation realism, benchmark rigor, reasoning, world model, safety, multi-embodiment로 확장되었다. 다음 단계의 핵심은 더 큰 모델을 만드는 것만이 아니라, 어떤 action interface가 안정적인지, 어떤 데이터가 일반화를 만드는지, 어떤 평가가 실제 배포 위험을 드러내는지 밝히는 것이다.

다음 장에서는 이 trend map을 바탕으로 좋은 VLA 연구 주제를 고르는 방법을 다룬다. 단순히 최신 모델을 따라가는 것이 아니라, 측정 가능한 병목과 검증 가능한 contribution을 선택하는 기준을 세운다.

Table 13.5: Emerging and industrial VLA trend claims

Trend claim	Anchor sources	Level	읽을 때 확인할 것
Compact deployment	SmolVLA	B	Consumer hardware 목표, async stack, task coverage, benchmark comparability.
Humanoid/ industrial VLA	GR00T N1, Helix, Gemini Robotics	B/C	Peer-reviewed 여부, 공개 범위, motor command interface, company demo 한계.
Reasoning/ world model	VLA-OS, WorldVLA, Gemini Robotics-ER	B/C	Reasoning text가 아니라 recovery, planning utility, safety metric 개선.
Safety/deployment	OpenVLA-OFT, GR00T N1, Gemini Robotics	B/C	Latency, safety envelope, intervention, logging, closed-stack governance.

Key papers

- Open X-Embodiment Collaboration (2023), project/paper: 1M+ real robot trajectories와 22 embodiments가 data-centric trend의 anchor이다.
- Octo Model Team et al. (2024), Octo, arXiv preprint: 800k trajectories 기반 open-source generalist policy이다.
- Kim et al. (2024), OpenVLA, arXiv preprint: 7B open-source VLA, 970k demonstrations, public checkpoints/code를 제공한다.
- Black et al. (2024/2026), π_0 , RSS 2025: VLM plus flow matching architecture가 general robot control의 anchor이다.
- Kim, Finn, and Liang (2025), OpenVLA-OFT, RSS 2025: fine-tuning, action chunking, continuous action, speed/success trend의 anchor이다.
- Shukor et al. (2025), SmolVLA: compact VLA와 affordable robotics trend의 anchor이다.
- NVIDIA et al. (2025), GR00T N1, and Google DeepMind Gemini Robotics page: humanoid/ closed industrial VLA를 분류하기 위한 anchor이다.

Reading assignment [Intermediate]

tables 13.4 and 13.5에서 한 trend claim을 골라, anchor source 두 편의 evidence가 충분한지 13-question framework의 검증/비교/한계 항목으로 평가하라.

Part V

Research Practice

Chapter 14

좋은 VLA 연구 주제를 고르는 법

13장에서는 VLA 연구 지형을 model, data, system, evaluation의 네 축으로 정리했다. 이 장은 그 지도를 연구 주제 선택 기준으로 바꾼다. 좋은 VLA 주제는 최신 모델을 가져와 새로운 task에 돌려보는 것만으로 만들어지지 않는다. 좋은 주제는 명확한 병목을 잡고, 그 병목이 왜 중요한지 설명하며, 방법이 그 병목을 직접 겨냥하고, 실험이 그 주장을 공정하게 검증할 때 만들어진다.

14.1 좋은 주제의 기본 조건

VLA 연구 주제는 다음 네 조건을 만족해야 한다.

1. 병목이 명확해야 한다.
2. contribution이 병목과 직접 연결되어야 한다.
3. 평가가 contribution을 분리해서 측정해야 한다.
4. 실패와 한계를 숨기지 않고 분석할 수 있어야 한다.

이를 간단한 score로 쓰면 다음과 같다.

$$Q_{\text{topic}} = w_b B + w_n N + w_v V + w_f F - w_c C. \quad (14.1)$$

B 는 bottleneck clarity, N 은 novelty, V 는 verifiability, F 는 feasibility, C 는 confound risk이다. 좋은 주제는 새로워 보이는 것보다 검증 가능한 것이 중요하다. 특히 VLA는 모델, 데이터, controller, benchmark가 얹혀 있어 confound가 크다.

14.2 피해야 할 주제 유형

가장 피해야 할 유형은 “최신 VLA를 우리 task에 적용했다”이다. 이 자체는 engineering report 일 수 있지만, 연구 contribution은 약하다. 또 다른 위험한 유형은 benchmark만 바꾸고 원인을 설명하지 않는 연구이다.

VLA 분야에서는 demo video가 강한 설득력을 갖지만, 논문 contribution을 대신하지는 못한다. Video는 qualitative evidence이고, 연구 주장은 controlled comparison으로 증명해야 한다.

Table 14.1: 약한 VLA 주제와 보강 방향

약한 주제	보강 방향
최신 모델 적용	실패 mode를 분석하고 그 mode를 줄이는 방법을 제안한다
데이터만 추가	어떤 data mixture가 어떤 일반화 축을 개선하는지 분리한다
새 benchmark 제안	baseline, split, failure taxonomy, reproducibility를 함께 제공한다
속도 개선 주장	success-latency-memory trade-off curve를 제시한다
reasoning 주장	reasoning trace가 아니라 recovery success와 safety 개선을 보인다
safety 주장	safety envelope와 risk metric을 실제 episode에서 검증한다

14.3 병목에서 시작하기

좋은 연구 질문은 모델 이름이 아니라 병목에서 시작한다. 예를 들어 다음은 모두 좋은 출발점이다.

- Action chunk가 길어질 때 접촉 단계에서 failure가 증가한다.
- Offline action loss는 낮아지지만 real robot success는 오르지 않는다.
- Language paraphrase에는 강하지만 spatial relation에는 약하다.
- Faster decoding은 평균 latency를 줄이지만 tail latency와 failure를 늘린다.
- VLA verifier가 unsafe instruction은 잡지만 scene-specific risk는 놓친다.

이런 병목은 측정 가능하다. 측정 가능한 병목은 method와 experiment를 자연스럽게 만든다.

$$\text{Claim} = \text{Bottleneck} + \text{Mechanism} + \text{Evidence}. \quad (14.2)$$

이 식은 논문 기획의 기본이다. 병목만 있으면 문제 제기이고, mechanism만 있으면 방법 소개이며, evidence가 없으면 주장이 아니다.

14.4 Contribution type 선택

VLA 논문의 contribution은 여러 유형이 있다.

학생 프로젝트에서는 architecture contribution보다 evaluation-centric 또는 system-centric contribution이 현실적일 때가 많다. 큰 foundation model을 처음부터 학습하기 어렵기 때문이다. 반대로 강한 실험실 infrastructure가 있다면 real-robot failure analysis와 robust fine-tuning이 좋은 주제가 될 수 있다.

14.5 Baseline을 먼저 설계하기

VLA 연구에서 baseline은 나중에 붙이는 비교 대상이 아니다. Baseline 설계가 주제의 품질을 결정한다. 좋은 baseline은 다음 조건을 갖는다.

1. 같은 data budget을 사용한다.
2. 같은 observation과 controller를 사용한다.

Table 14.2: VLA contribution 유형

유형	좋은 contribution의 조건
Architecture	기존 구조의 어떤 병목을 줄이는지 ablation으로 보여야 한다
Action representation	reconstruction이 아니라 closed-loop success와 failure mode를 보여야 한다
Fine-tuning method	data efficiency, forgetting, task adaptation을 분리해 측정해야 한다
Dataset	새로운 일반화 측과 evaluation protocol을 명확히 해야 한다
Benchmark	공정한 baseline, hidden split, failure taxonomy를 포함해야 한다
System	latency, controller, safety, real robot constraint를 end-to-end로 보고해야 한다
Theory/analysis	실제 VLA 실험 해석에 도움 되는 quantity를 제공해야 한다

3. 같은 evaluation split을 사용한다.

4. method가 주장하는 component만 다르다.

Ablation은 다음처럼 contribution을 분해한다.

$$\Delta J_{\text{method}} = J(\pi_{\text{ours}}) - J(\pi_{\text{baseline}}). \quad (14.3)$$

하지만 이 차이가 method 때문인지 data 때문인지 controller 때문인지 모르면 논문은 약해진다. 따라서 실험 설계의 목적은 높은 숫자를 얻는 것이 아니라 ΔJ 의 원인을 설명하는 것이다.

14.6 실험 규모와 현실성

좋은 주제는 feasible해야 한다. VLA는 대규모 GPU, robot hardware, data collection이 필요한 경우가 많다. 따라서 주제를 고를 때 resource budget을 명시해야 한다.

$$R_{\text{budget}} = (G, T_{\text{robot}}, N_{\text{episodes}}, H_{\text{human}}, D_{\text{storage}}). \quad (14.4)$$

G 는 GPU 자원, T_{robot} 은 robot 시간, N_{episodes} 는 실행 episode 수, H_{human} 은 annotation 또는 teleoperation 시간, D_{storage} 는 data 저장 비용이다. 좋은 연구 계획은 이 budget 안에서 검증 가능한 claim을 고른다.

실제 수업 프로젝트라면 다음 스케일이 현실적이다.

- Offline VLA 분석: 공개 dataset과 pretrained model을 사용해 action representation 또는 failure taxonomy를 분석한다.
- Small real-robot study: 3–5개 task, 3–5개 object, 20–50 episode 수준에서 명확한 hypothesis를 검증한다.
- Benchmark/tooling project: 기존 논문의 evaluation protocol을 재현하고 OOD split 또는 failure annotation을 추가한다.
- Efficient inference project: latency와 success trade-off를 동일 hardware에서 측정한다.

Table 14.3: 약한 제목과 강한 제목

약한 제목	강한 제목 방향
Applying VLA to Household Tasks	Failure-Aware Fine-Tuning for Long-Horizon Household Manipulation
Faster VLA Inference	Success-Preserving Action Chunking under Real-Time Robot Control Constraints
A New VLA Benchmark	An OOD and Failure-Taxonomy Benchmark for Language-Guided Manipulation
Reasoning for Robots	Grounded Replanning with Uncertainty-Aware Verifiers for VLA Policies
Multi-Robot VLA	Embodiment-Conditioned Action Decoding for Cross-Robot VLA Transfer

14.7 논문 제목으로 바꾸기

좋은 주제는 제목으로 바꿨을 때 contribution이 드러난다.

강한 제목은 task, 병목, 방법, 평가 측 중 최소 두 가지를 포함한다. 제목이 강하다고 논문이 강해지는 것은 아니지만, 좋은 제목으로 압축되지 않는 주제는 대개 contribution이 흐릿하다.

14.8 13-question framework로 주제 점검

3장에서 제시한 13-question framework는 논문 읽기뿐 아니라 주제 기획에도 쓸 수 있다. 아직 논문이 없는 상태에서는 각 항목을 질문으로 뒤집는다.

- 배경: 왜 지금 이 병목이 중요한가.
- 문제: 구체적으로 무엇이 실패하는가.
- 기존 한계: 기존 VLA가 왜 이 실패를 해결하지 못하는가.
- 목표: 어떤 metric을 얼마나 개선할 것인가.
- 방법: 어떤 mechanism으로 병목을 줄일 것인가.
- 검증: 어떤 dataset, robot, split, baseline으로 확인할 것인가.
- 한계: 어떤 조건에서는 여전히 실패할 것인가.

이 질문에 답하지 못하면 아직 연구 주제가 아니라 아이디어에 가깝다.

14.9 요약

좋은 VLA 연구 주제는 최신성을 따라가는 데서 나오지 않고, 명확한 병목과 검증 가능한 mechanism에서 나온다. Model-centric, data-centric, system-centric, evaluation-centric 중 어느 축에 있는지 정하고, baseline과 ablation을 먼저 설계하며, resource budget 안에서 증명 가능한 claim을 골라야 한다.

다음 장에서는 이 기준을 대표 논문 읽기에 적용한다. 모든 논문을 깊게 다룰 수는 없지만, VLA 연구를 처음 시작하는 학생이 반드시 읽어야 할 논문군을 유형별로 정리하고, 각 논문을 어떤 질문으로 읽어야 하는지 제시한다.

Key papers for topic selection

- OpenVLA-OFT (2025): Claim = fine-tuning bottleneck + optimized decoding/action representation mechanism + LIBERO/real robot evidence로 읽기 좋다.
- FAST (2025): Claim = action tokenization bottleneck + DCT compression mechanism + high-frequency dexterity evidence로 읽기 좋다.
- SmolVLA (2025): Claim = deployment cost bottleneck + compact/asynchronous system mechanism + affordable hardware evidence로 읽기 좋다.
- VLA-OS/WorldVLA (2025): Claim = planning/world-model bottleneck + structured intermediate representation + planning utility evidence로 평가해야 한다.
- GR00T N1/Gemini Robotics (2025/2026): Claim = humanoid or closed industrial VLA + dual-model/dual-system mechanism + 공개 evidence의 한계로 읽어야 한다.

Reading assignment [Advanced]

자신의 연구 아이디어를 Claim = Bottleneck + Mechanism + Evidence 형식으로 한 문장에 쓰고, 그 claim을 반박할 수 있는 baseline 두 개와 ablation 두 개를 설계하라.

Chapter 15

VLA 논문 읽기 실전

14장에서는 좋은 연구 주제를 고르는 기준을 세웠다. 마지막 장에서는 그 기준을 논문 읽기에 적용한다. 목표는 모든 논문을 같은 깊이로 요약하는 것이 아니라, VLA 연구자가 어떤 순서로 읽고 어떤 산출물을 남겨야 다음 연구 질문으로 이어지는지 정리하는 것이다. 전체 reading catalog는 [chapter A](#)에 두고, 본문에는 수업에서 바로 쓸 수 있는 읽기 경로와 worked example을 남긴다.

15.1 읽기 순서

VLA 논문은 최신순으로만 읽으면 구조가 잘 보이지 않는다. 먼저 foundation과 action representation을 읽고, 그 다음 open VLA backbone과 fine-tuning, efficiency, reasoning, benchmark를 읽는 것이 낫다.

Reading order = Foundation → Action → VLA → System → Evaluation. (15.1)

이 순서를 따르면 각 논문의 contribution을 “새 모델”이 아니라 “새 action interface”, “새 adaptation recipe”, “새 system constraint”, “새 evaluation protocol”로 분해할 수 있다.

15.2 4-week reading plan

대학원 수업이나 랩 세미나에서는 다음 4주 계획을 추천한다. 각 주의 산출물은 짧지만, 모든 논문에 대해 같은 형식을 요구해야 비교가 가능하다.

15.3 Worked example 1: OpenVLA-OFT 13-question 적용

OpenVLA-OFT는 같은 base model에서 fine-tuning recipe가 success와 throughput을 어떻게 바꾸는지 보여 주는 좋은 실전 예제이다 [11]. 다음 표는 논문을 수업에서 빠르게 해부하기 위한 압축형 13-question 적용이다.

Table 15.1: Compact reading path for a 2024–2026 VLA course

Stage	Papers	Expected output
Foundation	SayCan, Diffusion Policy, ACT/ALOHA	Pre-VLA와 VLA의 차이를 controller interface 관점에서 설명한다.
Open VLA	RT-2, Octo, OpenVLA, π_0	입력, 출력, action space, 학습 데이터, 공개 자원을 표로 비교한다.
Adaptation/action	OpenVLA-OFT, FAST, VQ-VLA, SmolVLA	success, latency, action representation trade-off를 분해한다.
Emerging front	GR00T N1, Gemini Robotics, WorldVLA/VLA-OS, RoboTwin 2.0	evidence level과 확인 불가능한 claim을 분리한다.

Table 15.2: Four-week VLA reading plan

Week	Papers	Output
1	SayCan, Diffusion Policy, ACT/ALOHA	Pre-VLA predecessor와 VLA의 경계를 1쪽으로 정리한다.
2	RT-2, Octo, OpenVLA, π_0	각 모델의 input-output interface와 action representation 표를 만든다.
3	OpenVLA-OFT, FAST, VQ-VLA, SmolVLA	action/ latency/ success trade-off를 claim-source-evidence-caution 표로 쓴다.
4	GR00T N1, Gemini Robotics, WorldVLA, VLA-OS, RoboTwin 2.0	evidence ladder를 적용해 peer-reviewed evidence와 company/preprint claim을 분리한다.

15.4 Worked example 2: WorldVLA/VLA-OS 13-question 적용

Reasoning과 world model 논문은 자칫 그럴듯한 개념 설명으로 끝나기 쉽다. WorldVLA와 VLA-OS는 “미래 예측이나 planning representation이 실제 action-level utility를 높이는가”라는 질문으로 읽어야 한다 [20, 22].

15.5 Worked example 3: Humanoid and closed industrial VLA evidence split

GR00T N1, Helix, Gemini Robotics를 한 표에 놓으면 VLA의 최신 흐름은 잘 보이지만, evidence level을 구분하지 않으면 hype와 검증이 섞인다. 이 worked example의 목적은 open arXiv source, company report, closed VLA, embodied reasoning model을 같은 기준으로 분리하는 것이다 [7–9, 13].

15.6 논문 읽기 노트 템플릿

매일 여러 편을 읽을 때는 13-question 전체를 항상 쓰기보다 짧은 note template으로 시작하는 것이 효율적이다.

- 이 논문의 정확한 병목:

Table 15.3: OpenVLA-OFT 13-question worked example

No.	항목	요약
01	배경	OpenVLA-style open VLA를 실제 manipulation benchmark와 lab robot에 맞추려면 pretrained backbone만으로는 부족하다.
02	문제	기존 OpenVLA fine-tuning은 성공률과 action generation throughput 모두에서 deployment 요구를 만족하기 어렵다.
03	기존 한계	Autoregressive action token decoding과 discrete action 표현은 느리고, continuous control에 필요한 정밀도를 잃을 수 있다.
04	목표	같은 base model을 유지하면서 LIBERO success와 action generation speed를 동시에 개선하는 fine-tuning recipe를 찾는다.
05	방법	Parallel decoding, action chunking, continuous action representation, L1 objective를 결합한 OpenVLA-OFT recipe를 구성한다.
06	핵심 아이디어	VLA 성능은 backbone보다 action interface와 fine-tuning objective가 지배할 수 있으므로, output side를 재설계한다.
07	검증	LIBERO suites와 real-world robot setup에서 기존 OpenVLA-style baseline과 recipe ablation을 비교한다.
08	결과	논문은 LIBERO 평균 success를 76.5%에서 97.1%로 올리고 action generation throughput을 $26\times$ 개선했다고 보고한다.
09	비교	비교의 핵심은 같은 base model 통제 아래 action representation, decoding, objective를 바꾼 ablation을 보는 것이다.
10	의의	Open VLA를 실제 lab task에 맞추는 병목이 단순 model size가 아니라 action decoding과 adaptation recipe임을 보여 준다.
11	한계	결과는 benchmark protocol, hardware, controller, LIBERO split 조건에 묶이므로 다른 robot deployment로 바로 일반화하면 안 된다.
12	향후 과제	Real-time safety layer, broader robot platforms, failure recovery data, cross-benchmark transfer로 recipe의 범위를 검증해야 한다.
13	자원 공개	OpenVLA의 공개 backbone과 함께 읽되, OFT의 code/checkpoint 공개 범위와 재현 가능한 evaluation script를 별도로 확인해야 한다.

- 제안한 mechanism:
- 가장 믿을 만한 evidence:
- 가장 약한 baseline 또는 confound:
- 내 연구로 가져올 수 있는 지점:
- 실제 deployment에서 남는 위험:

이 template은 13-question summary보다 짧다. 중요한 논문만 13-question으로 확장하고, 나머지는 이 노트로 cataloging하면 field map을 빠르게 만들 수 있다.

15.7 Assignment difficulty tags

각 장의 reading assignment는 다음 난이도 태그 중 하나로 읽는다. 수업에서는 Basic을 개인 과제로, Intermediate를 세미나 토론 과제로, Advanced와 Capstone을 프로젝트 제안서로 연결하면 좋다.

15.8 VLA 연구자가 가져가야 할 7개 원칙

이 책의 마지막 결론은 다음 일곱 문장으로 요약할 수 있다.

Table 15.4: WorldVLA/VLA-OS 13-question worked example

No.	항목	요약
01	배경	Long-horizon VLA는 현재 관측에서 바로 action을 내는 reactive policy만으로는 planning, recovery, safety를 충분히 다루기 어렵다.
02	문제	World model 또는 planning representation이 실제 행동 선택에 도움을 주는지, 단순 text reasoning이나 pixel prediction을 넘어 검증해야 한다.
03	기존 한계	많은 reasoning-style VLA는 중간 문장이나 plan의 자연스러움은 보이지만 closed-loop utility와 failure recovery evidence가 약하다.
04	목표	Action-conditioned future prediction 또는 structured planning representation이 candidate ranking, recovery, task success를 개선하는지 확인한다.
05	방법	WorldVLA는 action과 visual generation/understanding을 묶는 autoregressive world model 방향이고, VLA-OS는 planning representation과 paradigm을 구조적으로 비교한다.
06	핵심 아이디어	Reasoning의 가치는 설명문이 아니라 action 후보를 더 잘 고르고 실패를 더 빨리 감지하게 만드는 중간 변수에서 나온다.
07	검증	Prediction metric, candidate ranking, planning ablation, closed-loop 또는 benchmark task success를 분리해 확인해야 한다.
08	결과	핵심 결과는 pixel score가 아니라 world model 또는 planning representation을 넣었을 때 success, recovery, safety metric이 좋아지는지이다.
09	비교	No-world-model, no-planning, direct policy, text-only reasoning baseline과의 통제 비교가 필요하다.
10	의의	검증이 충분하면 VLA를 reactive policy에서 prediction-aware policy로 확장하는 근거가 된다.
11	한계	미래 예측은 contact dynamics, occlusion, long horizon에서 빠르게 틀어질 수 있고, rollout latency가 control loop를 망칠 수 있다.
12	향후 과제	Short-horizon verification, uncertainty-aware replanning, real-robot recovery benchmark, safety-critical counterfactual test가 필요하다.
13	자원 공개	Preprint/project report의 공개 코드, benchmark script, model checkpoint, evaluation protocol을 확인하고 없으면 claim-level evidence로 낮춰 읽는다.

1. 모델명보다 interface를 먼저 보라.
2. Action representation은 구현 세부사항이 아니라 핵심 contribution이다.
3. Success rate는 benchmark tuple에 조건부인 숫자다.
4. Real-time constraint 없이는 robot policy claim이 달하지 않는다.
5. Reasoning은 긴 text가 아니라 recovery, uncertainty, safety로 검증하라.
6. Closed industrial claim은 evidence ladder로 분리하라.
7. 좋은 VLA 연구 주제는 bottleneck, mechanism, evidence가 한 문장으로 연결될 때 나온다.

VLA 분야는 빠르게 변하지만, 좋은 논문을 읽는 기준은 크게 변하지 않는다. 입력, 출력, action space, controller interface, learning signal, evaluation protocol, source status를 확인하면 hype와 실제 engineering contribution을 분리할 수 있다. 이 책의 목적도 바로 그 분리 능력을 만드는 데 있다.

Final assignment [Capstone]

관심 있는 VLA 논문 한 편을 골라 6-line note, 13-question summary, claim-source-evidence-caution table을 모두 작성하고, 마지막에 그 논문이 새로운 연구 주제로 이어지는 병목 하나를 제안하라.

Table 15.5: Evidence-ladder worked example for humanoid and industrial VLA

Source	Level	What the source claims	How to read it
GR00T N1	B	Humanoid foundation model with vision-language module and diffusion transformer motor-action module.	Open foundation claim은 중요하지만, 공개 artifact, data mixture, simulation/real split, humanoid platform 조건을 따로 확인한다.
Helix	C	Generalist humanoid VLA for full-upper-body control in an industrial product setting.	Company report/demo evidence로 분류하고, benchmark score나 reproducible paper evidence처럼 비교하지 않는다.
Gemini Robotics 1.5	C	Closed industrial VLA that maps visual information and instruction to robot motor commands.	Closed model이므로 model weight, controller, data, safety layer의 비공개 범위를 먼저 표시한다.
Gemini Robotics-ER 1.6	C	Embodied reasoning model for robotics rather than the same object as the action VLA.	Reasoning model과 motor policy를 구분하고, reasoning claim이 recovery/safety/action metric으로 달하는지 본다.
Reviewer question	–	이 source가 field direction을 보여 주는가, 아니면 reproducible technical evidence를 제공하는가?	Trend discussion에는 쓸 수 있지만, SOTA claim에는 evidence level을 명시해야 한다.

Table 15.6: Assignment difficulty tags used in the book

Tag	Expected work
Basic	13-question summary, input-output interface, key claim extraction처럼 논문 독해의 기본 구조를 채운다.
Intermediate	Action space, benchmark tuple, claim-source-evidence-caution table처럼 비교 가능한 분석 표를 만든다.
Advanced	Ablation, failure taxonomy, safety checklist, deployment readiness처럼 논문 claim을 재설계하거나 검증 protocol을 제안한다.
Capstone	여러 source의 evidence level을 비교해 새로운 VLA 연구 병목과 실험 계획을 제안한다.

Part VI

Appendices

Chapter A

Annotated Reading Catalog

이 부록은 References와 역할이 다르다. References는 citation-only 목록이고, 이 catalog는 source status, evidence type, artifact, caution을 한눈에 보기 위한 annotated reading guide이다. Catalog는 완전한 survey가 아니라 Ch15의 4-week reading plan을 지원하는 실전용 점검표이다.

A.1 Foundation and Open VLA Sources

A.2 Emerging and Industrial VLA Sources

이 두 표는 source의 존재와 source의 claim이 검증되었는지를 분리하기 위한 장치이다. 특히 emerging-front와 company report는 field direction을 보여 주는 신호로는 중요하지만, 공개 benchmark evidence와 같은 무게로 읽으면 안 된다.

Table A.1: Foundation and open VLA reading catalog

Source	Status	Evidence	Usage and caution
SayCan [1]	arXiv	Real robot	Pre-VLA planning anchor; end-to-end VLA로 분류하지 않는다.
RT-1 [3]	arXiv	Real robot	Real-world robot transformer scaling의 선행 흐름으로 읽는다.
RT-2 [4]	arXiv/closed	Real robot	Web knowledge transfer anchor; closed-stack 조건을 분리한다.
PaLM-E [6]	arXiv	Embodied tasks	Embodied VLM antecedent; action policy evidence와 구분한다.
Diffusion Policy [5]	Peer-reviewed	Real/sim robot	Diffusion action decoder predecessor; VLA backbone 효과와 분리한다.
ACT/ALOHA [23]	Peer-reviewed	Real robot	Action chunking and bimanual imitation anchor; language grounding은 제한적이다.
Open Embodiment [15]	X- arXiv/project	Multi-site real robot	Dataset scaling anchor; action normalization and mixture confound를 확인한다.
LIBERO [12]	arXiv/project	Simulation benchmark	Language-conditioned manipulation benchmark; sim-to-real claim은 따로 둔다.
Octo [14]	arXiv	Offline/real robot	Open generalist policy; goal image/language interface와 split을 확인한다.
OpenVLA [10]	arXiv	Real robot/benchmark	Open VLA backbone; controller, split, public checkpoint 범위를 함께 본다.
π_0 [2]	arXiv/RSS	Real robot	Flow VLA anchor; real-time claim은 horizon/hardware 조건과 함께 읽는다.
OpenVLA-OFT [11]	arXiv/RSS	LIBERO/robot	Fine-tuning recipe anchor; success 와 throughput의 protocol fairness를 확인한다.
FAST [16]	arXiv	Offline/robot	DCT action tokenization anchor; tokenizer-only effect를 분리한다.
VQ-VLA [21]	Peer-reviewed	Tokenizer eval	VQ tokenizer scaling anchor; reconstruction 과 closed-loop success를 분리한다.

Table A.2: Emerging and industrial VLA reading catalog

Source	Status	Evidence	Usage and caution
$\pi_{0.5}$ [17]	arXiv	Real/co-training	Open-world generalization front; emerging-front wording을 유지한다.
SmolVLA [19]	arXiv	Deployment/robot	Consumer GPU/CPU deployment를 목표로 async stack을 제시한다; task coverage와 hardware setting을 확인한다.
GR00T N1 [13]	arXiv	Humanoid real/sim	Dual-system humanoid VLA anchor; 공개 artifact와 dataset mixture 범위를 확인한다.
Helix [7]	Company report	Company demo	Industrial humanoid VLA claim; peer-reviewed benchmark evidence처럼 쓰지 않는다.
Gemini Robotics 1.5 [8]	Company report	Company demo/report	Closed industrial VLA; motor command interface와 공개 검증 한계를 분리한다.
Gemini Robotics-ER 1.6 [9]	Company report	Embodied reasoning report	VLA action model이 아니라 embodied reasoning model로 구분한다.
WorldVLA [22]	Preprint/report	Reported model eval	Pixel quality보다 planning utility와 candidate ranking evidence를 본다.
VLA-OS [20]	arXiv	Planning comparison	Planning representation anchor; action-level success evidence를 확인한다.
RoboTwin 2.0 [18]	arXiv/project	Simulation benchmark	Bimanual simulation benchmark; real deployment validity는 분리한다.

Chapter B

Evidence Ladder

VLA 논문은 peer-reviewed paper, arXiv preprint, project page, company report, closed product announcement가 섞여 있다. 같은 문단에서 모두 인용할 수는 있지만, 같은 신뢰도로 읽으면 안 된다. 이 책은 다음 evidence ladder를 사용한다.

Table B.1: Evidence level used in this book.

Level	Meaning	How to use it
A	Peer-reviewed 또는 accepted paper 이고, code/ benchmark/ real-robot evidence 중 적어도 하나가 비교적 재현 가능하다.	본문 claim의 강한 anchor로 쓸 수 있지만, controller, split, hardware 조건은 여전히 확인한다.
B	arXiv preprint 이거나 project artifact가 있으며, 실험 protocol과 일부 artifact가 공개되어 있다.	Emerging result의 주요 근거로 쓸 수 있으나, peer review 전 claim과 artifact completeness를 구분한다.
C	Project page, company report, closed model report, demo video 중심 source이다.	Industrial trend 또는 product direction을 설명하는 데 쓰되, 검증된 학술 evidence처럼 쓰지 않는다.
D	Survey-only, blog-only, claim-level source 이거나 원문 artifact 접근이 제한적이다.	배경 신호로만 사용하고, 핵심 성능 claim의 직접 근거로 쓰지 않는다.

Company claim handling

Closed industrial VLA는 field direction을 이해하는 데 중요하다. 그러나 Helix, Gemini Robotics, Gemini Robotics-ER 같은 source는 public reproducibility가 제한될 수 있다. 따라서 본문에서는 다음 규칙을 따른다.

1. Company report는 “관측되는 산업 방향”의 근거로 쓰고, 일반 SOTA 성능의 확정 근거로 쓰지 않는다.

2. 공개되지 않은 model weight, data, controller, safety layer는 “확인되지 않음”으로 남긴다.
3. Demo success는 benchmark success와 분리한다.
4. Closed VLA action model과 embodied reasoning model은 versioned naming으로 구분한다.

이 ladder는 reviewer가 가장 먼저 묻는 질문, 즉 “이 주장의 근거가 무엇인가”에 답하기 위한 장치이다.

Chapter C

Glossary

이 부록은 본문에서 반복되는 용어를 고정한다. 한국어 번역이 어색한 용어는 영어를 기본 표기로 유지하고, 처음 등장할 때만 짧게 설명한다.

Table C.1: Core terminology for the VLA book.

Term	Definition
VLA	Vision, language, robot observation/history를 조건으로 실제 action 또는 action chunk를 출력하는 language-conditioned closed-loop policy.
fine-tuning	Pretrained VLA를 특정 robot, task, action space, controller interface에 맞추는 supervised adaptation.
post-training	fine-tuning 이후 success feedback, verifier, preference, correction signal, simulator feedback으로 실행 품질을 보정하는 절차.
action space	VLA가 출력하는 행동 변수의 공간; end-effector delta, joint velocity, gripper command, token, trajectory 등이 포함된다.
action chunk	한 번의 model inference에서 생성되는 여러 timestep의 action sequence.
action chunking	action chunk를 사용해 model call frequency를 낮추고 temporal consistency를 높이려는 기법.
controller interface	VLA output이 low-level controller로 전달될 때의 coordinate frame, frequency, normalization, constraint, safety filter 계약.
latency budget	sensing, preprocessing, model inference, decoding, safety filter, controller dispatch를 모두 포함한 control-loop 시간 예산.
source status	paper/source의 공개와 검증 성격; peer-reviewed, arXiv, project page, company report, closed model 등으로 구분한다.
evidence type	source가 제공하는 검증 유형; offline, simulation, real robot, multi-site, company demo 등으로 구분한다.
engineering utility	실제 논문 objective가 아니라 success, latency, memory, risk를 비교하기 위한 개념적 판단 함수.
diagnostic metric	benchmark 나 failure analysis를 위해 만든 측정치; training loss와 다를 수 있다.
emerging front	아직 합의된 trend가 아니라 2025–2026년에 빠르게 관측되는 연구 전선.

Bibliographic Note

이 원고는 2024–2026년 VLA 및 robot foundation model 관련 논문군을 기반으로 구성된 강의용 책이다. 상위 폴더의 01_pdfs, 02_summaries, 03_citation_survey에는 원문 PDF, 13-question 요약, Google Scholar 인용 조사 결과를 분리해 두었으며, 본문은 이를 수업용 개념 구조와 연구 주제 선정 절차로 재구성한 것이다. 각 장의 key papers는 장별 reading anchor이고, full metadata와 source status는 References와 Appendix의 reading catalog에 정리했다.

Bibliography

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, et al. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL <https://arxiv.org/abs/2204.01691>. arXiv preprint.
- [2] Kevin Black, Noah Brown, Danny Driess, Amir Esmail, Monica Equi, Chelsea Finn, Pete Florence, Shixiang Gu, et al. A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>. arXiv preprint.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, et al. Rt-1: Robotics transformer for real-world control at scale, 2022. URL <https://arxiv.org/abs/2212.06817>. arXiv preprint.
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>. arXiv preprint.
- [5] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023. URL <https://arxiv.org/abs/2303.04137>.
- [6] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, et al. Palm-e: An embodied multimodal language model, 2023. URL <https://arxiv.org/abs/2303.03378>. arXiv preprint.
- [7] Figure AI. Helix: A vision-language-action model for generalist humanoid control, 2025. URL <https://www.figure.ai/news/helix>. Company report.
- [8] Google DeepMind. Gemini robotics 1.5, 2025. URL <https://deepmind.google/models/gemini-robotics/>. Technical report and project page.
- [9] Google DeepMind. Gemini robotics-er 1.6, 2026. URL <https://deepmind.google/models/gemini-robotics/>. Technical report and project page.

-
- [10] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, et al. Openvla: An open-source vision-language-action model, 2024. URL <https://arxiv.org/abs/2406.09246>. arXiv preprint.
 - [11] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success, 2025. URL <https://arxiv.org/abs/2502.19645>. arXiv preprint.
 - [12] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023. URL <https://arxiv.org/abs/2306.03310>. arXiv preprint.
 - [13] NVIDIA et al. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>. arXiv preprint.
 - [14] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, et al. Octo: An open-source generalist robot policy, 2024. URL <https://arxiv.org/abs/2405.12213>. arXiv preprint.
 - [15] Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and rt-x models, 2023. URL <https://arxiv.org/abs/2310.08864>. arXiv preprint and project page.
 - [16] Karl Pertsch, Kevin Black, Noah Brown, Dibya Ghosh, Brian Ichter, Sergey Levine, and Chelsea Finn. Fast: Efficient action tokenization for vision-language-action models, 2025. URL <https://arxiv.org/abs/2501.09747>. arXiv preprint.
 - [17] Physical Intelligence. $\pi_{0.5}$: A vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>. arXiv preprint.
 - [18] RoboTwin 2.0 Authors. Robotwin 2.0: A scalable data generator and benchmark for bimanual robotic manipulation, 2025. arXiv preprint and project page.
 - [19] Mustafa Shukor et al. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025. URL <https://arxiv.org/abs/2506.01844>. arXiv preprint.
 - [20] VLA-OS Authors. Vla-os: Structuring and dissecting planning representations and paradigms in vision-language-action models, 2025. URL <https://arxiv.org/abs/2506.17561>. arXiv preprint.
 - [21] Yue Wang et al. Vq-vla: Improving vision-language-action models via scaling vector-quantized action tokenizers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. URL https://openaccess.thecvf.com/content/ICCV2025/papers/Wang_VQ-VLA_Improving_Vision-Language-Action_Models_via_Scaling_Vector-Quantized_Action_Tokenizers_ICCV_2025_paper.pdf.
 - [22] WorldVLA Authors. Worldvla: Towards autoregressive action world model, 2025. URL <https://aiskyeye.com/wp-content/uploads/2025/11/WorldVLA.pdf>. Preprint.

- [23] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023. URL <https://arxiv.org/abs/2304.13705>.